

Interface

Das Resource Construction Kit



Interface erlaubt die Erstellung und Bearbeitung von Resource-Dateien also Menüs, Dialogformulare, Alerts, Icons, usw.

Durch seine einfache Bedienung und einen mächtigen Funktionsumfang ist Interface seit Jahren der Standard für Resource-Construction-Kits.

- ◆ 100% GEM, läuft unter allen TOS Systemen (ST/TT/Falcon/Medusa)
- ◆ Unterstützung von Multitasking durch abschaltbares Desktop, Drag&Drop etc.
- ◆ Benutzung des GEM Clipboards; Cut/Copy/Paste von Text und Grafik
- ◆ Fensterdialoge und übersichtliches Design der Dialogboxen
- ◆ Die Resource-Größe ist (fast) unbegrenzt
- ◆ Sichern der Resource-Dateien in den Formaten RSD, DEF, DFN und HRI zum Einbinden in eigene Programme wird auch RSH unterstützt

Programmiersprachen – Über die Grenzen hinaus

- ◆ Export-Formate für C, Pascal, Modula 2, Basic und GFA-Basic
- ◆ Direkte Einbindung von Resource-Dateien in eigene Programme
- ◆ Im Testmodus können eigene Dialog-Routinen eingebunden werden

Der Icon-Editor

- ◆ Icons in 2,4,16 und 256 Farben erstellbar
- ◆ Getrenntes Editieren von Maske und normalem/selektiertem Icon
- ◆ Import von Windows-Icons sowie IMG-Grafiken
- ◆ Bearbeitungsfunktionen: Scrollen, Zentrieren, Spiegeln, Cut/Copy/Paste
- ◆ Zeichenfunktionen: Linie, Punkt, Rechteck, Kreis, Füllen
- ◆ Zusammenstellen von Desktop-Icon-Dateien (ATARI, Gemini, no Desk)

Moderne Dialogformulare leicht gemacht – Die MyDials

Zusammen mit Interface erhalten Sie die MyDials, eine komplette C Library für „Fliegende Dialoge“. Sie ermöglichen eine Einbindung der Dialog Routinen aus Interface in eigene Programme.

- ◆ „Fliegende Dialoge“, PopUps, erweiterte Objekttypen für übersichtliche Gestaltung der Dialogboxen, wie z.B. Titelboxen und Umrahmungen
- ◆ Erweiterte Editiermöglichkeiten für Text bis zu einer Länge von 256 Zeichen
- ◆ Abschaltbare 3D-Darstellung im modernen „Grau-Look“, sowie unterstützte Funktionen für Fensterdialoge
- ◆ Komplette Tastaturbedienbarkeit für eigene Programme
- ◆ Fertige Dialoge können gleich in Interface im Testmodus überprüft werden

Interface

Der Resource Editor

Autor: Olaf Meisiek

2. Auflage 1994

Vertrieb: no! Software GmbH
Ritzstr. 13
D - 54595 Prüm

Tel. (0 65 51) 95 55 - 0 1
Fax (0 65 51) 95 55 - 15 5

Copyright

Kein Teil der Software oder des Handbuchs darf ohne schriftliche Genehmigung von no| Software GmbH in irgendeiner Form reproduziert und weitergegeben werden. Interface ist urheberrechtlich geschützt.

Gewährleistungen

Die Software und das Handbuch wurden mit größter Sorgfalt erstellt. Nur bei Programmfehlern, die ein Arbeiten mit Interface unmöglich machen, leistet no| Software GmbH nach Rücksendung der Systemdisketten kostenfreien Ersatz.

Haftungsausschluß

no| Software GmbH übernimmt keine Haftung für Folgen, die auf Programmfehler oder inkorrekte Angaben im Handbuch zurückzuführen sind.

Die Wiedergabe von Warenbezeichnungen, Handelsnamen oder sonstigen Kennzeichen in diesem Buch berechtigt nicht zu der Annahme, daß diese von jedermann frei genutzt werden dürfen. Es kann sich auch dann um eingetragene Warenzeichen oder sonstige geschützte Kennzeichen handeln, wenn sie nicht als solche gekennzeichnet sind.

Druck: Druckerei Hoffmann, Neuerburg

Inhalt

1 Einführung	1
Rück- und Ausblick	1
Übersicht	1
Umgang mit dem Handbuch	3
Registrierkarte	3
Das sollten Sie unbedingt lesen	4
2 Arbeiten mit Interface	5
Interface installieren	5
Allgemeines	5
Sprachgebrauch in diesem Handbuch	7
Symbole auf dem Desktop	9
Öffnen von Dateien	11
Das Menü	11
Dialogboxen	12
Die Maus	16
Fenster	17
Nichts geht mehr - die Speicherverwaltung	17
Eine einfache Übung	17
Eine neue Resource entsteht	18
Im Detail	27
3 Referenz	51
Das ›Datei‹-Menü	51
Das ›Bearbeiten‹-Menü	54
Das ›Fenster‹-Menü	57
Das ›Objekt‹-Menü	57
Das ›Optionen‹-Menü	58
4 Referenz (Iconeditor)	65
Das ›Datei‹-Menü	65
Das ›Bearbeiten‹-Menü	67
Das ›Raster‹-Menü	68
Das ›Maske‹-Menü	68
Das ›Farbe‹-Menü	68

5 Interface anpassen	77
Tastaturanpassung	77
Anpassung von Dateiextensions	78
Änderungen für andere Sprachen	78
Ändern der Alerts	79
6 Ein EXTODFIX-Programms erstellen	81
Allgemeines	81
Grundsätzlicher Aufbau	81
7 Interface »ferngesteuert«	84
8 Ein Resource-Programm als Beispiel	85
9 Die MyDials	88
Allgemeines	88
Funktionsweise der MyDials	90
Die erweiterten Objekttypen	90
Programmierung mit den MyDials	94
Ablaufschema	94
Ein Beispielprogramm	94
Die Funktion der einzelnen MyDial-Routinen	96
Diverse nützliche Hilfsroutinen der MyDials	113
Ein Beispielprogramm für die Dialogbearbeitung	118
Dialoge in Fenster legen	119
Ressourcen von mehr als 64 KB Größe benutzen	124
Farbicon-Unterstützung	125
C-Source Ressourcen	129
Das AV-Protokoll	130
10 Die Dateiauswahl von Interface	132
Das Dateifenster	133
Dateien auswählen	133
Spezielle Funktionen	136
Tastatursteuerung	139
Index	141

Einführung

Rück- und Ausblick

Nachdem Interface auf der ATARI-Messe 1991 das Licht der Welt erblickte, haben uns eine Menge Verbesserungswünsche erreicht. In der aktuellen Version haben wir versucht, davon so viel wie möglich zu verwirklichen. Deshalb hier nochmals unser Dank an alle, die nicht locker gelassen haben und es geschafft haben, uns - und vor allem Olaf Meisiek, den Programmierer von Interface - von ihren Änderungswünschen zu überzeugen.

Wir haben das Programm weitestmöglich an die neuen Multitasking-Betriebssysteme wie Mag!X, MultiGEM und MultiTOS angepaßt.

Wenn Sie Anregungen zur Verbesserung von Interface haben, die von allgemeinem Interesse sind, würden wir uns über eine entsprechende Nachricht freuen.

Übersicht

Dieses Handbuch soll Sie in die Lage versetzen, die vielfältigen Funktionen, die Interface Ihnen bietet, für Ihre Zwecke zu nutzen. Zwar wurde schon bei der Entwicklung des Programms großer Wert auf einfache und durchgängige Bedienung gelegt, aber zur Beherrschung aller Funktionen ist eine kurze Einarbeitung notwendig. Mit dem vorliegenden Handbuch soll Ihnen das Konzept und die Bedienung des Programms in allen Einzelheiten erläutert werden. Es gibt folgende Abschnitte:

Kapitel 1

ist die Einführung, die Sie gerade lesen.

Kapitel 2

Arbeiten mit Interface, erklärt im ersten Abschnitt, was eine Resource ist, beschreibt die Arbeitsfläche von Interface und erläutert die besonderen Funktionen der Maus und der Dialoge von Interface. Der zweite Abschnitt macht Sie mit den wichtigsten Fachausdrücken bekannt, und zeigt, wie Sie ein einfaches Menü und einen Dialog aufbauen und gestalten können. Der dritte Abschnitt geht dann ins Detail und erläutert die Feinheiten bei der Gestaltung einer Resource. Abschließend wird der Iconeditor erklärt.

Kapitel 3

ist der Referenzteil, in dem die Menü-Optionen zum Nachschlagen zusammengefaßt erläutert werden.

Kapitel 4

ist der Referenzteil zum Iconeditor zum Nachschlagen.

Kapitel 5

erklärt, wie Sie Interface an Ihre eigenen Bedürfnisse anpassen können (Tastaturbelegung, Headerdatei, Dateinamen-Extensions).

Kapitel 6

erläutert, wie Sie ein eigenes »EXTOBFIX«-Programm für Ihre erweiterten Objekttypen programmieren können.

Kapitel 7

Interface ferngesteuert, verweist auf eine weitere Möglichkeit, Interface zu erweitern: die Accessory-Pipeline.

Kapitel 8

zeigt ein kleines C-Programm, das eine Resource lädt, einen Dialog anzeigt und auswertet.

Kapitel 9

dokumentiert die MyDials, ein System mit dem Sie in Ihren Programmen »fliegende Dialoge«, Popups, Tastatursteuerung in Dialogen, Farbicons und ähnliches zur Verfügung stellen können. Außerdem werden dort die »XRSRC«-Routinen erläutert, die Farbicons unter jeder beliebigen TOS-Version unterstützen.

Kapitel 10

erklärt schließlich die Bedienung der Dateiauswahl von Interface.

Dieses Handbuch erklärt eines nicht, nämlich die grundsätzliche Verwendung von Resource-Dateien. Dies würde den Rahmen des Handbuchs bei weitem sprengen. Wir empfehlen Ihnen statt dessen die Anschaffung zweier Bücher, auf die Sie beim Programmieren kaum verzichten können. Das erste, das »ATARI Profibuch« aus dem Sybex-Verlag (ISBN 3-88745-888-5), ist ein absolutes Muß für jeden Programmierer. Es kostet 79 DM und dokumentiert auf 1.491 Seiten (kein Schreibfehler) die Hard- und Software der ATARI STs, STEs und TTs. Das zweite Buch kommt aus dem Hüthig Buch Verlag und heißt »Vom Anfänger zum GEM-Profi« (ISBN 3-7785-1792-9). Es ist mit einem Preis von 99 DM nicht billig, aber es gehört eine sehr gute C-Library im Sourcecode zum Lieferumfang (auf Diskette), die z.B. von Interface und den MyDials genutzt wird. Die MyDials sind eine Funktionsbibliothek, die wir mit Interface liefern.

Umgang mit dem Handbuch

In diesem Handbuch hat die Schrift eine besondere Bedeutung. Wenn wir Ihnen beispielsweise sagen möchten, daß die Taste ›Control‹ zusammen mit der Taste ›O‹ zu drücken ist, schreiben wir schlicht Control O. Beschreiben wir eine Option aus dem Menü, steht im Text nicht ›die Option »öffnen...« aus dem »Datei«-Menü...‹, sondern die Option Datei: öffnen... Ebenfalls in einer hervorgehobenen Schrift werden Buttons und ähnliche Elemente dargestellt, so daß Sie sich bereits an der Gestaltung orientieren können.

Um Ihnen beim späteren Nachschlagen das Auffinden der für Sie interessanten Passagen zu erleichtern, gibt es neben dem Stichwortverzeichnis eine weitere Übersicht, die sich am Fuß der einzelnen Seiten befindet. Dort steht in wenigen Worten, worum es auf der jeweiligen Seite geht, so daß Sie sich schon beim Durchblättern einen Überblick verschaffen können.

Registrierkarte

Ein wichtiger Hinweis: Bitte senden Sie umgehend die beigefügte Registrierkarte an unseren Vertrieb zurück. Interface ist ein Produkt, das ständig weiterentwickelt wird. Aufgrund der Komplexität des Programms können wir trotz größter Sorgfalt nicht ausschließen, daß bei der Anwendung kleine Fehler auftreten. Sie sollten uns dann benachrichtigen, und wir werden dafür Sorge tragen, daß dieser Fehler baldmöglichst behoben - und Ihnen das korrigierte Programm zur Verfügung gestellt wird. Erweiterte Versionen von Interface können Sie gegen eine geringe Gebühr als Update bzw. Upgrade erhalten, sofern Sie als Anwender bei uns eingetragen sind, also Ihre Registrierkarte eingeschickt haben.

Unsere telefonische und schriftliche Hilfe steht ebenfalls nur registrierten Anwendern zur Verfügung.

Kritik, Rat und Tat

Auch für Kritik - die ja durchaus positiv sein kann - und Verbesserungsvorschläge sind wir dankbar. Aber eine kleine Bitte möchten wir äußern: Beschreiben Sie eventuelle Fehler, Vorschläge und Wünsche so genau wie möglich.

Besonders im Fehlerfall bitten wir Sie, Ihre Rechnerkonfiguration exakt anzugeben. Hierzu gehört auch die Nennung aller Programme, die aus dem Auto-Ordner oder als Accessory gestartet wurden. Bei Multitasking-Betriebssystemen ist außerdem die Information wichtig, welche Programme parallel zu Interface liefen.

Wann immer Sie - im Zusammenhang mit Interface - vor einer Situation stehen, in der Sie ohne Hilfe nicht weiterkommen, stehen wir Ihnen gerne zur Verfügung. Rufen Sie uns einfach während der normalen Bürozeiten an.

Die meisten Fragen lassen sich aber bereits durch die Lektüre des Handbuchs oder der Hilfstexte, die die wichtigsten Dialoge von Interface anbieten, beantworten. Prüfen Sie vor einem Anruf also ob Sie über das Stichwortverzeichnis nicht ebenfalls die erwünschte Information erhalten.

Wenn Sie schriftlich um Rat fragen möchten, bitten wir Sie, uns Ihre Schwierigkeiten so umfassend wie möglich zu beschreiben. So vermeiden Sie Mißverständnisse. Wenn möglich fügen Sie bitte eine Diskette mit der entsprechenden Resource bei, damit wir prüfen können, wie Sie zum gewünschten Ergebnis kommen können.

Tip für Modembesitzer:

Kleinere Updates (Bugfixes ohne Änderung der Versionsnummer) können in der Mailbox „no | where“ bezogen werden.

Die Nummer der „no | where“: 02448 / 8009. Da nicht ausgeschlossen werden kann, daß sich diese Rufnummer irgendwann ändert, erfragen Sie sie bitte bei no | Software.

Der Autor von Interface - Olaf Meisiek - ist auch im MausNet erreichbar: Olaf Meisiek @ FL

Das sollten Sie unbedingt lesen

Auf Ihrer Programmdiskette befindet sich unter anderem eine Datei mit dem bezeichnenden Namen ›LIESMICH.TXT‹. Hier handelt es sich um einen Text, den Sie sich mit der Desktop-Option Datei:anzeigen ansehen können. Darauf sollten Sie nicht verzichten, denn in diesem Text finden Sie aktuellste Informationen zu Interface, die unter Umständen vom Handbuch abweichen. In diesem Fall gilt der Text, der in der Datei ›LIESMICH‹ vorgefunden wird.

Arbeiten mit Interface

In diesem Abschnitt lesen Sie eine Einführung in die Arbeit mit Interface. Nach einer Beschreibung der allgemeinen Bedienung und einigen grundsätzlichen Erklärungen geht es mit zwei Übungen weiter. In diesen Übungen erfahren Sie alles über den Aufbau einer einfachen Resource-Datei.

Interface installieren

Bevor Sie Interface zum ersten Mal starten, sollten Sie eine Sicherheitskopie der Programmdiskette anfertigen. Im Fall eines späteren Diskettenfehlers erspart Ihnen das eine Menge Ärger. Die Erfahrung zeigt, daß Disketten (Festplatten, Drucker, ...) immer dann kaputt sind, wenn man sie dringend braucht. Selbstverständlich dürfen Sie in keinem Fall Kopien von Interface (oder des Handbuchs) verkaufen oder verschenken. Wie Disketten kopiert werden, können Sie gegebenenfalls im Handbuch Ihres Computers nachlesen. Da Interface nicht kopiergeschützt ist, können Sie das Programm (bestehend aus den Dateien ›INTRFACE.PRG‹ und ›INTRFACE.RSC‹ sowie optional ›EXTOBFIX.PRG‹) auch auf einer Festplatte installieren.

Zum Lieferumfang von Interface gehören noch weitere Dateien, aber die drei eben genannten genügen für den Anfang völlig. Nur wenn Sie mit speziellen Libraries (Funktionssammlungen) arbeiten, benötigen Sie weitere Dateien. Diese befinden sich unter Umständen in gepackter Form (als ›*.TOS‹) auf den Interface-Disketten. Kopieren Sie sie auf Ihre Festplatte und starten Sie diese »Programme« durch Doppelklick. Es handelt sich um selbst entpackende Archive, d.h., daß Sie nach Beendigung des Programms die ausgepackten Dateien auf Ihrer Festplatte finden werden.

Allgemeines

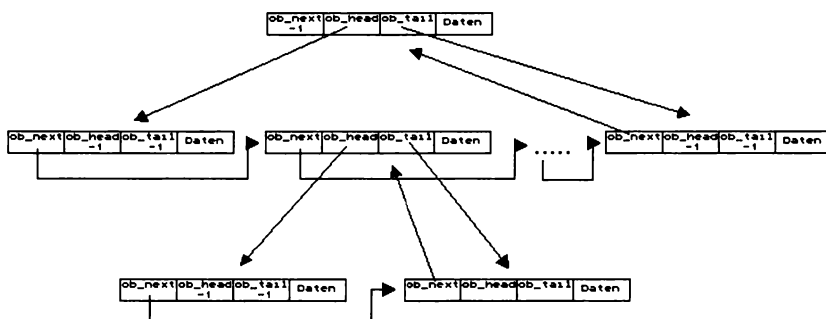
Resource?!

Interface ist ein Programm, mit dem Resource-Dateien (Menüs, Dialoge, Alertboxen, etc.) für GEM-Programme entwickelt werden können.

Resourcen sind in GEM-Programmen normalerweise alles, was außer Fenstern und deren Inhalten auf dem Bildschirm erscheint, z.B. Dialoge, Pulldown-Menüs, Desktop-Symbole oder Alertboxen. Resourcedaten befinden sich im allgemeinen nicht im Hauptprogramm, sondern werden in einer separaten Datei mit der Endung ›RSC‹ gespeichert. Das hat verschiedene Vorteile:

- Eine Resource-Datei kann auch ohne Programmierkenntnisse aufgebaut und verändert werden.
- Die Resource-Datei kann (zumindest teilweise) geändert werden, ohne daß man das Hauptprogramm neu compilieren muß.

- Ein Programm kann unterschiedliche Sprachen unterstützen, ohne daß man den Programmcode ändern muß. Für jede Sprache braucht nur eine eigene Resource-Datei aufgebaut werden. Ressourcen bestehen aus ›Objekten‹. Ein Objekt ist ein einzelnes Element eines Menüs oder Dialogs, also z.B. ein leerer Rahmen, Text enthaltende Rahmen, Text oder Buttons, Icons usw... Um ein Pulldown-Menü oder einen Dialog zu erzeugen, kombinieren Sie mehrere Objekte zu einem sogenannten ›Objektbaum‹. Die Bezeichnung ›Baum‹ rührt daher, daß die Verknüpfung der Objekte untereinander eine baumartige, d.h. verzweigte Struktur hat.



Das erste Objekt in einem Baum wird als ›Parent‹ (Elternobjekt) bezeichnet, die Objekte, die sich innerhalb eines Parents befinden, sind die ›children‹ (Kinder). Children-Objekte können auch selber wieder andere Objekte enthalten, zu denen sie sich dann wie ein Parent-Objekt verhalten. Da sich in der Fachliteratur die englischen Ausdrücke eingebürgert haben, werden wir auch weiterhin vom Parent und den Children reden. Das erste Objekt eines kompletten Objektbaums wird auch das ›Root‹ Objekt (Wurzelobjekt) genannt.

Ein Programm, mit dem man Resource-Dateien aufbauen und verändern, nennt man im allgemeinen ein Resource Construction Set (RCS). Mit Interface können Sie folgende Baumarten erzeugen:

Pulldown-Menüs: Sie bilden Gruppen von Wahlmöglichkeiten, die dem Benutzer in einem Programm zur Auswahl stehen. Die einzelnen Menüpunkte bestehen in der Regel aus Text.

Dialoge: Ein Dialog wird benutzt, um Informationen anzuzeigen und/oder abzufragen (es müssen also Eingaben gemacht werden).

Alerts: Ein Alert ist eine besondere Art von Dialog. Sie wird benutzt, um wichtige Hinweise, Warnungen oder Fehler anzuzeigen.

Freie Strings/Images: Hier können in einer Resource einzelne Texte oder Grafiken gespeichert werden, die vom Programm selbst verwaltet werden müssen.

Das ATARI RCS ab Version 2.0 (Bestandteil des offiziellen Entwicklungspaketes von ATARI) kennt noch die Baumart ›PANEL‹, die sich aber von einem Dialog kaum unterscheidet. Laut Digital Research (dem Entwickler des ATARI RCS) sind Objekte in Dialogen immer buchstabengenau und Objekte in Panels immer pixelgenau positioniert. Da dies von keinem anderen Resource-Editor unterstützt wird und es auch möglich ist, in Dialogen Objekte pixelgenau zu positionieren, gibt es diese Baumart nicht explizit in Interface. Sie entspricht vielmehr einem Dialog, der im Pixelraster konstruiert wurde.

Sprachgebrauch in diesem Handbuch

Damit wir nicht so oft aneinander »vorbeireden« möchten wir zu Anfang des eigentlichen Handbuchs gleich einige Begriffe einführen. Wenn Sie sich schon ein wenig mit dem ATARI ST auskennen, werden Ihnen die meisten schon bekannt sein. Wenn nicht, werden Sie sie schnell erlernen.

Mausklick: Kurzes Drücken der linken Maustaste. Mit ›einem Mausklick auf ...‹ ist das Bewegen des Mauscursors auf einen bestimmten Bereich und anschließendes, kurzes Drücken der Maustaste gemeint. Statt Mausklick wird gelegentlich auch nur ›Klicken‹ oder ›Anklicken‹ verwendet.

Doppelklick: Zweimaliger Mausklick, mit einer kurzen Pause, so wie beim Starten eines Programms vom Desktop.

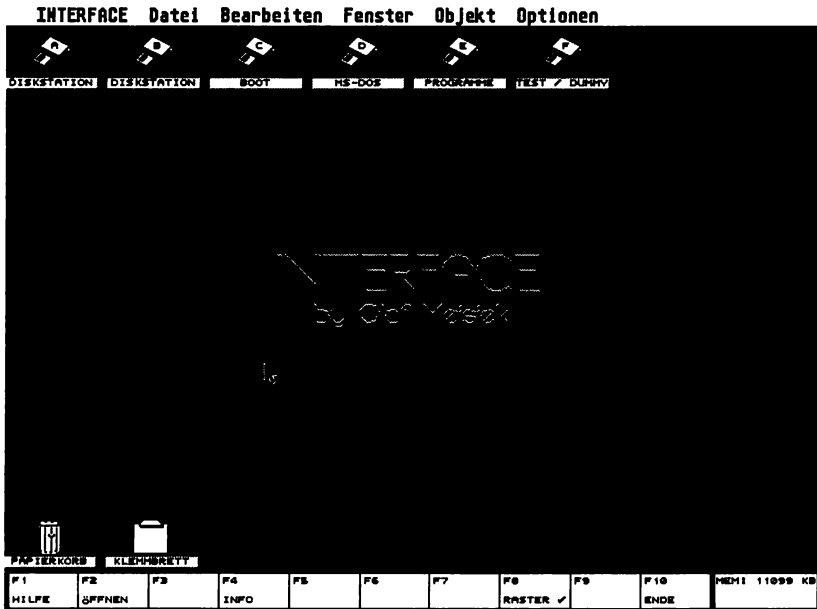
Aufziehen: Markieren eines Bereichs mit der Maus. Konkret: Mauscursor an den Anfang des Bereichs stellen, linke Maustaste drücken und so lange halten, bis der Mauscursor auf das Ende des Bereichs zeigt.
Button: Ein Text in einem Rahmen zum Anklicken. Zum Beispiel die ›Ok‹-Felder in den Dialogen. Als Button werden aber auch andere Elemente in den Dialogen bezeichnet, die angeklickt werden können (s.u.).

Radiobutton: In einigen Dialogboxen von Interface gibt es (runde) Buttons, von denen - in einer Gruppe - immer nur einer ausgewählt sein kann und bei denen die Auswahl eines »Knopfes« die anderen ausschaltet. Die Knöpfe verhalten sich wie die Programmtasten eines Radios - daher der Name. In Interface können solche Buttons auch durch Anklicken des zugehörigen (nebenstehenden) Textes ausgewählt werden.

- Checkboxes:** Sind Buttons, die man auswählen kann oder auch nicht. Werden sie ausgewählt, wechselt die Darstellung zu einem durchgekreuzten Rechteck, genau wie die »Bitte ankreuzen« Felder in einem Formular. Auch Checkboxes können durch Anklicken des zugehörigen Textes ausgewählt werden.
- Icon:** Grafisches Symbol, das z.B. eine Diskette, einen Mülleimer oder ähnliche Sinnbilder zeigt.
- Selektieren:** Auswählen, bzw. Anklicken eines Objektes, meistens ein Feld in einem Dialog.
- FlyDial:** »Fliegende Dialoge« sind schon seit einiger Zeit der Stand der Dinge bei modernen Programmen für ATARI. Ein fliegender Dialog ist durch ein »Eselsohr« in der oberen rechten Ecke des Dialogs gekennzeichnet. Klickt man auf diese Ecke, kann der Dialog auf dem Desktop verschoben werden.
- MyDial:** Die MyDials sind eine Funktionssammlung, die zum Lieferumfang von Interface gehört. Mit ihnen können Sie in Ihre C- und Basic-Programme mit einfachen Mitteln fliegende Dialoge, Tastatursteuerung in Dialogen, Popups oder Farbicons (und einiges mehr) einbauen. Im Handbuch wird häufiger auf die MyDials eingegangen und ein eigenes Kapitel beschreibt Ihre Anwendung.

Symbole auf dem Desktop

Interface verfügt über einen eigenen Desktop, der sofort nach dem Start angezeigt wird und die wesentlichen Verwaltungselemente in grafischer Form bereithält. Die Desktopfarbe kann selbst definiert werden, indem man in der Resource die Farbmuster im Baum "DESKCOL" entsprechend ändert.



Unterhalb der Menüleiste beginnt der Desktop. Auf ihm befinden sich Symbole für die Laufwerke, ein Papierkorb, ein Klemmbrett (das ›Clipboard‹), Symbole für gela-

dene Resource-Dateien und eine Funktionstastenleiste. Die Symbole auf dem Desktop können - wie vom ATARI-Desktop gewohnt - beliebig verschoben werden. Die einzige Ausnahme ist der Block mit den Funktionstasten, der Ihnen anzeigt, welche Taste welche Option auslöst. Er wird immer am unteren Bildschirmrand angezeigt.

Taste	Name	Menü Option
F1	HILFE	Datei Hilfe...
F2	ÖFFNEN	Datei Öffnen
F3	SCHLIESS	Datei Schließen
F4	INFO	Datei Info...
F5	SICHERN	Datei Sichern
F6	SUCHEN	Bearbeiten Label suchen...
F7	FLAGS	Bearbeiten Flags setzen...
F8	RASTER	Optionen Raster benutzen
F9	TESTEN	Bearbeiten Objektbaum testen
F10	ENDE	Datei Ende

Anmerkung: In diesem Handbuch werden Hardcopies für s/w und Farbe in verschiedenen Auflösungen verwendet. Daher kann es vorkommen, daß eine Abbildung nicht genau dem Bild entspricht, das Sie auf Ihrem Monitor sehen.

Wenn Sie mit einem Doppelklick ein Laufwerkssymbol anklicken, öffnet sich das dazu gehörige Fenster, in dem alle Ordner und Resource-Dateien, die sich auf dem Laufwerk befinden, angezeigt werden.

Der Papierkorb dient zum Löschen von Dateien und Objekten. Um etwas zu löschen, muß es nur mit der Maus auf den Papierkorb gezogen und dort losgelassen werden. Das Clipboard wird für Objekte aus Formular- und Menübäumen benutzt. Objekte zieht man einfach auf das Clipboard, bzw. zieht sie wieder aus dem Clipboard auf ein Dialog- oder Menü-Fenster. Wenn das Clipboard-Icon verschoben werden soll, muß man es erst selektieren (Mausklick oder Rubberbox) und kann es dann verschieben. Solange es nicht selektiert ist, kann man den Inhalt »herausziehen«.

Man darf selbstverständlich auch mehrere Objekte gleichzeitig aufs Clipboard kopieren. Wenn man die Shift-Taste beim Loslassen der Maustaste (Mauscursor über dem Clipboard) gedrückt hält, werden die Objekte auf das Clipboard kopiert. Im anderen Fall werden sie ins Clipboard verschoben. Die Kopie verbleibt solange auf dem Clipboard, bis ein neues Objekt darauf kopiert wird, d.h. Sie können mehrere Kopien des gleichen Objekts vom Clipboard in ein oder mehrere Formulare ziehen. Das hier beschriebene Clipboard ist nur Interface-Intern, d.h. es hat nichts mit einem eventuell vorhandenen Clipboard-Ordner zu tun. Dagegen funktionieren die eben beschriebenen Clipboard-Funktionen in den Textfeldern von Dialogboxen und im

Iconeditor mit dem Standard-GEM-Clipboard. Alle Funktionen des Clipboards lassen sich mit Bearbeiten: Ausschneiden, Bearbeiten: Kopieren und Bearbeiten: Einfügen ausführen.

Geladene Resource-Dateien werden durch ein Icon über der Funktionstasten-Anzeige dargestellt. Durch einen Doppelklick auf ein solches Icon kann ein Fenster mit den Objektbäumen dieser Datei geöffnet oder, falls es schon offen ist, in den Vordergrund geholt werden. Außerdem können Dateisymbole auf ein Laufwerk oder den Papierkorb gezogen werden, um die zugehörige Resource zu speichern bzw. zu löschen. Ein Tip für alle, die den Interface-Desktop abschalten wollen: Desktop ins Fenster legen (siehe Referenz, Optionen: Einstellungen...), Parameter speichern, Interface erneut laden, das Desktop-Fenster schließen, Parameter erneut speichern. Bei allen weiteren Starts erscheint dann kein Desktop mehr. Man kann den Desktop wieder öffnen, indem man die durch alle Copyright-Meldungen klickt(etwas extravagant, aber was soll man machen...), oder indem man den Desktop nicht mehr ins Fenster legt.

Empfehlenswerter ist es aber, den Desktop als Fenster soweit zu verkleinern, daß nur noch der Papierkorb, das Clipboard und ein bis zwei Resource-Dateien sichtbar sind. Wenn der Desktop in einem Fenster liegt, kann man unter MultiTOS auch Objekte anwählen und verschieben, wenn der Desktop im Hintergrund liegt. (Dies geht auch ohne MultiTOS. Die Objekte müssen dazu nur mit der linken und rechten Maustaste angeklickt werden.)

Öffnen von Dateien

Resource-Dateien können per Tastatur (s.u.), mit dem Menüpunkt „Öffnen“ oder mit den Desktopfunktionen geladen werden.

Drag&Drop des Betriebssystems (z.B. MultiTOS) wird unterstützt. Akzeptiert werden Resource-Dateien, die auf ein beliebiges Interface-Fenster gezogen werden, und IMG-Dateien und WINDOWS- bzw OS/2-Dateien, die auf ein Iconeditorfenster gezogen werden können.

Das Menü

Das Menü von Interface unterscheidet sich grundsätzlich nicht von denen anderer Programme. Bis auf eine Besonderheit: am Ende jeder Zeile stehen meistens zwei bis drei Zeichen. Nicht ohne Grund, denn (fast) alle Optionen, die in den Menüs aufgeführt sind, können auch über die Tastatur aufgerufen werden, und die Zeichen hinter der Option geben an, welche Tastenkombination dafür zu drücken ist.

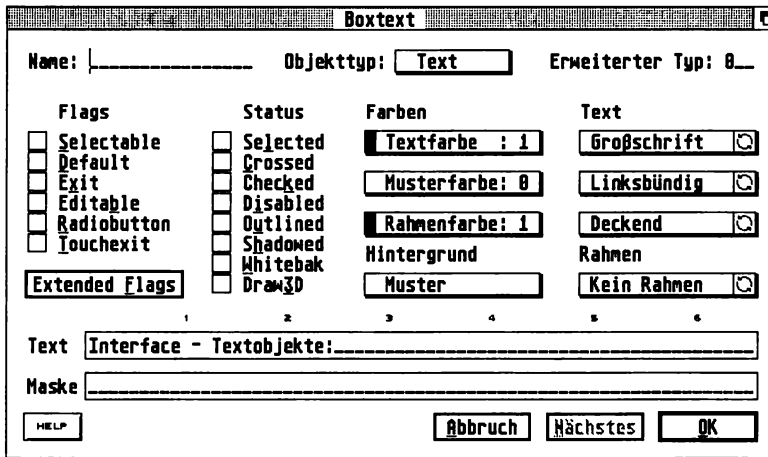
Da ein »normaler« Tastendruck zu einer Eingabe in einen Text führt, sind zur Vermeidung von Mißverständnissen bei der Auswahl einer Menüoption zusätzliche Tasten zu drücken. Die Zeichen in den Menüs bedeuten:

- A** für die Taste Control
- ⌘** für die Taste Alternate
- ⇧** für eine der Shift-Tasten

Zum Öffnen einer Resource sind z.B. die Tasten Control und O gleichzeitig zu drücken.

Dialogboxen

Auch die Dialogboxen von Interface unterscheiden sich grundsätzlich nicht von denen anderer Programme oder denen aus dem Desktop. Man kann mit den Dialogen von Interface aber eine ganze Reihe sehr praktischer Dinge machen, die sonst nicht möglich sind. Die hier abgebildete Dialogbox enthält z.B. zusätzliche Funktionen.



Da wäre zunächst - und am auffälligsten - der Mover oberhalb des Dialogs. Interface stellt seine Dialoge optional in Fenstern dar, so daß Sie die Dialoge verschieben und vor allem uneingeschränkt im Multitasking arbeiten können. Mit Optionen: Einstellungen... können Sie diese Option ausschalten. In diesem Fall finden Sie in der rechten oberen Ecke der Dialoge ein »Eselsohr«, das den gleichen Zweck erfüllt. Sie können diese Ecke anklicken und mit gedrückter Maustaste den Dialog verschieben. Hierbei gibt es eine weitere Besonderheit: Wenn Sie gleichzeitig eine Shift-Taste drücken, wird der Dialog ausgeblendet und der Hintergrund sichtbar. Eine unge-

mein praktische Sache, wenn einen das Kurzzeitgedächtnis im Stich läßt und die notwendigen Informationen gerade von dem zu bearbeitenden Dialog verdeckt werden.

In allen drei Fällen merkt Interface sich die Position der Dialoge oder Fenster und öffnet sie bis zum Beenden des Programms jeweils an der zuletzt gewählten Position. Bei genauer Betrachtung fällt außerdem auf, daß einige Buchstaben im Dialog unterstrichen dargestellt sind. Damit soll sichtbar gemacht werden, daß durch Drücken der dem unterstrichenen Buchstaben entsprechenden Taste zusammen mit der Alternate-Taste das gleiche geschehen würde, als wenn diese Option angeklickt würde. Interface ist dadurch fast vollständig über die Tastatur zu steuern, denn auch Buttons wie Abbruch können grundsätzlich mit Undo und die Hilfsfunktion mit Help ausgelöst werden.

Etwas auffälliger sind da schon die runden »Knöpfe«, die es in normalen Dialogboxen nicht gibt. Es handelt sich um die etwas weiter oben schon erwähnten Radiobuttons. Sie sehen zwar elegant aus, sind aber nicht sehr groß und damit nicht besonders gut auszuwählen. Deshalb haben wir dafür gesorgt, daß Sie auch den Text neben dem Button anklicken können und dabei den gleichen Effekt erzielen, als würden Sie den Button selbst anklicken.

Und noch ein besonders dargestellter Button findet sich in den Dialogen von Interface: Checkboxes. Dies sind Buttons, die man auswählen kann oder auch nicht. Werden sie ausgewählt, wechselt die Darstellung zu einem durchkreuzten Rechteck, genau wie die »Bitte ankreuzen« Felder in einem Formular.

Boxtext

Name: _____ Objektyp: **BoxText** Erweiterter Typ: 0...

Flags	Status	Farben	Text
☐ Selectable	☐ Selected	Textfarbe : 1	Großschrift
☐ Default	☐ Crossed	Musterfarbe: 0	Linksbündig
☐ Exit	☐ Checked	Rahmenfarbe: 1	Zentriert
☐ Editable	☐ Disabled	Hintergrund	Rechtsbündig
☐ Radiobutton	☐ Outlined	Muster	Deckend
☐ Touchexit	☐ Shadowed		Rahmen
Extended Flags	☐ Whitebak		Kein Rahmen
	☐ Draw3D		

1 2 3 4 5 6

Text **Boxtext** _____

Maske _____

HELP **Abbruch** **Nächstes** **OK**

Die Dialogbox für Boxtext-Parameter zeigt (beispielhaft) noch ein weiteres Element, sogenannte »Popup-Menüs«. Erscheinen in einem Dialog Buttons mit einem Schatten (s.o. bei der Text-Ausrichtung), so wird beim Klicken auf den Button an dieser Stelle eine Auswahl möglicher Werte angezeigt, die Sie - wie die Ihnen bekannten Menü-Optionen - durch Anklicken selektieren können. Wollen Sie die Auswahl abbrechen, klicken Sie einfach neben das Menü.

Popup-Menüs können außer mit der Maus auch über die Tastatur bedient werden. Mit den vertikalen Cursortasten kann der Balken, der einen Eintrag markiert, durch das Popup bewegt werden, mit ClrHome und Shift ClrHome kann er an den Anfang bzw. das Ende gestellt werden. Return wählt einen Eintrag aus, als ob er mit der Maus angeklickt wurde, Undo oder Esc brechen das Popup wie mit einem Mausklick neben das Menü ab. Einige Popups weisen ein zusätzliches Element zur Steuerung auf, nämlich ein Feld am rechten Rand des Popups, das einen kreisförmigen Pfeil zeigt. Wenn Sie in dieses Feld klicken, wird (von oben nach unten) die nächste Option des Popup-Menüs ausgewählt. Besonders bei Popups mit wenigen Einträgen erspart man sich so einen Mausklick (ein besonderer Service für »ökonomische« Anwender).

Erweiterte Editier-Funktionen

GEM stellt dem Benutzer in Dialogen verschiedene Editiermöglichkeiten zur Verfügung, wie z.B. das Setzen des Cursors mit der Maus auf ein bestimmtes Eingabefeld. Interface bietet zusätzlich zu den bekannten GEM Funktionen noch einige mehr:

Funktion	Effekt
Shift + Cursor links	Cursor auf den Anfang der Zeile setzen.
Shift + Cursor rechts	Cursor auf das Ende der Zeile setzen.
Control + Cursor links	Cursor ein Wort nach links bewegen.
Control + Cursor rechts	Cursor ein Wort nach rechts bewegen.
Insert	Wenn auf dem Feld unter dem Cursor beliebige Zeichen erlaubt sind, dann erscheint ein Dialog mit allen Sonderzeichen die im allgemeinen nicht über die Tastatur eingegeben werden können. Wählen Sie das gewünschte Zeichen mit der Maus aus.
Control C	Text aus dem aktuellen Edit-Feld in das GEM-Clipboard kopieren.
Control X	Text aus dem aktuellen Edit-Feld in das GEM-Clipboard kopieren und das Edit-Feld anschließend löschen.
Control V	Text aus dem GEM-Clipboard einfügen.
Mausklick auf	Der Cursor wird vor oder hinter den angeklickten Buchstaben gesetzt.

Die Maus

Die meisten der Mausfunktionen in Interface, wie z.B. klicken und ziehen, arbeiten genauso wie im GEM-Desktop oder in anderen Programmen, die sich an den Standard des Desktops halten. Trotzdem werden hier noch einmal alle wichtigen Mausfunktionen vorgestellt. »Klick« bedeutet dabei ein kurzes Drücken der linken Maustaste, während »ziehen« das Verschieben der Maus bei gleichzeitig gedrückter linker Maustaste beschreibt.

Funktion

Effekt

Klick

Selektiert ein unter der Maus befindliches Objekt. Ein selektiertes Objekt wird von Interface invertiert dargestellt.

Shift + Klick

Selektiert zusätzliche Objekte, wenn schon eines oder mehrere selektiert sind. Shift + Klick auf ein selektiertes Objekt hebt die Auswahl wieder auf.

Control + Klick

Selektiert das Parent eines Objektes.

Doppelklick

Öffnet eine Datei, einen Baum oder ein Objekt zum Editieren.

Ziehen

Holt ein Objekt aus der Teilebox in ein Fenster oder verschiebt alle selektierten Objekte, wie z.B. Dateien, Bäume oder Objekte.

Shift + Ziehen

Kopiert die selektierten Bäume oder Objekte.

Control + Ziehen

Kopiert das Parent eines Objektes.

Shift + Control + Ziehen

Kopiert das Parent eines Objektes mit allen Children.

Alternate + Ziehen

Zieht in Formularfenstern ein Rechteck auf. Alle in dem Rechteck befindlichen Objekte werden nach der Operation selektiert.

Es sind beliebige Kombinationen aus Shift, Control und Alternate erlaubt. Außerdem können die beschriebenen Funktionen auch in inaktiven, d.h. im Hintergrund liegenden Fenstern angewendet werden, wenn gleichzeitig die rechte Maustaste gedrückt gehalten wird.

Fenster

Der Inhalt von Interface-Fenstern kann nicht nur durch Anklicken der Slider mit der Maus verschoben werden, sondern auch mit den Cursortasten. Wird dabei gleichzeitig eine Shift-Taste gedrückt, kann um den gesamten Fensterinhalt geblättert werden. Mit `ClrHome` und `Shift ClrHome` gelangt man an den oberen bzw. unteren Rand des Fensterinhalts.

Nichts geht mehr - die Speicherverwaltung

Interface ist nicht in der Lage, mit beliebig großen oder beliebig vielen Ressourcen oder Objekten zu arbeiten. Die Grenzen sind allerdings sehr großzügig bemessen. Ihre Maximale Anzahl beträgt ...

Ressourcen-Dateien	15
Bäume einer Resource	32767
Objekte in einem Baum	1024
Images pro Resource	10240
Objektnamen pro Resource	$32767 * 1024$

Die maximale Anzahl der Strukturen (OBJECT, TEDINFO, ICONBLK, BITBLK und USERBLK) einer Resource beträgt 5120 (je Struktur). Die maximale Länge aller Strings darf 160 KB nicht überschreiten.

Die Länge einer Resource-Datei selbst ist als unsigned long definiert, d.h. sie könnte theoretisch vier Gigabyte groß werden. Praktisch ist sie begrenzt durch die maximale Anzahl oder Länge der eben genannten Strukturen. Wie Sie selbst in Ihren Programmen mit Resource-Dateien arbeiten, die größer sind als 64 KByte, erfahren Sie in Kapitel 8.

Eine einfache Übung

In diesem Kapitel sollen Sie mit den Grundfunktionen von Interface vertraut werden.



MYMENU



PLANET

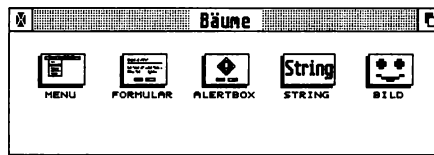
Anhand der abgebildeten Beispiel-Resource, bestehend aus einem Pulldown-Menü und einem Dialog, lernen Sie die Funktionen zum Aufbau und Bearbeiten einer Resource kennen.

Eine neue Resource entsteht

Starten Sie Interface und wählen Sie die Option Datei: Neu aus. Daraufhin erscheint auf dem Desktop ein neues Icon mit der Bezeichnung ›NAMENLOS‹ (um anzuzeigen, daß Sie zunächst nur mit Datei: Speichern als... gespeichert werden kann). Außerdem öffnet sich ein leeres Fenster mit der gleichen Bezeichnung, in dem später die Objektbäume angezeigt werden.

Aufbau eines neuen Menüs

Wählen Sie im Pulldown-Menü Bearbeiten die Option Bäume... aus. Dann erscheint auf dem Bildschirm eine Box oder ein Fenster (einstellbar - siehe oben), in der alle verfügbaren Baumarten angezeigt werden.



Ziehen Sie von dort den Baum ›MENU‹ auf das Fenster, das zu dem Icon ›NAMENLOS‹ gehört. Es erscheint ein Dialog, in dem Sie einen Namen für das neue Menü eingeben müssen.



Nennen Sie den neuen Menübaum ›MYMENU‹ und beenden Sie die Eingabe durch einen Klick auf den OK-Button des Formulars oder durch einen Druck auf die Return-Taste.

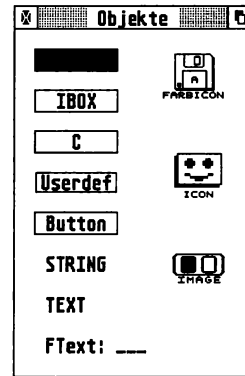
Öffnen des Menübaums

In dem Fenster der neu angelegten Resource-Datei ist jetzt ein Menü-Icon mit dem Namen ›MYMENU‹ zu sehen. Um dieses Menü zu editieren, führen Sie einen Doppelklick auf das Icon aus. Daraufhin erscheint ein weiteres Fenster mit dem Titel ›MYMENU‹. In diesem Fenster sehen Sie eine Menüzelle, die bereits die Einträge ›Desk‹ und ›Datei‹ hat.

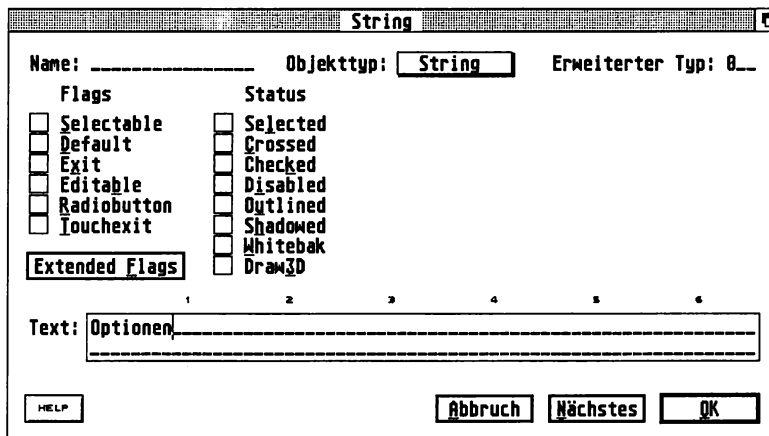
Hinzufügen neuer Objekte in ein Menü

Zum Hinzufügen eines weiteren Menüeintrags wählen Sie Bearbeiten: Objekte... aus. Jetzt erscheint auf dem Bildschirm eine Box, in der sich die verschiedenen Objekttypen befinden.

Ziehen Sie das Objekt ›STRING‹ rechts neben den im Menüfenster befindlichen Eintrag ›Datei‹. Wenn Sie alles richtig gemacht haben, müsste jetzt der neue Menüpunkt ›STRING‹ neben dem Menüpunkt ›Datei‹ im Fenster zu sehen sein.



Führen Sie nun einen Doppelklick auf den Eintrag ›STRING‹ aus. Es erscheint ein großer Dialog, in dem alle Daten für das neue Objekt geändert werden können.



Bewegen Sie mit den vertikalen Cursortasten oder der Tab-Taste den Cursor auf die untere Eingabezeile dieser Box. In dieser Zeile sollte der Text ›STRING‹ zu lesen sein. Löschen Sie diesen Text durch einen Druck auf die Esc-Taste und geben Sie statt dessen den Text ›Optionen‹ ein.

Schließlich bewegen Sie den Cursor noch auf das links oben stehende Eingabefeld Name: und geben dort den Namen ›MOPTIONS‹ ein.

Weitere Einstellungen für den Menütitel sind nicht erforderlich, da sie von Interface automatisch vorgenommen werden. Zum Verlassen des Dialogs klicken Sie auf den OK-Button oder drücken die Return-Taste. Um neue Einträge in das Optionen-Menü zu machen, müssen Sie zuerst den Eintrag ›Optionen‹ (im Resource-Fenster) einmal anklicken. Dadurch wird der Titel invertiert und ein kleines Rechteck, die Menübox, erscheint darunter. Rufen Sie wieder die Objektebox auf und ziehen Sie das Objekt ›STRING‹ in diese Menübox (auf die weitere Bearbeitung wird weiter unten noch eingegangen). Sie müssen sich um die Größe der Box keine Gedanken machen, sie wird von Interface automatisch angepaßt.

Außerdem sollten Sie noch neue Einträge in das ›Datei‹-Menü einfügen. Wenn Sie dieses Menü anklicken, werden Sie feststellen, daß dort bereits ein Eintrag, nämlich ›Ende ^Q‹, vorhanden ist. Diesen kopieren Sie drei mal, indem Sie ihn bei gedrückter Shift-Taste mit der Maus verschieben. Als nächster Schritt wird jeder neue Eintrag in ein sinnvolles Kommando geändert.

Führen Sie einen Doppelklick auf den ersten Eintrag im ›Datei‹-Menü aus und ändern Sie den Text von ›Ende ^Q‹ in ›Öffnen... ^O‹. Die 3 Punkte hinter dem Text zeigen einem Anwender des Programms, daß nach der Wahl dieser Option ein Dialog (hier z.B. die Dateiauswahl) erscheinen wird, um den Benutzer um weitere Angaben zu bitten. Da in einem »guten« GEM-Programm die wichtigsten Menü-Funktionen auch über die Tastatur zur Verfügung stehen sollten, steht hinter dem ›Öffnen...‹ die Bezeichnung des äquivalenten Tastaturbefehls ›^O‹. Das ›^‹ steht für die Control-Taste.

Nach den GEM-Programmierrichtlinien gehören vor einen Menüeintrag zwei Leerzeichen (damit Häkchen vor einem Eintrag angezeigt werden können) und dahinter eins. Sie brauchen sich aber nicht selber darum zu kümmern, Interface fügt diese Leerzeichen automatisch ein. Geben Sie jetzt in das Feld Name: den Namen ›MOPEN‹ ein und beenden Sie dann den Dialog mit dem OK-Button oder durch Return.

Anmerkung: Bei Tastaturfunktionen in Menüzeilen haben sich folgende Abkürzungen eingebürgert:

Control =  das Zeichen das auf der deutschen Tastatur links von Backspace zu finden ist.

Alternate =  das Zeichen, das in den meisten Fenstern in der rechten oberen Ecke steht. Um es eingeben zu können, müssen Sie die Insert-Taste drücken und das 7. Zeichen links oben in der ASCII Tabelle mit der Maus auswählen.

Shift =  (ebenfalls aus der ASCII Tabelle zu wählen).

Führen Sie jetzt die selben Schritte noch zwei mal aus, um den zweiten Eintrag in ›Schließen ^U‹ mit dem Namen ›MCLOSE‹ zu ändern.

Damit der letzte Eintrag von den ersten beiden besser getrennt wird, ändern Sie jetzt den dritten Eintrag in ›-‹. Bevor Sie den Dialog verlassen, klicken Sie noch mit der Maus auf das Feld Disabled. Danach ist in dem Kästchen neben dem Feld ein Kreuz zu sehen. Durch diese Einstellung wird der neue Text hell gezeichnet und kann nicht ausgewählt werden. Außerdem fügt Interface automatisch so viele Striche zu dem vorgegebenem hinzu, daß ein durchgehender Trennstrich entsteht.

Ändern Sie auf die oben beschriebene Weise auch noch den neuen Eintrag unter Optionen von ›STRING‹ auf ›Einstellungen... C‹. Wenn Sie alles richtig gemacht haben, dann müßte das Menü jetzt so aussehen:

Wenn bei Ihnen die Tastaturkürzel ›^O‹, ›^U‹ und ›^Q‹ im Menü ›Datei‹ nicht genau untereinander stehen, fügen Sie einfach an den entsprechenden Stellen noch einige Leerzeichen ein.

Öffnen...	^O
Schließen	^U
Ende	^Q

(Auch dies kann durch Interface automatisiert werden, wenn Sie in Optionen: Einstellungen... die Option Menü-Tastaturkürzel anordnen auswählen.)

Testen des Menüs

Wählen Sie Option Bearbeiten: Objektbaum testen oder drücken Sie die Funktionstaste F9, um das neue Menü auszuprobieren.

Es öffnet sich ein neues Fenster, an dessen oberen Rand die Menüzeile zu sehen ist. Nachdem Sie auf die Menüzeile geklickt haben, reagiert das Menü bei Mausbewegungen in dem Fenster genauso wie ein »echtes« Menü am oberen Bildschirmrand. Klicken Sie jetzt bitte den Eintrag ›Einstellungen... C‹ an. Es sollte eine kleine Alertbox erscheinen, die Ihnen anzeigt, welchen Namen der gewählte Menüpunkt hat und um welche Objektnummer es sich handelt. Wenn Sie weiter testen wollen, wählen Sie in diesem Dialog die Option Weiter.

Um das Testen zu beenden, gibt es zwei Möglichkeiten: Wählen Sie einen Menüpunkt und klicken Sie in der dann erscheinenden Alertbox (s.o.) auf Abbruch oder schließen Sie das Fenster in dem das Menü angezeigt wird.

Anmerkung: die in das Menü eingetragenen Tastaturfunktionen ›^O‹, ›^U‹ oder ›C‹ funktionieren schon in der Testfunktion, da Interface diese Tastenkombinationen direkt aus dem Menü ausliest. Wenn Sie ein Menü später in einem eigenen Programm verwenden, müssen Sie selbst eine Routine schreiben, die diese Tastendrucke abfängt und auswertet. Eine fertige Routine in C finden Sie in dem Buch ›Vom Anfänger zum GEM-Profi‹ von Dieter und Jürgen Geiß. Eine ähnlich Routine wird auch von den MyDials unterstützt und in Interface benutzt.

Unser Beispielenü ist jetzt fertig. Um den Menübaum zu schließen, müssen Sie die linke obere Ecke des zugehörigen Fensters anklicken.

Aufbau eines neuen Formulars (Dialogs)

Wählen Sie wieder Bearbeiten: Bäume... aus. Dann erscheint auf dem Bildschirm eine Box oder ein Fenster, in der die verfügbaren Baumarten angezeigt werden. Ziehen Sie davon das Icon ›FORMULAR‹ auf das Fenster, in dem sich bereits das ›MYMENU‹-Icon befindet. Geben Sie dem neuen Baum den Namen ›PLANET‹.

Öffnen des Formularbaums

Öffnen Sie jetzt den Baum ›PLANET‹ durch einen Doppelklick auf das entsprechende Icon. Es öffnet sich ein Fenster mit dem Titel ›PLANET‹, in dem eine leere Box mit einem Rahmen zu sehen ist. Diese Box ist das bereits zu Anfang dieses Abschnitts erwähnte Parent-Objekt.

Hinzufügen neuer Objekte in ein Formular

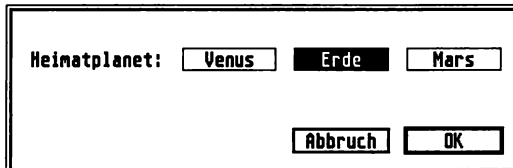
Rufen Sie die Objektkbox auf. Sie können sie mit der Maus über das Bearbeiten-Menü oder auch mit der Taste O (ausnahmsweise einmal ohne eine weitere Taste) auswählen. Immer, wenn Sie daraus ein Objekt herausziehen, verschwindet die Box wieder. Rufen Sie sie bei Bedarf wieder mit O auf. (Die Baum- und Objektkboxen können auch in einem Fenster dargestellt werden und stehen so ständig zur Verfügung. Eine entsprechende Option finden Sie im Einstellungen...-Dialog im Optionen-Menü.)

Ziehen Sie ein STRING-Objekt links oben in das Fenster und ein IBOX-Objekt rechts daneben.

Vergrößern Sie den IBOX-Rahmen in der Breite, so daß er fast bis an den rechten Rand des Fensters reicht. Vorab (gleich kommt mehr dazu): Sie können Objekte vergrößern, indem Sie mit der Maus an der rechten unteren Ecke des Rahmens ziehen. Dann ziehen Sie ein BUTTON-Objekt aus der Objektkbox in den linken Teil des IBOX-Rahmens. Öffnen Sie in dem Fenster das STRING Objekt durch einen Doppelklick und ändern Sie im String-Dialog den Text ›STRING‹ in ›Heimatplanet:‹.

Da das Objekt keinen Namen bekommen soll, klicken Sie gleich darauf den OK-Button an.

Wenn der neue Text des STRING-Objekts das BUTTON-Objekt verdecken sollte, ziehen Sie einfach das IBOX-Objekt, in dem der Button liegt, weiter nach rechts. Sollte das nicht gehen, weil das Objekt schon rechts an den Rand reicht, dann müssen Sie es vorher etwas verkleinern (die rechte untere Ecke nach links ziehen). Zu Ihrer Orientierung zeigen wir hier schon mal wie der fertige Dialog aussehen soll.



Öffnen Sie das BUTTON-Objekt durch einen Doppelklick und ersetzen Sie den Text ›Button‹ durch den Text ›Venus‹. Dann tragen Sie als Objektnamen in das Feld ›Name:‹ den Text ›PLVENUS‹ ein. Sorgen Sie außerdem dafür, daß im Button-Dialog nur die Checkboxes Selectable und Radiobutton ausgewählt, d.h. mit einem Kreuz versehen sind.

›Selectable‹ bewirkt, daß der Button ›Venus‹ ausgewählt (selektiert) werden kann. Außerdem wird er invertiert wenn er angeklickt wird.

Ein ›Radiobutton‹ hat - genau wie die Knöpfe eines alten Radios - die folgenden Eigenschaften: Innerhalb eines Objekts (hier: der IBOX-Rahmen) kann immer nur genau ein Radiobutton zur Zeit selektiert sein, es muß aber immer einen selektierten Button geben. Wenn man einen nicht ausgewählten Button anklickt, wird der vorher selektierte Button in diesem Rahmen automatisch deselektiert. Bestätigen Sie die Einstellung mit dem OK-Button des Dialogs. In diesem Dialog soll es noch zwei weitere Radiobuttons geben. Kopieren Sie dazu den Button ›Venus‹ zunächst einmal, indem Sie ihn bei gedrückter Shift-Taste mit der Maus rechts neben den ersten Button ziehen. Dann wiederholen Sie das ganze für den dritten Button. Wichtig: Auch alle neuen Buttons müssen innerhalb des IBOX-Objekts liegen. Wenn der Platz nicht ausreichen sollte, vergrößern Sie bitte diese Box. Mehr zu Thema IBOX folgt gleich.

Ändern Sie den Text des zweiten Buttons in ›Erde‹ um, und geben Sie ihm den Namen ›PLERDE‹. Den dritten Button ändern Sie in ›Mars‹ und ›PLMARS‹. Da in einer Gruppe von Radiobuttons immer einer selektiert sein sollte, öffnen Sie jetzt den Button ›Erde‹ durch einen Doppelklick. Anschließend klicken Sie auf den Status Selected und verlassen den Dialog mit OK. Jetzt ist der Button ›Erde‹ selektiert und wird dementsprechend invertiert angezeigt.

Vergrößern oder Verkleinern von Objekten

Innerhalb eines Formularfensters (ein Fenster, in dem ein Formular editiert wird) können alle darin befindlichen Objekte in der Größe geändert werden.

Um die Größe eines Objektes zu ändern, können Sie mit der Maus an der rechten unteren Ecke des Objektes ziehen, bis die gewünschte Größe erreicht ist. Diese im allgemeinen unsichtbare »Ziehecke« hat die Größe eines Buchstabens (die Ecke kann im Einstellungen...-Dialog des Optionen-Menüs sichtbar gemacht werden). Wenn das Objekt kleiner wird, dann beträgt die Größe die Hälfte der Höhe oder Breite des Objektes.

Objekte lassen sich in übrigens in vier Richtungen vergrößern. Man kann das Objekt zwar immer noch nur in der unteren rechten Ecke mit der Maus zur Größenänderung »anfassen«, aber man kann die Maus danach nach links oder über das Objekt ziehen. Interface schaltet dann automatisch auf die jeweils der Maus am nächsten liegende Ecke um.

Wenn ein Objekt nach dem Vergrößern andere Objekte komplett einschließt, fragt Interface, ob diese Objekte als Children adoptiert werden sollen. Wird diese Abfrage bejaht, wird das vergrößerte Objekt zum Parent der überdeckten Objekte.

Die IBOX und ihre besondere Bedeutung

Zuerst einmal eine Erklärung, warum alle Radiobuttons in dieser Box liegen müssen: Wenn Sie in einem Dialog mehr als eine Gruppe von Radiobuttons verwenden, müssen diese Gruppen in unterschiedlichen Parent-Objekten (Boxen) liegen. Ansonsten würden alle Radiobutton als eine Gruppe betrachtet, da es keine Möglichkeit gäbe, die Zugehörigkeit zu einer bestimmten Gruppe festzustellen.

Da es in unserem Beispiel-Dialog nur eine Gruppe gibt, könnte man sich das IBOX-Objekt sparen. Aber wenn dieser Dialog um eine weitere Gruppe erweitert werden soll, muß man die Radiobutton-Gruppen wieder durch unterschiedliche Parent-Objekte voneinander trennen. Daher ist es sinnvoll, Radiobutton-Gruppen von vornherein in Parent-Objekten zusammenzufassen. Verkleinern Sie nun das IBOX-Objekt soweit wie möglich. Da das IBOX-Objekt nur dazu dient, die Radiobuttons zusammenzufassen und es im fertigen Dialog nicht zu sehen sein soll, öffnen Sie es bitte durch einen Doppelklick. Im daraufhin erscheinenden Dialog sehen Sie unten rechts

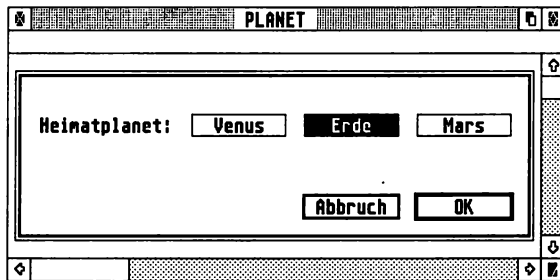
ein Feld mit der Überschrift Rahmen. Klicken Sie das Feld Außen 1 Pixel an und wählen Sie in dem anschließend angezeigten Popup-Menü den Eintrag Kein Rahmen aus. Wenn Sie den Einstellungs-Dialog jetzt mit OK beenden, sollte die IBOX nicht mehr sichtbar sein. Sie läßt sich aber trotzdem immer noch anwählen und verschieben.

✖ Wichtig: die ›Exit‹-Buttons ›Exit‹-Buttons sind Objekte, mit denen ein Dialog verlassen werden kann. Die meisten Dialoge haben mindestens zwei Exitbuttons: ›OK‹ und ›Abbruch‹. Öffnen Sie erneut die Objektebox mit der Taste O und ziehen Sie daraus das BUTTON-Objekt auf die rechte untere Ecke des neuen Dialogs. Kopieren Sie dann den neuen Button mit der Maus (anklicken und Shift-Taste drücken) links neben sich selbst.

Öffnen Sie jetzt den rechten Button, ändern seinen Text in ›OK‹ und geben ihm den Namen ›PLOC‹. Sorgen Sie dafür, daß die Buttons für die Flags Selectable, Exit und Default selektiert sind.

›Exit‹ bewirkt, daß die AES-Funktion `form_do()` beim Anklicken eines solchen Buttons verlassen wird.

›Default‹ bedeutet, daß die Funktion dieses Buttons auch durch die Return-Taste ausgelöst werden kann, was durch eine dickere Umrandung des Buttons angezeigt wird. Dieses Flag kann logischerweise nur für einen Button pro Dialog vergeben werden. Danach öffnen Sie den linken Button, ändern den Text in ›Abbruch‹ und geben ihm den Namen ›PLCANCEL‹. Schalten Sie die Attribute Selectable und Exit ein.



Der fertige Dialog sollte ungefähr so aussehen, wie ihn die Abbildung hier zeigt.

Testen des Formularbaums

Der Dialog ist jetzt fertig. Wählen Sie Bearbeiten: Objektbaum testen oder Drücken Sie F9, um ihn zu testen. Der Dialog wird dann in der Mitte des Bildschirms gezeichnet.

Klicken Sie nacheinander die Buttons ›Venus‹, ›Erde‹ und ›Mars‹ an. Wenn Sie allen Buttons das Attribut ›Radiobutton‹ gegeben haben, wird immer nur der angeklickte Button selektiert, alle anderen werden sofort deselektiert.

Anschließend drücken Sie die Return-Taste. Da der Button mit dem Text ›OK‹ das Attribut ›Default‹ besitzt, wird er dadurch selektiert. Da er außerdem noch das Attribut ›Exit‹ besitzt, beendet GEM den Dialog und Interface zeigt Ihnen in einer kleinen Alertbox, mit welchem Button der Dialog verlassen wurde. In dieser Alertbox können Sie jetzt wählen, ob Sie den Dialog weiter testen wollen, oder ob Sie das Testen beenden möchten.

Klicken Sie das Fenster des neuen Dialogs in der linken oberen Ecke an, um es zu schließen und die Bearbeitung des Formulars zu beenden.

Löschen von Dateien, Bäumen oder Objekten

Um Dateien, Bäume oder Objekte zu löschen, wählen Sie sie aus und ziehen die selektierten Objekte auf den Papierkorb. Wenn der Papierkorb invertiert wird (d.h. er liegt genau unter der Maus), lassen sie den Mausknopf los.

Das Löschen eines Objektes aus einem Formularfenster kann durch Drücken der Undo-Taste gleich danach rückgängig gemacht werden. Das funktioniert aber nur mit einfachen Objekten, nicht mit ganzen Bäumen oder Dateien - also Vorsicht beim Löschen!

Speichern der Resource-Datei

Um die eben entworfene Resource als Datei zu sichern, wählen Sie die Option Datei: Sichern als... Daraufhin erscheint die Dateiauswahl.

Drücken Sie die Esc-Taste, um das Namen-Eingabefeld, auf dem der Cursor steht, zu löschen und geben Sie den Namen ›DEMO‹ ein. Nach einem Druck auf die Return-Taste speichert Interface die folgenden Dateien: DEMO.RSC: Dies ist die eigentliche Resource-Datei. In ihr sind alle Daten mit Ausnahme der Objektnamen gespeichert. DEMO.HRD: In dieser Datei werden die Namen aller Objekte gespeichert. Interface braucht diese Datei, um die Namen den richtigen Objekten zuzuordnen. Außerdem werden in dieser Datei zur Resource gehörende Parameter wie z.B. das Ausgabeformat gespeichert. DEMO.H: Diese Datei enthält ebenfalls alle Namen und die dazugehörigen Objektnummern, allerdings als einfachen ASCII-Text. Sie kann von einem C Programm mit dem Befehl ›#include‹ übernommen werden. Interface kann Daten für die Programmiersprachen C, Pascal, Modula, Basic (z.B. Omikron) und GFA Ba

sic ausgeben. Außerdem besteht die Möglichkeit, die Ausgabe an eine weitere Programmiersprache (z.B. Assembler oder Fortran) anzupassen. Näheres dazu erfahren Sie am Ende dieses Abschnitts.

Headerdateien werden nur abgespeichert, wenn Labels verändert oder gelöscht wurden, die Objekthierarchie geändert wurde, oder wenn der Resourceinfo-Dialog mit OK verlassen wurde. Das dürfte alle Compilerbenutzer freuen, deren Compiler sonst das komplette Projekt neu compilieren würde.

Im Detail

Im folgenden gehen wir detailliert auf die Bearbeitung einer Resource ein. Als Referenz dient zunächst die Demo-Resource, die im vorangegangenen Übungsteil beschrieben wurde. Falls Sie die Übung nicht nachvollzogen haben, finden Sie auf einer der Interface Disketten eine Resource mit dem Namen ›DEMO.RSC‹, die den gleichen Aufbau hat.

Bearbeiten eines Menüs

Ein GEM-Programm verfügt im allgemeinen über eine Menüzelle, mit der dem Anwender die wichtigsten Optionen zur Verfügung gestellt werden. In unserem Beispiel ist es der Baum mit dem Namen ›MYMENU‹.

Menütitel

Zur Erweiterung eines Menüs um einen neuen Titel öffnen Sie zunächst den entsprechenden Baum durch Doppelklick. Anschließend öffnen Sie mit Bearbeiten: Objekte... die Objektebox und ziehen ein STRING-Objekt mit der Maus an die gewünschte Stelle im Menüfenster.

In das Menüfenster können nur STRING-Objekte gezogen werden. Wenn Sie unbedingt Icons oder ähnliches in der Menüzelle haben wollen (welche da aber eigentlich nicht hingehören!), müssen Sie das Fenster schließen, das Symbol des Menübaums selektieren und ihn mit Datei: Info... in einen Formularbaum umwandeln. Danach sind Sie aber für die komplette Gestaltung dieses Baums selbst verantwortlich, d.h. Interface kann ein korrektes Format darin nicht mehr sicherstellen bzw. überwachen. Wenn Sie den Text eines Menütitels ändern wollen (immer nur ›STRING‹ ist ja nicht sehr informativ) klicken Sie mit einem Doppelklick auf den entsprechenden Titel. Es erscheint der Dialog für die String-Bearbeitung, in dem Sie den Namen des Objektes und den Text ändern bzw. eingeben können. Leerzeichen vor und hinter dem Text werden bei eingeschalteter Leerzeichenoptimierung (siehe Referenz, Optionen: Einstellungen...) von Interface selbständig gesetzt, so daß Sie sich darum nicht kümmern müssen. Die Flags und der Status, die in diesem Dialog eingestellt werden können, sind für Menüzellen ohne Bedeutung.

Zur Veränderung der Reihenfolge der Menütitel klicken Sie auf einen Titel und schieben ihn grob an seine neue Position, also z.B. auf den Titel, an dessen Stelle er künftig erscheinen soll. Eine Überschrift kann nur innerhalb ihrer Menüzeile oder auf den Papierkorb verschoben werden. Das dazugehörige Pulldown-Menü wird dabei mitverschoben. Beim Verschieben auf den Papierkorb wird auch das Pulldown-Menü gelöscht.

Beim Kopieren eines Titels durch das Verschieben mit gedrückter Shift-Taste oder mit den Optionen des Bearbeiten-Menüs wird das dazu gehörige Pulldown-Menü nicht (!) kopiert, weil es zu einer anderen Objekthierarchie gehört. Titel können Sie auch in Formularbäume oder das Pulldown-Menü eines anderen Menübaums kopieren.

Menüeinträge

Ein Pulldown-Menü wird geöffnet, wenn Sie den dazugehörigen Titel mit der Maus anklicken.

Um einen neuen Menüeintrag zu erzeugen, öffnen Sie die Objektebox und ziehen ein STRING-Objekt mit der Maus an die gewünschte Stelle im Pulldown-Menü (In ein Menüfenster können nur STRING-Objekte gezogen werden.). Zum Ändern des Textes einer Menü-Option öffnen Sie mit einem Doppelklick auf den zu ändernden Eintrag den Dialog zur String-Bearbeitung, in dem Sie den Namen des Eintrags und seinen Text ändern bzw. eingeben können.

Leerzeichen vor und hinter dem Text werden bei eingeschalteter Leerzeichenoptimierung (siehe Referenz, Optionen: Einstellungen...) von Interface selbständig gesetzt, so daß Sie sich darum nicht mehr kümmern müssen. Dabei werden vor dem Eintrag immer zwei Leerzeichen eingefügt. Ist darüber hinaus die Option Menü-Tastaturkürzel anordnen (im gleichen Dialog) ausgewählt, werden auch die eventuell eingetragenen Shortcuts automatisch rechtsbündig angeordnet. Interface berücksichtigt folgende Regeln zur Erkennung:

- Der Shortcut startet hinter dem ersten Leerzeichen, das von rechts gefunden wird.
- Vor der Tastenangabe muß mindestens ein Kontrollzeichen für Shift, Control oder Alternate stehen, oder es folgt sofort die Tastenangabe; dann sind aber nur Sondertasten wie Help, Insert usw. erlaubt.
- Nach dem oder den Kontrollzeichen muß eine 1 Zeichen lange Tastenangabe folgen, oder eine Angabe einer Sondertaste.

Wenn diese Bedingungen erfüllt sind, werden die Shortcuts nach folgenden Regeln angeordnet:

- Vor jedem Shortcut müssen mindestens 2 Spaces stehen.
- Steht am Textende ein ›...‹ (z.B. in ›Öffnen...‹), dann reicht in dieser Zeile 1 Space vor dem Shortcut aus.

Die erlaubten Zeichen für die Kontrolltasten stehen in der Interface Resource im Free String ›SHORTCUT‹, der von Ihnen geändert und in geänderter Fassung gespeichert werden kann. Die erlaubten Sondertasten stehen im Free String ›SH_KEYWORDS‹. Vor und hinter jeder Sondertaste in diesem Free String muß ein ›/‹ stehen.

Dadurch sind fast alle denkbaren Shortcuts von Interface erkennbar. Einzige Ausnahme ist ein einzelner Buchstabe, vor dem kein Kontrolltastenzeichen steht. Wir haben diese Möglichkeit absichtlich ausgeklammert, damit Interface nicht versehentlich ein Zeichen aus einem Menüeintrag ohne Shortcut falsch anordnet (z.B. "Bild 1"). Ein Eintrag kann nur innerhalb des Pulldown-Menüs oder auf den Papierkorb verschoben werden, kann aber auch in Formularbäume, in die Menüzeile oder Pulldownboxen eines anderen Menübaums kopiert werden.

Bearbeiten eines Dialogs

Dialoge stellen die am häufigsten genutzte Form der Kommunikation zwischen Anwender und Programm dar. In unserem Beispiel gibt es nur einen Dialog, den Baum mit dem Namen ›PLANET‹. Um ihn zu ändern öffnen Sie mit einem Doppelklick das zugehörige Formularfenster.

Allgemeines

Zum Hinzufügen eines neuen Objektes öffnen Sie die Objektebox und ziehen ein Objekt des benötigten Typs in das Formularfenster. Wenn ein Objekt geändert werden soll (z.B. nach einem Doppelklick darauf), erscheint ein Dialog, in dem die Daten des Objektes nicht nur angezeigt werden, sondern auch editiert werden können. Alle Dialoge ähneln sich im Aufbau, Unterschiede ergeben sich nur dadurch, daß einige Objekte nicht alle Attribute haben können. So kann z.B. ein Button weder Farbe noch Muster haben, außerdem kann die Randstärke seines Rahmens nicht eingestellt werden (weil es für GEM nicht vorgesehen ist).

Im folgenden werden alle Einstellungsmöglichkeiten besprochen. Falls eine der aufgezählten Einstellungsmöglichkeiten bei einem bestimmten Objekt nicht vorhanden sein sollte, können Sie dem selektierten Objekttyp dieses Attribut nicht geben.

Name:

Dieses Feld enthält den Namen des Objekts, unter dem es später von einer Programmiersprache angesprochen werden kann. Alle Objekttypen können mit einem Namen versehen werden.

Objekttyp:

Dieser Eintrag enthält ein Popup-Menü, mit dem der Objekttyp nachträglich geändert werden kann. Der Typ kann aber nur in ähnliche Typen (z.B. STRING in BUTTON oder TITLE) umgeändert werden, damit bei der Umwandlung keine Daten verloren gehen. Dieser Eintrag ist in allen Dialogen vorhanden. Wollen Sie ein Objekt in einen fremden Objekttypen (z.B. STRING in TEXT) umwandeln, müssen sie den Dialog verlassen und das Objekt anklicken, woraufhin ein (abschaltbares) Popup-Menü erscheint. Wenn eine Typumwandlung möglich ist, erscheint in der letzten Zeile des Popups die Bezeichnung der jeweils anderen Objektart. Durch Auswahl dieses Eintrags wird das Objekt umgewandelt. Dabei können aber u.U. Daten des Objektes verloren gehen.

Erweiterter Typ:

GEM erlaubt dem Programmierer, auch eigene Objekttypen zu benutzen. Damit man verschiedene selbstdefinierte Objekttypen benutzen kann, werden sie durchnummeriert. Ein eigener Objekttyp kann eine Nummer zwischen 1 und 255 haben. Diese Nummer wird in der OBJECT-Struktur im Highbyte von ob_type eingetragen. Interface besitzt eine Schnittstelle, die es dem Benutzer erlaubt, eigene Routinen zum Zeichnen solcher Objekte in die Testfunktion einzubinden. Wie man eigene Objekttypen definiert und benutzt, können Sie z.B. in dem Buch ›Vom Anfänger zum GEM Profi‹ nachlesen. Das Einbinden eigener Routinen zur Darstellung erweiterter Objekttypen in Interface wird weiter hinten in diesem Handbuch erläutert. Flags: Die Flags beschreiben die Eigenschaften eines Objektes. Da ein Objekt mehrere Eigenschaften haben kann - teilweise haben muß - sind die Flags durch Checkboxes auswählbar. Auf die verschiedenen Flags werden wir gleich noch detailliert eingehen.

Status:

Der Status beschreibt die Darstellung bzw. den Zustand eines Objektes. Da auch hier mehrere Eigenschaften kombiniert werden können ist auch der Status durch Checkboxes zu definieren (dazu gleich mehr).

Extended Flags:

GEM nutzt nur ein Teil der möglichen Bitkombinationen für Flags und Status. Mit dem Extended Flags-Button können Sie einen Dialog öffnen, in dem Sie (für eigene Objekttypen) die ungenutzten Bits setzen können.

Farben und Hintergrund (Muster):

Die Farben einzelner Objektelemente sowie das Füllmuster des Hintergrundes können in vier Popup-Menüs eingestellt werden. Die Farbe kann dabei getrennt für Text, Rahmen und Füllmuster eingestellt werden.

Wenn Interface in einer Farbauflösung gestartet wurde, werden die verfügbaren Farben in den Popups angezeigt. Ansonsten kann die Farbe an den Texten des Menüs abgelesen werden, welche sich an der GEM-Standardeinstellung orientiert. Text: Objekte, die Text enthalten, können hier mittels Popup-Menüs die Attribute Großschrift, Kleinschrift, Linksbündig, Zentriert, Rechtsbündig, Deckend und Transparent bekommen. Rechts neben den aktuellen Einträgen befindet sich ein kleiner Kreis, der Circlebutton genannt wird. Wenn Sie ihn anklicken, wird der jeweils nächste Popup-Eintrag ausgewählt. Rahmen: Mit diesem Popup können Sie die Dicke des Rahmens eines Objektes festlegen und bestimmen, ob der Rahmen im Innern oder Äusseren des Objektes dargestellt werden soll. Texteingabe: Der Text eines Objekts kann mit den in der Einführung beschriebenen Editiermöglichkeiten eingegeben werden. Wenn das Eingabefeld länger als eine Zeile sein sollte, werden beide Zeilen nach der Eingabe von Interface zu einer einzigen, längeren Zeile zusammengefaßt.

Maske:

Die Möglichkeit, eine Eingabemaske vorzugeben, besteht nur für TEXT-Objekte und ist auch dort nur für die Objekttypen FTEXT und FBOXTEXT sinnvoll. Wenn diese Objekttypen das Flag EDITABLE bekommen, können in solchen Objekten Tastatureingaben gemacht werden, wobei in der Maske festgelegt wird, welche Zeichen bei der Eingabe akzeptiert werden. Außerdem kann die Maske Textteile zur Formatierung der Eingabe enthalten. Ein Beispiel: Um ein Eingabefeld zu erzeugen, in dem nur Zahlen erlaubt sind und vor dem der Text ›Geld:‹ steht sowie der Wert ›32.60‹ vorgegeben ist, müssen Text- und Maskenfeld folgende Inhalte haben: Im Textfeld: ›3260‹

Im Maskenfeld: `field: 99.99` : Um die hellen Neunen einzugeben, muß man während der Eingabe die Alternate-Taste gedrückt halten. Nach dem Verlassen des Dialogs wird das Objekt als ›Geld: 32.60‹ dargestellt. Im Textfeld steht immer der Text, der später in die Platzhalter (im obigen Beispiel die hellen Neunen) eingesetzt werden soll. Es muß ein Text eingegeben werden, der mindestens so lang ist wie alle verwendeten Platzhalter. Wenn Sie nicht möchten, daß dieser Text angezeigt wird, müssen Sie als erstes Zeichen einen Klammeraffen ›@‹ eingeben. GEM zeigt dann alle nachfolgenden Buchstaben nicht an und setzt den Cursor auf das erste Zeichen. Der Text muß eingegeben werden, da er später durch die Eingabe des Benutzers ersetzt wird. Wird der Text vergessen, schreibt GEM einfach alle vom Benutzer eingegebenen Zeichen an andere Speicherstellen und kann so u.U. ein Programm zum Absturz bringen.

gen. Interface versucht zwar, diesem durch automatische Eingabe von Null-Bytes zuvorzukommen, aber das funktioniert z.B. nicht, wenn Sie die Resource-Datei mit einem anderen Resource-Editor weiterverarbeiten.

Alle im Maskenfeld normal eingegebenen Zeichen werden später auch normal angezeigt. Dagegen werden die mit Alternate eingegebenen Zeichen im Maskenfeld hell dargestellt und bestimmen die zulässigen Eingabedaten. Sie werden später durch den Inhalt des Textfelds ersetzt. Als Platzhalter sind folgende Zeichen zulässig:

- ›9‹ nur Ziffern sind erlaubt (Kann unter TOS 1.0 durch einen Fehler im Betriebssystem bei der Eingabe eines Unterstriches ›_‹ zum Absturz führen).
- ›A‹ nur Großbuchstaben und Leerzeichen
- ›a‹ nur Buchstaben und Leerzeichen
- ›N‹ Großbuchstaben, Ziffern und Leerzeichen
- ›n‹ Buchstaben, Ziffern und Leerzeichen
- ›F‹ Alle Zeichen, die zu einem Filenamen gehören dürfen und zusätzlich ›?‹
- ›f‹ Alle Zeichen, die zu einem Filenamen gehören dürfen
- ›P‹ Alle Zeichen, die zu einem Pfadnamen gehören dürfen
- ›p‹ Wie ›P‹, allerdings ohne ›?‹ und ›*‹
- ›X‹ Alle Zeichen sind erlaubt
- ›x‹ Wie ›X‹, die Zeichen werden aber in Großbuchstaben gewandelt

Alle Platzhalter müssen, wie bereits oben erwähnt, mit gedrückter Alternate-Taste eingegeben werden.

Im Boxtext-Dialog sind für te_pvalid (die Eingaben für die Maske) die Zeichen ›A‹, ›Z‹ und ›a‹-›z‹ zugelassen, damit man für eigene benutzerdefinierte Objekte neue Eingabetypen verwenden kann. Von Interface, bzw. den MyDials werden nur die oben aufgeführten Platzhalter unterstützt.

In jeder Dialogbox kann man durch den Button Nächstes direkt von einem Objekt zum nächsten gelangen. Dabei werden erst alle Daten der Dialogbox übernommen, bevor darin die Daten des nächsten Objekts angezeigt werden.

Der Button HELP zeigt eine Hilfeseite an, in der alle erweiterten Objekttypen der EXTOBFIX-Datei der MyDials erläutert werden. Im BoxText-Dialog wird außerdem noch eine Hilfeseite über alle te_pvalid-Platzhalter der TEDINFO-Struktur angezeigt.

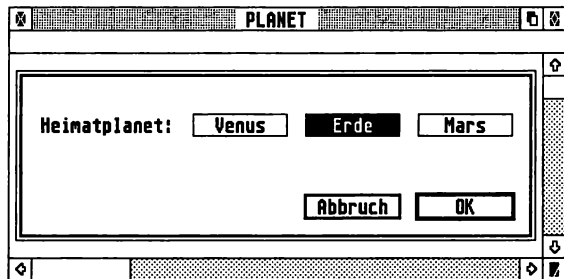
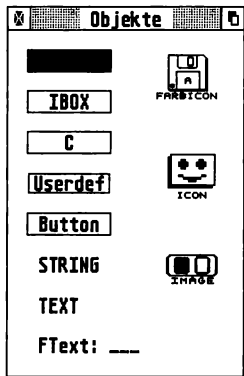
Objekttypen und die Objektkbox

GEM kennt mit den neuen Farbicons des TOS 4.01 insgesamt vierzehn verschiedene Objekttypen, die Sie entsprechend in der Objektkbox von Interface wiederfinden.

Objekttyp	Bedeutung
G_BOX	Rechteckige Box zur Darstellung gerasterter oder farbiger Flächen z.B. im Slider einer Listbox.
G_TEXT	Formatierter Text in zwei Größen (in der kleinen und in der »normalen« Systemschrift) und drei Ausrichtungen (linksbündig, zentriert und rechtsbündig). Die Ausrichtung ist möglich, weil die Objektgröße größer sein kann als die vom Text beanspruchte Fläche.
G_BOXTEXT	Entspricht G_TEXT mit dem Unterschied, daß um den Text noch eine Box gezeichnet wird.
G_IMAGE	Eine monochrome Grafik ohne Maske.
G_USERDEF	Dieser Objekttyp erlaubt es, eigene Funktionen zur Ausgabe zu definieren. Die MyDials benutzen sie bei mehreren Objekttypen (Checkbox, Radiobutton, Dialogtitel, Farbicons u.a.).
G_IBOX	Unsichtbares Rechteck ohne innere Fläche, so daß es nur sichtbar ist, wenn der Rahmen eine Stärke ungleich Null hat. Es wird zum Zusammenfassen mehrerer Objekte benutzt.
G_BUTTON	Zentrierter Text in einem Rechteck, dessen Stärke von den Flags DEFAULT, EXIT und TOUCHEXIT abhängig ist.
G_BOXCHAR	Ein Rechteck, das einen einzigen Buchstaben enthält. Im Dialog zur Objektdefinition (s.o.) erscheint das Eingabefeld Zeichen: an der Stelle, an der sonst die Textoptionen stehen.
G_STRING	Eine Zeichenkette ohne die Attribute der G_TEXT- und G_BOXTEXT-Objekte.
G_FTEXT	Formatierter Text in der »normalen« Systemschrift, der durch den Anwender editiert werden kann.
G_FBOXTEXT	Entspricht G_FTEXT mit dem Unterschied, daß um den Text noch eine Box gezeichnet wird.
G_ICON	Eine monochrome Grafik mit Maske.
G_TITLE	Der Titel eines Drop-Down-Menüs.
G_CICON	Ein Farbicon.

Der Objekttyp `G_USERDEF` wird direkt von Interface unterstützt. Dazu legt Interface eine `OBJECT`-Struktur an, deren `ob_spec`-Pointer auf eine `USERBLK`-Struktur zeigt. Für jedes Objekt wird in der Resource eine eigene `USERBLK`-Struktur angelegt.

`ub_parm` kann direkt in Interface eingegeben werden. `ub_code` enthält einen `NULL`-Pointer, d.h. nach dem Einladen einer solchen Resource muß man als erstes in `ub_code` aller Userdef-Objekte einen Zeiger auf die eigene Zeichenroutine eintragen. Sonst würden die AES beim Zeichnen eines solchen Objekts sofort abstürzen.



Die Unterstützung selbstdefinierter Objekte wird durch die Möglichkeit, eigene Objekte in die Objektkiste von Interface einzubinden (FlyDial-Ecke, OK-Button, ...), noch erweitert. Dazu lädt man die Interface-Resource und kopiert das gewünschte Objekt in den `OBJEKTE`-Baum. Die Sortierreihenfolge ist egal, es kommt nur darauf an, daß das neue Objekt in der inneren Box landet, in der sich auch die anderen Objekte befinden. Das neue Objekt muß ein direktes Child der inneren Box werden und darf selbst keine Childs enthalten.

Es wird nach dem Start von Interface nicht als ein `G_USERDEF`-Objekt angezeigt, damit nicht versehentlich von der Anpassungsroutine Daten verändert werden (z.B. Objektgröße, Flags, ...).

Wenn die Objektkiste in einem Fenster liegt, kann man auch dann Objekte herausziehen, wenn die Box im Hintergrund liegt. Dazu müssen Sie das Objekt mit der linken und rechten Maustaste gleichzeitig anklicken. Unter MultiTOS entfällt das gleichzeitige Drücken der rechten Maustaste.

Objektflags

Die Flags eines Objektes bestimmen seine Eigenschaften. Ein Objekt kann mehrere Eigenschaften haben, also z.B. auswählbar (SELECTABLE) sein und mit der Auswahl den Dialog beenden (EXIT). Sie sind hier in einer Übersicht zusammengefaßt.

Objektflag	Bedeutung
SELECTABLE	Der Benutzer kann das Objekt durch Anklicken anwählen.
DEFAULT	Hebt BUTTON-Objekte optisch hervor und bedeutet, daß das Drücken von Return mit dem Anklicken dieses Objekts gleichgesetzt wird (bei Aufruf der AES-Funktion ›form_do()‹). Innerhalb einer Dialogbox sollte man nur bei einem Objekt dieses Flag setzen.
EXIT	›form_do()‹ wird beendet, wenn der Benutzer das Objekt anklickt.
EDITABLE	Das Objekt ist editierbar. Dieses Flag darf nur FTEXT und FBOX-TEXTObjekten gegeben werden. Wird dies mißachtet, stürzen die AES beim Ausführen der Dialogbox ab!
RBUTTON	Das Objekt ist ein sogenannter Radioknopf (Radiobutton). Dieser ist vergleichbar mit den Stationswahltasten eines Radios (daher der Name), von denen immer nur eine zur Zeit gedrückt sein kann. Auf die AES übertragen bedeutet dies, daß von allen Objekten, bei denen dieses Flag gesetzt ist und die in der selben Hierarchieebene eines Objektbaums stehen, nur eines ausgewählt sein kann. Wird ein anderes angeklickt, werden alle anderen automatisch zurückgesetzt.
TOUCHEXIT	Wird ein solches Objekt mit gedrückter Maustaste berührt, bricht die Funktion ›form_do()‹ sofort ab, ohne daß erst der Mausknopf wieder losgelassen werden muß.
Extended	Diese Flags werden von GEM nicht verwendet, und dürfen für eigene Zwecke (z.B. erweiterte Objekttypen) verwendet werden.

Objektstatus

Der Status eines Objektes bestimmt seinen Zustand . Ein Objekt kann mehrere Zustände haben, also z.B. ausgewählt (SELECTED) sein und schattiert gezeichnet (SHADOWED). Sie sind hier in einer Übersicht zusammengefaßt.

Objektstatus Bedeutung

SELECTED	Das Objekt wird invertiert dargestellt. Dieser Status wird automatisch bei dem Objekt gesetzt, mit dem ein Dialog verlassen wurde.
CROSSED	Im Inneren des Objektes wird ein weißes Kreuz gezeichnet.
CHECKED	In der linken, oberen Ecke des Objektes wird ein Häkchen gezeichnet.
DISABLED	Das Objekt wird hell gezeichnet und ist nicht anwählbar.
OUTLINED	Um das Objekt wird ein zusätzlicher Rahmen gezeichnet.
SHADOWED	Unter dem Objekt wird ein Schatten gezeichnet, der rechts und unter dem Objekt sichtbar ist. Buttons die mit diesem Status gezeichnet werden weisen auf ein Popup hin.
DRAW 3D	Nur für ICONBLK-Strukturen und auch nur ab GEM 2.0 wird ein dreidimensionaler Effekt erzeugt, indem das Icon dreimal schräg übereinander gezeichnet wird. DRAW 3D wird von keiner TOS-Version unterstützt.
WHITEBAK	Ebenfalls nur für ICONBLK-Strukturen und erst ab GEM2.0: bei weißem Hintergrund wird die Icon-Maske nicht mitgezeichnet. WHITEBAK funktioniert auch auf allen für den Atari erhältlichen TOS-Versionen, aber wurde von Atari niemals dokumentiert. Der Atari Desktop verwendet selber WHITEBAK in Fenstern um die Bildschirmausgabe zu beschleunigen.

Kopieren und verschieben

Das Verschieben eines Objektes ist vollkommen unkompliziert. Ziehen sie es einfach mit der Maus auf seine neue Position. Das war's.

Beim Verschieben eines Objekts kann man das Objekt jetzt »unter« ein anderes Objekt schieben. Es wird dem Zielobjekt nicht automatisch als Child zugeordnet, wenn man auf die Alertbox „Soll die Objektstruktur geändert werden?“ mit Nein antwortet. Dieser Alert erscheint nur, wenn das verschobene Objekt und das Zielobjekt ein gemeinsames Parent besitzen. Dadurch soll verhindert werden, daß ein Objekt versehentlich außerhalb des Parents liegen kann.

Wenn man mehrerer Objekte innerhalb eines Fensters verschiebt, bleiben die Objekte auch nach der Verschiebeaktion selektiert, damit man sie ggf. noch weiter verschieben kann und nicht wieder alle neu selektieren muß. Außerdem läßt Interface dabei die Objektreihenfolge unverändert, nur die Verkettung wird u. U. geändert.

Wenn ein oder mehrere Objekte kopiert werden sollen, verschieben Sie die Objekte an eine andere Position und drücken beim Loslassen der Maustaste eine Shift-Taste. Die Kopieraktion wird nicht durchgeführt, wenn das Objekt nicht mindestens um ein Pixel verschoben wurde.

Selektieren mehrerer Objekte

Um mehrere Objekte gleichzeitig für die Bearbeitung auszuwählen, gibt es zwei Möglichkeiten. Erstens: Halten Sie die Shift-Taste gedrückt und klicken Sie alle gewünschten Objekte an. Zweitens: Halten Sie die Alternate-Taste gedrückt und ziehen Sie einen Rahmen um die gewünschten Objekte auf. Nach dem Loslassen der Maustaste werden alle vollständig im Rahmen liegenden Objekte selektiert. Mehrere selektierte Objekte lassen sich gemeinsam verschieben, kopieren, löschen und öffnen. Durch ein gemeinsames Öffnen (z.B. mit F2) können alle selektierten Objekte ohne erneutes Anklicken nacheinander abgearbeitet werden. Außerdem können bei mehreren selektierten Objekten die Objektflags gleichzeitig geändert werden (siehe Referenz, Bearbeiten: Flags setzen...).

Das Popup-Menü: Noch mehr Funktionen

Wenn ein Objekt nur einmal angeklickt wird, erscheint ein Popup-Menü, das einige besondere Funktionen zur Verfügung stellt. Mit der Option Objekt-Popup im Optionen: Einstellungen...-Dialog läßt sich es sich allerdings auch ausschalten. Alle darin enthaltenen Funktionen können nämlich auch über die Menüleiste im Menü Objekt erreicht werden. Dadurch ist das Selektionsverfahren von Interface »kompatibler« zu anderen Programmen (ein Klick auf ein Objekt selektiert es, anstatt das Popup erscheinen zu lassen).

Über die Option Löschen im Objekt-Menü lassen sich auch Bäume aus einem Resource-Datei Fenster löschen.

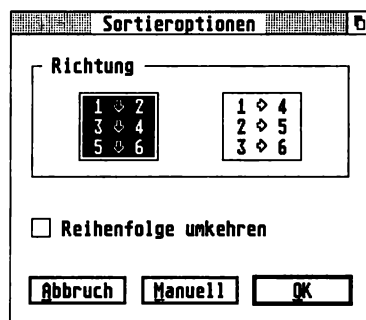
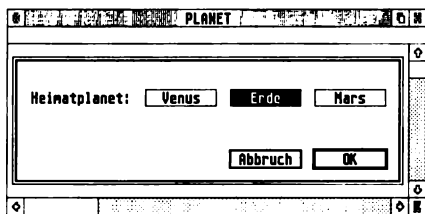
Editieren

Hat die gleiche Funktion wie Datei: Öffnen, Objekt: Editieren bzw. ein Doppelklick auf ein Objekt. Dieser Eintrag ist für den Fall vorhanden, daß Ihnen der Doppelklick mißlingen sollte.

Sortieren: Ordnung ist das halbe Leben

GEM zeichnet die Objekte eines Dialogs immer in der Reihenfolge, in der Sie sie in den Dialog kopiert haben. Wenn Sie nun als letztes die Überschrift hereinkopiert hätten, dann würde diese Überschrift, obwohl sie ganz oben steht, auch immer als letztes gezeichnet werden. Das sieht natürlich nicht besonders gut aus, und um das zu verhindern, kann man die Reihenfolge aller Objekte eines Dialogs sortieren lassen. Außerdem ist die Reihenfolge der Objekte auch für die Programmierung von Bedeu-

tung, denn durch die Sortierung kann man erreichen, daß hinter- oder untereinander liegende Objekte eine durchgehende Numerierung erhalten. Dadurch ist es möglich, im Programm nicht alle diese Objekte einzeln abzufragen, sondern zu testen, ob ein Objektindex zwischen dem ersten und dem letzten Objektindex liegt. Alle Ebenen eines Dialog müssen einzeln sortiert werden. In unserer Box (>PLANET<.) gibt es davon zwei. Die erste Ebene bildet der große Rahmen (das Root-Objekt), in dem sich alle anderen Objekte befinden. Die zweite Ebene wird von dem IBOX Objekt gebildet, da in ihm weitere Objekte enthalten sind. Das bedeutet, daß das Sortieren nur die Children des gewählten Objektes betrifft. Die Children von Children werden nicht mit-sortiert. Dadurch könnten z.B. die Objekte des Root-Objektes von oben nach unten gezeichnet werden, während die Objekte in der IBOX von links nach rechts gezeichnet werden.



Zum Sortieren klicken Sie das Root-Objekt einmal an und wählen in dem folgenden Popup-Menü den Eintrag Sortieren an. Der nun erscheinende Dialog (s.o.) bietet Ihnen zwei Möglichkeiten, die durch Pfeile symbolisiert werden. Entweder können alle Children des selektierten Objektes von oben nach unten (linkes Feld) oder von links nach rechts (rechtes Feld) sortiert werden. Der Button Reihenfolge umkehren kehrt (nomen est omen) die Reihenfolge um, d.h. die Children werden von unten nach oben oder von rechts nach links sortiert.

Für ganz spezielle Fälle gibt es außerdem die Option Manuell, mit der die Reihenfolge beliebig definiert werden kann. Dazu beendet Interface den Dialog und zeigt alle in Frage kommenden Objekte invers an. Sortiert wird jetzt in der Reihenfolge, in der Sie die Objekte mit der Maus anklicken. Ein so ausgewähltes Objekt wird wieder »normal« dargestellt, so daß Sie eine Übersicht haben, welche Objekte noch ausgewählt werden müssen. Wichtig: es müssen alle betroffenen Objekte, also alle Children des Parent-Objektes, angeklickt werden, um die Funktion zu beenden. Zum vorzeitigen Abbruch drücken Sie Esc oder die rechte Maustaste.

Für unsere Übung wählen Sie den linken Button (sortieren von oben nach unten) und beenden den Dialog mit OK. Klicken Sie das unsichtbare IBOX-Objekt mit den Radiobuttons an. Wenn Sie es nicht finden, klicken Sie am besten zwischen den ›Venus‹- und den ›Erde‹-Button. Danach wählen Sie in dem Popup wieder den Eintrag ›Sortieren‹ an und beenden den Sortieren-Dialog mit OK.

Zentrieren

Ein Objekt kann hiermit horizontal und/oder vertikal (relativ zum Parent) zentriert werden. Außerdem ist es in dem angezeigten Dialog möglich, die Objektgröße an die Größe des Parent-Objektes anzupassen zu lassen.

Einrasten

Wenn ein Objekt nicht genau auf Zeichenkoordinaten steht, kann es mit dieser Funktion auf eine entsprechende Position gebracht werden.

Verstecken

Mit dieser Option wird in einem Objekt das HIDETREE-Flag gesetzt, das im »normalen« Editier-Dialog nicht verändert werden kann. Dadurch werden das Objekt und alle seine Children versteckt und sind weder sichtbar noch mit der Maus wählbar. Eingaben in EDITABLE FTEXT-Objekte sind aber weiterhin möglich, werden aber nicht mehr auf dem Bildschirm angezeigt. Dieses Feature (oder Bug?) läßt sich u.a. für Paßworteingaben verwenden.

Um ein verstecktes Objekt wieder sichtbar zu machen, müssen Sie die folgende Funktion verwenden.

Aufdecken

Dieser Popup-Eintrag erscheint nur, wenn sich in dem angeklickten (Parent-) Objekt irgendwelche (Child-) Objekte befinden, in denen das HIDETREE-Flag gesetzt ist (siehe oben). Nach dem Anwählen des Eintrags werden sie wieder angezeigt.

Plazieren

Hiermit können Objekte durch direkte Koordinateneingabe plaziert werden. In dem erscheinenden Dialog kann man alle Koordinaten (x, y, Breite, Höhe) selbst eintragen. Interface ignoriert dabei aber unsinnige Werte. Außerdem gibt es noch einige weitere Buttons im Dialog mit folgender Bedeutung:

In der Gruppe relativ zu die Radiobuttons Parent-Objekt und Root-Objekt. Ist Parent-Objekt ausgewählt beziehen sich die Koordinatenangaben auf das Parent des Objekts. Ist Root-Objekt ausgewählt, beziehen sie sich auf das Root-Objekt, d.h. sie sind relativ zum gesamten Baum.

In der Gruppe Eingabe in bewirkt der Button Zeichenraster, daß die Objektkoordinaten im Zeichenraster (Buchstabenbreite/Höhe, i. A. also 8 x 16) ausgewertet werden. Hinter dem Querstrich kann die Abweichung davon in Pixeln eingegeben werden. Ist der Button Pixelraster ausgewählt, werden die Objektkoordinaten in Pixel ausgewertet.

Entfernen

Das selektierte Objekt wird gelöscht, aber alle seine Children bleiben im Dialog. Dabei ist kein Undo möglich!

Löschen

Das selektierte Objekt und alle seine Children werden gelöscht. Das Löschen läßt sich durch die Undo-Taste wieder rückgängig machen.

Objekttyp ändern

Das Objekt kann in einen anderen Objekttyp umgewandelt werden. Es sind folgende Umwandlungen möglich:

STRING in TEXT

BUTTON in BOXTEXT

IMAGE in ICON und umgekehrt. Dabei können aber Daten verloren gehen, da die einzelnen Objektformate zum Teil stark voneinander abweichen. Bei der Umwandlung ICON zu IMAGE gehen z.B. der Buchstabe, der Icontext und die Maske des Icons verloren.

Last not least: Esc und Undo

Manchmal kann es dazu kommen, daß GEM einen Dialog in einem Fenster nicht korrekt zeichnet. Beim Hoch- oder Herunterscrollen kann es z.B. passieren, daß der Rand eines Dialogs plötzlich statt 2 Pixel nur noch 1 Pixel dick ist. Das ist kein Fehler von Interface, sondern eine etwas seltsame Angewohnheit vom GEM. Wenn Sie dieser (nur sehr selten auftretende) Fehler stören sollte, können Sie das Neuzeichnen des kompletten Fensters mit der Esc-Taste erzwingen. Diese Taste ist auch besonders praktisch, wenn man nach dem Sortieren eines Dialoges den neuen Dialogaufbau (die Wirkung beim Zeichnen) überprüfen will. Verschiebeoperationen innerhalb des selben Fensters sowie Löschen und Größenänderungen von Objekten kann man gleich danach mit der Undo-Taste rückgängig machen. Da die Undo-Funktion von Interface aber nur die gelöschten bzw. verschobenen Objekte an ihre alte Bildschirmposition zurückkopiert, wird die Objektreihenfolge nicht unbedingt auf den alten Stand gebracht. Deshalb sollte man sicherheitshalber nach einem Undo den Dialog neu sortieren. Wenn man ein Objekt vergrößert hat und dabei ein neues Objekt adoptiert wurde, gehört das adoptierte Objekt nach der Undo Operation immer noch

zu dem vergrößerten Objekt. Daher kann das Parent-Objekt nur teilweise wieder auf die alte Größe gebracht werden, da das adoptierte Objekt nicht aus dem Parent-Objekt herausragen kann.

Bearbeiten eines Alerts

Interface stellt alle Daten einer Alertbox innerhalb eines speziellen Dialogs dar. Im oberen Teil kann man das gewünschte Icon wählen, darunter maximal 5 Textzeilen und 3 Buttontexte eingeben.

GEM kann eine Alertbox mit drei unterschiedlichen Icons darstellen: Ausrufungszeichen, Fragezeichen und Stoppschild. Außerdem kann auch auf ein Icon verzichtet werden. Diese vier Möglichkeiten können direkt ausgewählt werden. Zusätzlich gibt es einen fünften Button, in dem eine Zahl aus dem 36er Dezimalsystem (0-Z) eingegeben werden kann. Die Werte Eins bis Drei entsprechen dabei den oben genannten Icons, während Null keinem Icon entspricht. Die weiteren Werte sollten Sie nur dann eingeben, wenn Sie eigene Routinen zur Darstellung von Alertboxen verwenden (z.B. FlyDials oder MyDials). GEM kommt mit höheren Werten als 3 nicht zurecht und zeigt entweder eine kaputte Grafik innerhalb der Alertboxen an oder stürzt sofort ab. Die 5 Textzeilen des Alert-Dialoges bilden genau genommen ein einziges großes Eingabefeld. Wenn Sie mit dem Cursor am Ende einer Zeile angelangt sind und weitere Zeichen eingeben, erscheinen Text und Cursor in der nächsten Zeile. Die Eingabetasten Delete und Backspace arbeiten ebenfalls zeilenübergreifend. Dadurch ist es z.B. möglich, einen Text, der in der zweiten Zeile steht, in die erste Zeile hochzuziehen. Wenn Sie das Gegenteil erreichen wollen, also der Text ab der Cursorposition in die nächste Zeile soll, können Sie Shift Insert drücken.

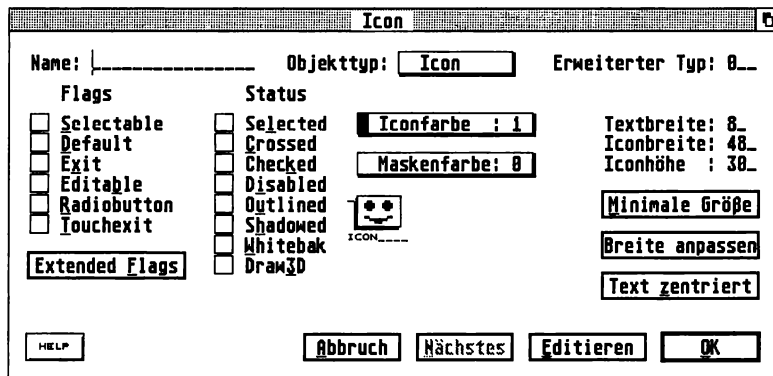
Diese Funktionen können natürlich nicht den Komfort eines Editors bieten, aber sie dürften trotzdem eine große Arbeitserleichterung darstellen. Die gleichen Editiermöglichkeiten gibt es in den mehrzeiligen Eingabefeldern für Buttons und Free Strings.

Im Alert-Dialog gibt es den Button "Test". Dadurch muß man den Dialog nicht extra verlassen, um die Alertbox zu testen.

Bearbeiten eines Icons oder Images

Eines der »Sahnestückchen« von Interface ist sein Iconeditor, mit dem Sie Icons, Farbicons und Images bearbeiten können.

Wenn eines dieser Objekte geändert werden soll (z.B. nach einem Doppelklick darauf), erscheint ein Dialog, in dem die Daten des Objektes nicht nur angezeigt werden, sondern auch editiert werden können. Die Dialoge ähneln sich im Aufbau, Unterschiede ergeben sich nur dadurch, daß Images nicht alle Attribute haben können. So kann z.B. ein Image weder mit Text versehen werden, noch hat es eine Maske.



Im folgenden werden alle Einstellungsmöglichkeiten besprochen. Falls eine der aufgezählten Einstellungsmöglichkeiten bei einem Image nicht vorhanden sein sollte, können Sie ihm dieses Attribut nicht geben. Wir sprechen allgemein von Icons und schließen damit Images und Farbicons ein. Die meisten Einstellungen kennen Sie bereits aus den vergleichbaren Dialogen für andere Objekttypen. Auf sie wird hier nur kurz eingegangen.

Name: Dieses Feld enthält den Namen des Objekts, unter dem es später von einer Programmiersprache angesprochen werden kann.

Objektyp: Dieser Eintrag enthält ein Popup-Menü, mit dem der Objekttyp nachträglich von »Icon« in »Farbicon« und umgekehrt geändert werden kann.

Erweiterter Typ: GEM erlaubt dem Programmierer, auch eigene Objekttypen zu benutzen. Damit man verschiedene selbstdefinierte Objekttypen benutzen kann, werden sie durchnummeriert. Ein eigener Objekttyp kann eine Nummer zwischen 1 und 255 haben. Diese Nummer wird in der OBJECT-Struktur im Highbyte von ob_type eingetragen. **Flags:** Die Flags beschreiben die Eigenschaften eines Objektes. Da ein Objekt mehrere Eigenschaften haben kann - teilweise haben muß - sind die Flags durch Checkboxes auswählbar.

Status: Der Status beschreibt die Darstellung bzw. den Zustand eines Objektes. Da auch hier mehrere Eigenschaften kombiniert werden können ist auch der Status durch Checkboxes zu definieren.

Extended Flags: GEM nutzt nur ein Teil der möglichen Bitkombinationen für Flags und Status. Mit dem Extended Flags-Button können Sie einen Dialog öffnen, in dem Sie (für eigene Objekttypen) die ungenutzten Bits setzen können.

Die Icon-Farbe sowie die Farbe der Maske können in zwei Popup-Menüs eingestellt werden. Wenn Interface in einer Farbauflösung gestartet wurde, werden die verfügbaren Farben in den Popups angezeigt. Ansonsten kann die Farbe an den Texten des Menüs abgelesen werden, welche sich an der GEM-Standardeinstellung orientiert. Unterhalb der Farbeinstellung für die Icon-Maske wird das Icon selbst angezeigt. Wenn das Icon eine bestimmte Größe überschreitet, wird nur dessen oberer linker Teil angezeigt. Zu einem Icon bzw. Farbicon gehören noch zwei Textelemente, nämlich ein Laufwerksbuchstabe und ein erläuternder Text, dessen Länge in Textbreite angegeben wird. Um in den entsprechenden Feldern Einträge vorzunehmen, müssen Sie den Cursor mit den Cursortasten dorthin positionieren. Die Maus kann nicht benutzt werden, weil mit ihr die beiden Felder verschoben werden.

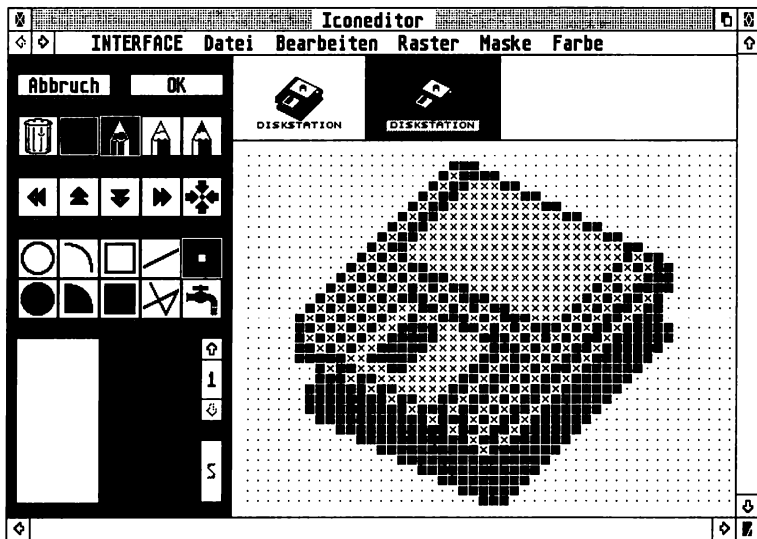
Der (gewöhnlich unter einem Icon stehende) erläuternde Text kann mit dem Button Text zentrieren horizontal so ausgerichtet werden, daß er mittig unter dem Icon steht. Die Textlänge kann mit dem Button 'Breite anpassen' direkt geändert werden, ohne daß man den Icon-Dialog dazu verlassen müßte.

In den Feldern Iconbreite und Iconhöhe können Sie die Größe eines Icons definieren. Da die Breite ein Vielfaches von 16 sein muß, korrigiert Interface Ihre Eingaben unter Umständen so, daß auf das nächste Vielfache abgerundet wird. Das größte mögliche Icon, das Sie mit Interface bearbeiten können, ist 640 x 400 Pixel groß.

Wenn Sie die Größe eines Icons nachträglich verringern, wird die Breite und Höhe so verändert, daß rechts und unten Daten entfernt werden. Interface weist Sie darauf hin, wenn dies zu einem tatsächlichen Bildverlust führen würde (es kann ja auch sein, daß Sie leere Bildteile entfernen).

Um nicht unnötig Speicher zu verbrauchen, können Icons und Images durch den Button Minimale Größe soweit wie möglich verkleinert werden, ohne daß Daten verloren gehen. Dabei wird das Image soweit nach links oben geschoben, bis kein weißer Rand mehr existiert. Dann wird der rechte und untere Rand abgeschnitten. Zuletzt wird das Image wieder zentriert.

Schließlich haben Sie über diesen Dialog Zugang zum Iconeditor. Klicken Sie dazu entweder mit einem Doppelklick auf das im Dialog angezeigte Icon oder auf den Editieren-Button.



Im Iconeditor kann ein Icon geändert, neu gezeichnet oder aus einer Image-Grafik ausgeschnitten werden. Sobald das Fenster geschlossen oder der OK-Button angeklickt wird, werden die neuen Daten übernommen. Abbruch verwirft dagegen die Änderungen.

Die jeweils letzte Änderung des Bildes im Iconeditor kann mit Undo rückgängig gemacht werden.

Der Editor präsentiert sich in einem Fenster mit eigener Menüzeile. Um die Menüzeile zu aktivieren, müssen Sie einen der Titel anklicken. Das Fenster ist in drei Bereiche aufgeteilt: Links finden Sie die Zeichenfunktion und die Farbauswahl, oben eine Anzeige des Icons in Originalgröße, die »Preview« und darunter die eigentliche Zeichenfläche.

Die Preview oberhalb der Zeichenfläche hat zwei Funktionen: Erstens zeigt sie (nomen est omen), wie das Icon in normaler Größe vor verschiedenen Hintergründen aussieht. Zweitens können Sie die Icons (unabhängig voneinander) anklicken und so feststellen, wie das Icon in selektiertem Zustand aussieht.

Zeichnen mit dem Editor

Nachdem der Editor geöffnet wurde, ist automatisch die Option zum Zeichnen einzelner schwarzer Punkte ausgewählt. Wenn Sie jetzt auf der Zeichenfläche ein weißes Feld anklicken, wird es schwarz - halten Sie die linke Maustaste gedrückt und bewegen die Maus, so werden alle anderen Felder, die mit dem Mauscursor »berührt« werden, ebenfalls schwarz. Klicken Sie dagegen auf ein schwarzes Feld, so wird es weiß und alle bei gedrückt gehaltener Maustaste berührten Felder werden ebenfalls weiß.

Sinngemäß funktioniert das Zeichnen von Farbicons. Nachdem Sie eine Farbe ausgewählt haben, klicken Sie im Editor auf ein Feld mit einer (beliebigen) anderen Farbe, um in der ausgewählten Farbe zu zeichnen. Klicken Sie statt dessen auf ein Feld, das bereits in der ausgewählten Farbe angezeigt wird, so löschen Sie dieses Feld. Die aktuell ausgewählte Zeichenfarbe wird links neben dem Mülleimer angezeigt.

Unterhalb des OK-Buttons sehen Sie drei Zeichenstifte, von denen - wenn Sie nichts geändert haben - der linke Stift mit der schwarzen Spitze ausgewählt ist (siehe oben). Er zeigt an, daß Sie mit der ausgewählten Farbe zeichnen und der eben beschriebene Farbwechsel eingestellt ist.

Wählen Sie den Stift mit der weißen Spitze, können Sie nicht mehr zeichnen, sondern nur noch löschen.

Ist der rechte Stift mit der »grauen« Spitze ausgewählt, zeichnen Sie die Maske. Dieser Stift arbeitet wie die Farbstifte, d.h. Sie können mit der gleichen Funktion Maskenpunkte setzen und (beliebige Punkte) löschen.

Exkurs: Maske

Maske? Was'n das? Icons bestehen nicht nur aus dem eigentlich angezeigten Bild. Sie verfügen über ein zweites Bild, das als »Maske« bezeichnet wird. Die Maske wird automatisch dann angezeigt, wenn ein Icon ausgewählt wird. Immer wenn ein Icon selektiert wird, werden alle Punkte dieses Icons, in denen die Maske gesetzt ist, invertiert. Interface zeichnet (für Sie unsichtbar) unter jedem Farbpunkt eines Icons auch einen Maskenpunkt.

Wenn ein Icon nicht selektiert dargestellt wird, werden alle Punkte, in denen nur die Maske sichtbar ist, weiß gezeichnet.

Die drei Stifte werden bei allen Zeichenfunktionen berücksichtigt. Das bedeutet, daß Sie z.B. einen gefüllten Kreis mit einer Farbe zeichnen, ihn löschen oder als Maske zeichnen können. Die folgende Übersicht stellt die verschiedenen Zeichenfunktionen dar.

	Kreise und Ellipsen
	Kreis- bzw. Ellipsenbögen
	Quadrate und Rechtecke
	Linien
	einzelne Punkte
	ausgefüllte Kreise und Ellipsen
	ausgefüllte Kreis- bzw. Ellipsenbögen
	ausgefüllte Quadrate und Rechtecke
	Polygone (Linienzüge)
	Fläche ausfüllen

Alle geometrischen Zeichenfunktionen arbeiten auf die gleiche Weise. Mit dem ersten Mausklick definieren Sie bei Kreisen den Mittelpunkt, bei Bögen, Linien und Polygonen den Startpunkt und bei Rechtecken einen beliebigen Eckpunkt. Nach dem ersten Klick können Sie die Maustaste loslassen und durch Bewegen der Maus die entsprechende Figur bestimmen, die mit dem zweiten Mausklick in den Editor übernommen wird. Das Zeichnen kann vor dem zweiten Mausklick durch einen Klick mit der rechten Maustaste abgebrochen werden. Für das Setzen eines einzelnen Punktes sowie zum Füllen einer Fläche genügt ein einfacher Mausklick.

Die jeweils letzte Änderung des Bildes im Iconeditor kann mit Undo rückgängig gemacht werden.

Zeichenfläche verändern

Die Zeichenfläche des Editors zeigt grundsätzlich eine Vergrößerung des Icons. Dadurch kann es bei großen Icons (und kleinem Monitor) vorkommen, daß ein Icon nicht vollständig sichtbar ist. Daher unterstützt Interface drei verschiedene Vergrößerungsfaktoren, die im Raster-Menü ausgewählt werden können. Groß bedeutet, daß für jeden Punkt 8 x 8 Pixel gezeichnet werden. Mittel steht für 6 x 6 Pixel und Klein für 4 x 4 Pixel pro Bildpunkt.

Wenn auch dies nicht ausreicht, können Sie den Preview oberhalb der Zeichenfläche ausblenden. Schalten Sie dazu die Option Iconvorschau im Bearbeiten-Menü des Editors aus.

Ausrichten eines Icons

Oberhalb der Palette mit den Zeichenfunktionen finden Sie fünf Icons mit Pfeilen, die in verschiedene Richtungen weisen. Durch Anklicken der ersten vier Icons (von links) verschieben Sie das angezeigte Bild in die Richtung, in die der Pfeil weist. Das »Pfeilkreuz« dient dem Zentrieren des Bildes innerhalb der Zeichenfläche.

Das Clipboard

Der Iconeditor unterstützt das GEM-Clipboard. Alle Icons und Images werden im XIMG- (Farbimage-Grafik) und im ICO- (IconEdi) Format gespeichert und geladen. Mit Bearbeiten: Kopieren oder Control C wird ein Icon auf das Clipboard kopiert. Über die Tastatur können Sie mit Shift Control C außerdem einen Ausschnitt des Editors auf das Clipboard kopieren. Nachdem Sie die Tastenkombination gedrückt haben, können Sie auf der Zeichenfläche einen Rahmen aufziehen. Sein Inhalt wird in das Clipboard kopiert. Diese Funktion können Sie mit einem Klick mit der rechten Maustaste abbrechen.

Bearbeiten: Ausschneiden bzw. Control X kopiert das Icon auf das Clipboard und löscht danach die Zeichenfläche.

Mit Bearbeiten: Einfügen oder Control V wird der Inhalt des Clipboards in den Editor übernommen. Wenn das einzufügende Bild kleiner als die Zeichenfläche des Iconeditors ist, kann man es vorher noch mit der Maus genau plazieren. In diesem Fall wird die neue Grafik transparent (!) über die alte Grafik gezeichnet, d.h. weiße Teile der eingefügten Grafik löschen keine Punkte der alten Grafik.

Wenn das einzufügende Bild größer oder genauso groß ist, wie das »alte« Bild, ersetzt es die alte Grafik komplett.

Tip: Wenn man im Iconeditor ein Windows- oder OS/2 Icon lädt, gibt es oft das Problem, daß das Icon »nur« in 16 oder 256 Farben vorliegt. Es gibt einen kleinen Trick, um zumindest die Umrisse ohne Aufwand in monochrom zu übernehmen:

Zuerst kopiert man das 16-farbige Icon mit Bearbeiten: Kopieren auf das Clipboard. Danach ruft man den Fileselektor (mit Control O) auf und sucht darin den ›CLIPBRD‹-Ordner (der sich auf Laufwerk A oder C befinden wird).

In dem Ordner sollten Sie jetzt zwei Dateien finden: ›SCRAP.ICO‹ und ›SCRAP.IMG‹. Wenn man ›SCRAP.ICO‹ mit dem Fileselektor löscht und in der monochromen Auflösungsstufe des Farbicons Bearbeiten: Einfügen wählt, wird die erste Plane der Image-Datei eingefügt. Die Umriße des Farbicons sollten jetzt sichtbar sein und dienen zur Vorlage der monochromen Auflösung.

Das ganze funktioniert, weil Interface ein Image nur dann in den Iconeditor einfügt, wenn keine ICO-Datei im Clipboard vorhanden ist. Eine ICO-Datei enthält nämlich auch die monochrome Auflösung des Icons, die aber beim Windows- oder OS/2-Icon noch leer ist. Dadurch, daß man die ICO-Datei löscht, zwingt man Interface dazu, die IMG-Datei einzufügen.

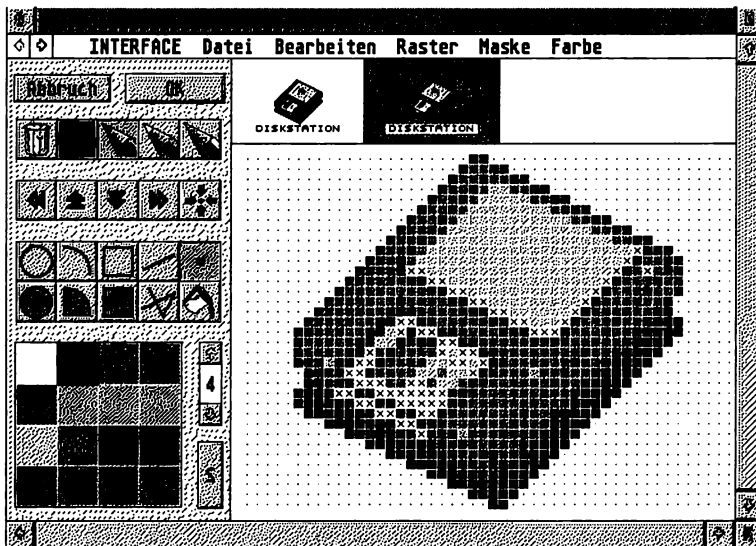
Farbicons und Animierte Icons

Auf die Besonderheiten von Farbicons sind wir bisher nicht eingegangen, weil es für die grundsätzliche Bedienung des Editors nicht wichtig war. Jetzt wird also das Versäumte nachgeholt.

Farbicons stehen im Desktop des Falcon und unter MultiTOS zur Verfügung. Mit den Farbicon-Routinen, die zum Lieferumfang von Interface gehören, sind Sie in der Lage, Farbicons in eigenen Programmen unter allen TOS-Versionen zu benutzen.

Ein Farbicon weist im allgemeinen einige Besonderheiten auf, die Sie berücksichtigen sollten. Erstens kann ein Farbicon aus bis zu vier verschiedenen Icons für unterschiedliche Auflösungen bestehen. Interface unterstützt Icons in 2, 4, 16 und 256 Farben. Damit Ihre Programme auch in monochrom Icons darstellen können, sollten Sie Farbicons also mit mindestens zwei Bildern (eines monochrom, das andere in Farbe)

versehen. Zweitens können Farbicons ein Selektionsbild darstellen, d.h., daß ein selektiertes Icon nicht die Maske anzeigt, sondern ein zweites Bild. ATARI bezeichnet solche Icons als »animiert«. Steht kein Selektionsbild zur Verfügung, werden selektierte Icons mit einer gepunkteten (schwarzen) Maske verknüpft gezeichnet.



Wenn Sie mit Bearbeiten: Objekte... ein Farbicon in ein Formular ziehen, so besteht dieses Icon bereits aus vier Bildern. Je ein Bild in monochrom und mit sechzehn Farben sowie den dazugehörigen Selektionsbildern.

Um zwischen den verschiedenen Bildern zu wechseln, haben Sie zwei Möglichkeiten: Im Farbe-Menü des Editors wechseln Sie mit Auflösung wählen zu der von Ihnen gewünschten Auflösung. Gibt es für diese Auflösung noch kein Bild, wird es angelegt. Mit Farbe: Selektiertes Icon wechseln Sie zwischen dem »normalen« Bild und dem Selektionsbild. Existiert noch kein Selektionsbild, wird es angelegt.

Praktisch die gleiche Funktion haben die Buttons neben der Farbauswahl am linken Rand des Editors. Der Button mit dem S schaltet zwischen normalem Icon und Selektionsbild um. Bei angezeigtem Selektionsbild ist auch der Button selektiert. Mit den Pfeilen über und unter der Zahl (in der Abbildung die »4«) wechseln Sie zwischen den verschiedenen Auflösungen. Die angezeigte Zahl gibt an, wieviele Planes das aktuell bearbeitete Bild hat.

Welche zusätzlichen Optionen Ihnen bei Farbicons zur Verfügung stehen, entnehmen Sie der folgenden Referenz.

Referenz

Das ›Datei‹-Menü

Neu

Mit dieser Funktion können Sie eine neue Resource-Datei anlegen. Die neue Datei erhält die Bezeichnung ›NAMENLOS‹, um anzuzeigen, daß Sie zunächst nur mit Datei: Speichern als... gespeichert werden werden kann. Auf dem Interface-Desktop wird ein Icon für die Datei angezeigt.

Öffnen

Wenn Icons oder Objekte selektiert sind, können sie mit dieser Funktion wie mit einem Doppelklick geöffnet werden. Andernfalls erscheint der Fileselektor, in dem eine Resource-Datei zum Laden ausgewählt werden kann. Wird der im Fileselektor angegebene Name nicht gefunden, wird eine neue Resource-Datei mit diesem Namen angelegt.

Schließen

Ist beim Aufruf dieser Option ein Fenster geöffnet, wird es geschlossen. Wenn außerdem das entsprechende Icon einer Resource-Datei selektiert ist, wird die Datei nach einer Sicherheitsabfrage aus dem Speicher entfernt.

Verwerfen

Die Resource-Datei, der das oberste Fenster gehört, wird aus dem Speicher entfernt. Wurde die Datei geändert, wird vorher noch gefragt, ob sie gespeichert werden soll.

Letzte Fassung

Diese Option verwirft die Resource, der das oberste Fenster gehört, und lädt ihren zuletzt gespeicherten Zustand wieder ein.

Sichern

Falls auf dem Desktop das Icon einer Resource-Datei selektiert ist, wird die entsprechende Datei gespeichert. Ansonsten wird die Datei, zu der das oberste Fenster gehört, gesichert. Headerdateien werden nur abgespeichert, wenn Labels verändert oder gelöscht wurden, die Objekthierarchie geändert wurde, oder wenn der Resourceinfo-Dialog mit OK verlassen wurde. Das dürfte alle Compilerbenutzer freuen, deren Compiler sonst das komplette Projekt neu compilieren würde.

Wenn die Resource-Datei neu angelegt wurde und dementsprechend noch die Bezeichnung »NAMENLOS« hat (s.o.), ist Sichern hell dargestellt und kann nicht ausgewählt werden. In diesem Fall muß die Datei mit Datei: Sichern als... gespeichert werden.

Sichern als...

Mit dieser Funktion kann eine Resource-Datei mit einem im Fileselektor einzugebenden neuen Namen abgespeichert werden. Wenn ein neuer Name eingegeben wurde, wird der bisherige Name im dazugehörigen Icon durch den neuen Namen ersetzt.

Info...

Ist auf dem Desktop ein Icon selektiert, erscheint ein Dialog mit Informationen über dieses Icon. Wenn kein Icon selektiert ist, dann erhält man Informationen über das oberste Fenster. Falls auch kein Fenster offen ist, wird die Copyrightbox von Interface angezeigt. In einigen Informations-Dialogboxen können auch Daten geändert werden:

Info einer ausgewählten Resource-Datei

Es erscheint ein Dialog mit zahlreichen Informationen über die Resource-Datei.

Geändert werden können

- Der Name der Datei
- Das Format der Definitionsdatei: DEF, DFN, RSD oder HRD (siehe Kapitel 2, Speichern der Resource-Formate). In der Definitionsdatei speichert Interface alle wichtigen Daten einer Resource (z.B. Labels und Voreinstellungen). Wenn möglich, sollten Sie das HRD-Format verwenden, da in diesem Format am meisten Informationen gespeichert werden können. Das HRD-Format ist auch das einzige Format, das Labels mit 16 Buchstaben erlaubt (sonst 8).
- Die Programmiersprache
- Der Prozessortyp (MSDOS GEM oder ATARI GEM)
- Die Schreibweise der Namen
- Die RSH-Sourcecode-Ausgabe

Wenn man den static-Button selektiert, wird jeder Variablen der Sourcecode-Ausgabe ein static vorangestellt. Dadurch werden alle Variablen modul-lokal definiert, und man kann mehrere Ressourcen im C-Source in einem Programm verwalten, ohne daß es Namenskonflikte gibt.

Aktiviert man den Button Langes Format, wird die Resource in einem speziellen Format ausgegeben, das (praktisch) beliebig lange Ressourcen erlaubt. Das Resourceformat bleibt trotzdem weitgehend kompatibel zum alten Format; Interface ändert nur alle Variablen des Resource-Headers von ›WORD‹ in ›LONG‹ um, damit mehr als 64 KB verwaltet werden können.

Auf den Interface-Disketten finden Sie Routinen in C und Basic, mit denen man diese Ressourcen laden kann.

Info eines Objektbaums

Geändert werden können

- Der Name des Baumes
- Die Baumart. Dabei dürfen Alerts in Free Strings, Free Strings in Alerts (wenn erlaubt), Menübäume in Formularbäume und Formularbäume (sofern möglich) in Menübäume gewandelt werden.

Letztere Option sollte **nicht** verwendet werden, da Interface den Aufbau eines Menübaums, wenn er zuvor in einen Formularbaum gewandelt wurde, nicht mehr überprüfen und korrigieren kann. Gebraucht wird diese Option höchstens, wenn man Icons u.a. Objekte in Menüzeilen unterbringen möchte. Mann sollte dies aber nach Möglichkeit vermeiden, da solche Objekte in Menüzeilen grundsätzlich nichts zu suchen haben.

Hilfe...

Hier erscheint normalerweise ein Dialog, der die Funktion der Tasten Control, Shift und Alternate erläutert. Wenn kein Icon einer Resource-Datei selektiert wurde und auch das oberste Fenster zu keiner Resource-Datei gehört, gibt es keine Hilfsinformation.

Abbrechen

Interface wird verlassen, ohne daß die Konfiguration gespeichert wird (auch wenn "Autosave" aktiviert ist).

Ende

Mit diesem Punkt kann Interface verlassen werden. Wenn vor dem Verlassen noch irgendwelche Änderungen an einer Resource vorgenommen wurden, wird vorher gefragt, ob die Datei gespeichert werden soll. Außerdem speichert Interface auf Wunsch (siehe Optionen: Einstellungen...) eine Konfigurations-Datei, so daß Sie beim nächsten Programmstart die gleiche Arbeitsumgebung vorfinden wie beim Beenden des Programms.

Das ›Bearbeiten‹-Menü

Objekte...

Ein Dialog erscheint, in dem die verschiedenen Objekttypen enthalten sind. In Kapitel 2 finden Sie eine Liste aller Objekttypen mit ihren Bedeutungen. Wenn Sie ein Objekt benötigen, dann ziehen Sie es aus diesem Dialog auf Ihr Arbeitsfenster. In Menüleisten und dazugehörigen Pulldown-Menüs ist nur das Objekt ›STRING‹ erlaubt. Alle anderen Objekte werden ignoriert. Objekte können nur auf Formularbaumfenster oder Menübaumfenster gezogen werden. Wenn die Objektkbox in einem Fenster liegt, kann man unter MultiTOS auch Objekte herausziehen, wenn die Box im Hintergrund liegt. Ohne MultiTOS erreichen Sie den gleichen Effekt, wenn Sie beim Anklicken eines Objektes gleichzeitig die linke und rechte Maustaste drücken.

Bäume...

In diesem Dialog sind alle Baumarten enthalten, als da sind:

- Menübaum: Er enthält Pulldown-Menüs, in denen der Benutzer eines Programms verschiedene Menü-Optionen anwählen kann. Menü-Optionen bestehen aus STRING-Objekten.
- Dialog: Ein Dialog wird benutzt, um Informationen anzuzeigen und Daten einzugeben. In einem Dialog können alle Arten von Objekten enthalten sein.
- Alert: Ein Alert ist eine besondere Art von Dialog. Sie wird benutzt, um wichtige Meldungen, Warnungen oder Fehler anzuzeigen. Ein Alert besteht, genau wie freie Strings, nur aus Text, aus dem das GEM dann in der ›form_alert()‹-Funktion den Alert erzeugt.
- Freie Strings/
Images: Hier können in einer Resource einzelne Texte oder Grafiken gespeichert werden, die nicht von GEM, sondern vom Programm selbst verwaltet werden sollen.

Bäume dürfen entweder auf ein Resource-Datei-Fenster oder auf das Icon einer Resource-Datei gezogen werden. Wenn die Baumbox in einem Fenster liegt, kann man unter MultiTOS auch Bäume herausziehen, wenn die Box im Hintergrund liegt. Ohne MultiTOS erreichen Sie den gleichen Effekt, wenn Sie beim Anklicken eines Baums gleichzeitig die linke und rechte Maustaste drücken.

Ausschneiden, Kopieren und Einfügen

Über diese Punkte kann man alle selektierten Objekte aus Formular- und Menübäumen auf das Interface-Clipboard kopieren, bzw. von dort aus an der Mausposition einfügen lassen.

Flags setzen...

Wenn in einem Formularfenster mehrere Objekte selektiert worden sind, kann man für diese Objekte die Attribute (SELECTABLE, EXIT, etc.) mit dieser Funktion gleichzeitig setzen, ohne jedes Objekt einzeln öffnen zu müssen.

Nach der Auswahl dieser Option erscheint ein Dialog, in der der momentane Status der Attribute fast wie in den normalen Objektdialogen angezeigt wird. Ein kleiner Unterschied liegt darin, daß Attribute bei mehreren Objekten auch unterschiedlich eingestellt sein können. Um das anzuzeigen, sind die Checkboxes solcher Attribute (Tri-State-Button) mit einem gepunkteten Muster gefüllt.

Wenn Sie nun in diesem Dialog einen Button selektieren oder deselektieren, dann wird dieses Attribut bei allen angewählten Objekten gesetzt bzw. gelöscht. Setzen Sie das Kästchen auf »gepunktet«, wird an diesem Attribut nichts geändert.

Suchen...

Hier können Sie in allen selektierten Resource-Dateien einen bestimmten Objektnamen (ein Label) oder Text suchen lassen. Falls keine Datei markiert ist, wird in der Datei gesucht, der das oberste Fenster gehört. Bei der Suche durchsucht Interface die komplette Resource. Die Unterscheidung von Groß- und Kleinschreibung ist einschaltbar.

Im Suchstring sind folgende Wildcards erlaubt:

- * Eine beliebige Anzahl beliebiger Zeichen
- ? Ein beliebiges Zeichen
- [a-z] Nur bestimmte Zeichen sind an dieser Stelle zugelassen. [c-e] würde bedeuten, daß an dieser Stelle nur die Zeichen "c", "d" und "e" zugelassen sind.

Einige Beispiele:

text sucht alle Textstellen, die die Zeichenfolge »text« enthalten.

text* sucht alle Textstellen, die mit »text« beginnen.

te?t sucht alle Textstellen, die mit »te« beginnen, ein beliebiges Zeichen an der dritten Stelle enthalten, und mit »t« enden.

te[w-y]t sucht alle Textstellen, die mit »te« beginnen, entweder »w«, »x« oder »y« an der dritten Stelle enthalten, und mit »t« enden.

Findet Interface den eingegebenen Namen, gibt es zwei Möglichkeiten: Ist der gesuchte Name ein Baumname, öffnet Interface das dazugehörige Baumfenster und selektiert den Baum. Handelt es sich bei dem gesuchten Objekt um ein Element eines Menü- oder Formularbaums, öffnet Interface den Baum, läßt das Objekt 3 mal blinken und selektiert es dann.

Objektbaum testen

Wenn das Icon eines Menü-, Formular- oder Alertbaums ausgewählt wurde oder ein entsprechendes Fenster oben liegt, kann der Baum mit dieser Option getestet werden. Sollte ein Formular- oder Alertbaum keinen Button besitzt, mit dem er verlassen werden kann, wird Interface Sie darauf hinweisen. Die Funktion kann dann nicht ausgeführt werden. Wenn der getestete Baum durch das Anklicken eines Buttons oder Menüeintrags verlassen wird, zeigt Interface die Objektnummer und, falls vorhanden, den Namen des angeklickten Objekts an.

Wenn Sie eigene Zeichenroutinen für selbstdefinierte Objekttypen benutzen (extended ob_types) und diese Routinen von Interface nachgeladen wurden (EXTOBFIX.PRG), dann werden diese Objekte in der Testfunktion korrekt gezeichnet und ausgeführt. Diese nachgeladenen Routinen können auch die komplette Testroutine der Formular- und Alertbäume ersetzen. Diese Routinen lassen sich mit Optionen: Einstellungen... bei Bedarf ausschalten. Näheres zu selbstdefinierten Objekttypen und dem EXTOBFIX.PRG erfahren Sie in Kapitel 5. Wenn ein Dialog getestet werden soll und dieser ein EDITABLE-Objekt enthält, daß nicht vom Typ FText oder FBoxText ist, wird eine Warnung ausgegeben, da das AES beim Bearbeiten einen solchen Objekts abstürzt.

Da es möglich ist, daß so ein Objekt von einer EXTOBFIX-Datei speziell behandelt wird, ist der Testvorgang trotzdem noch über die Alertbox aufrufbar. Also Vorsicht! Wenn ein Menübaum getestet werden soll und im Einstellungen...-Dialog des Optionen-Menüs Menü-Tastaturkürzel anordnen eingeschaltet ist, gibt Interface bei doppelt verwendeten Shortcuts eine Warnung aus.

Resourcevergleich...

Zwei Resourcedateien können miteinander verglichen werden. Der Vergleich erleichtert z.B. die Bearbeitung fremdsprachiger Ressourcen, die hierdurch leichter zu pflegen sind, und Fehler schneller aufgefunden werden. Um die Funktion auszulösen, muß man zwei Resourcedateisymbole selektieren und dann den Resourcevergleich aufrufen. Die Option ist auch anwählbar, wenn genau zwei Ressourcen geladen wurden oder die Baumfenster von genau zwei Ressourcen offen sind.

Wenn ein bestimmter Baum einer der Ressourcen selektiert oder geöffnet ist, kann der Vergleich wahlweise ab dem Anfang oder ab diesem Baum durchgeführt werden.

Das ›Fenster‹-Menü

Wechseln

Mit dieser Funktion kann das jeweils unterste Fenster nach oben gebracht werden. Durch mehrfache Anwendung können so alle offenen Fenster nacheinander in den Vordergrund geholt werden. Wenn man diese Option über Tastatur bedient (Control W), ist es möglich, ohne viele Mausaktionen schnell an jedes Fenster heranzukommen. Das Desktopfenster wird nicht mitge"ycled".

Datei n

In diesem Menü sind die ersten 10 geladenen Resourcedateien anwählbar. Sinn der Aktion ist, daß man auch mit ausgeschaltetem Desktop Fenster für Resourcedateien öffnen kann. Man kann es natürlich auch benutzen, um schnell das Fenster einer bestimmten Datei zu toppen (bzw. zu öffnen).

Die Resourcedateien 11 bis 15 sind weiterhin nur über den Interface- Desktop erreichbar.

Das ›Objekt‹-Menü

Alle Einträge dieses Menüs entsprechen dem Objekte-Popup (siehe Kapitel 2) und dienen dazu, diese Funktionen aufrufen zu können, wenn das Popup abgeschaltet ist (siehe Referenz, Optionen: Einstellungen...).

Editieren

Öffnet den Dialog mit den Einstellungen für das selektierte Objekt. Diesen Dialog können Sie auch mit einem Doppelklick auf ein Objekt aufrufen.

Sortieren

Öffnet den Dialog zum Sortieren der Objekte innerhalb eines Parents.

Zentrieren

Öffnet den Dialog zum Ausrichten eines Objektes relativ zu seinem Parent-oder Root-Objekt.

Einrasten

Richtet das ausgewählte Objekt im eingestellten Raster aus, verschiebt es also ggf. nach links und oben.

Verstecken

Setzt für das ausgewählte Objekt (und dessen Children) das HIDETREE-Flag, d.h., daß Objekt ist nicht mehr sichtbar und kann auch nicht ausgewählt werden.

Aufdecken

Macht ein verstecktes Objekt wieder sichtbar.

Plazieren

Öffnet einen Dialog in dem für das ausgewählte Objekt direkt Koordinaten und Größenangaben eingetragen werden können.

Entfernen

Entfernt im ausgewählten Objekt das Parent-Objekt, beläßt aber dessen Children im Formular.

Löschen

Das selektierte Objekt und alle seine children werden gelöscht. Das Löschen läßt sich durch die Undo-Taste wieder rückgängig machen.

Neuer Typ

Das Objekt kann in einen anderen Objekttyp umgewandelt werden. Es sind folgende Umwandlungen möglich:

STRING in TEXT

BUTTON in BOXTEXT

IMAGE in ICON und umgekehrt. Dabei können aber Daten verloren gehen, da die einzelnen Objektformate zum Teil stark voneinander abweichen. Bei der Umwandlung ICON zu IMAGE gehen z.B. der Buchstabe, der Icontext und die Maske des Icons verloren.

Das ›Optionen‹-Menü**Einstellungen...**

Im Einstellungen...-Dialog können eine ganze Reihe von Parametern verändert werden, die die Arbeit mit Interface beeinflussen. Es gibt folgende Einstellungen:

GEM Fileselektor

Interface verwendet einen eigenen Fileselektor (Dateiauswahl). Wenn er Ihnen nicht gefallen sollte oder Sie lieber einen eigenen benutzen wollen, können Sie ihn hiermit ausschalten und damit den originalen GEM-Fileselektor (oder einen anderen Fileselektor, der den von GEM ersetzt) wieder anschalten.

Lange Überschriften

Diese Option ist das Ergebnis einer Meinungsverschiedenheit. Früher (in der ersten Entwicklungsphase) verwendete Interface die FlyDials von Julian Reschke. In dieser Zeit hatte Interface unter jeder Boxüberschrift einen langen Unterstrich, der immer

fast bis an den rechten Rand eines Dialogs reichte. Leider fand das nicht die Zustimmung von Julian Reschke, der auf ein FlyDial-konformes Aussehen (mit kurzer Unterstreichung) bestand (und bestehen durfte). Schließlich wurde Interface mit den MyDials entwickelt und da uns aber die langen Unterstriche wesentlich besser gefallen, sind sie immer eingeschaltet. Alle, denen die FlyDial-Überschriften besser gefallen, können die langen Überschriften hier wieder ausschalten. Damit man die Änderung auch sehen kann, muß man nach dem Selektieren dieser Option zunächst die Parameter speichern und dann das Programm verlassen und wieder starten.

Grow/Shrinkboxen

Beim Öffnen und Schließen von Fenstern und Dialogen wird normalerweise immer ein sich vergrößerndes bzw. verkleinerndes Rechteck gezeichnet. Wer Zeit sparen will, kann das hier abschalten.

3D-Dialoge

Schaltet die Darstellung der 3D-Dialoge in Interface aus bzw. an.

Objektnamen mitkopieren

Wenn dieser Punkt selektiert ist und Sie Objekte oder ganze Bäume in eine andere Resource-Datei kopieren, werden die Objektnamen mitkopiert. Sollten bestimmte Namen in der neuen Datei schon vorhanden sein, weist Interface Sie darauf hin und gibt Ihnen Gelegenheit, einen anderen Namen einzugeben.

Externe Objektanzeige

Wenn eine Programmdatei mit dem Namen ›EXTOBFIX.PRG‹ im Verzeichnis von Interface liegt, wird sie bei Programmstart mitgeladen. Diese Datei kann Interface Zeichenroutinen für erweiterte Objekttypen (extended ob_types) zur Verfügung stellen. Außerdem kann sie die Testfunktionen für Formular- und Alertboxen ersetzen (siehe Kapitel 5).

Möchten Sie auf die Funktionen dieser Datei verzichten, können sie hier ausgeschaltet werden.

Menü-Tastaturkürzel anordnen

Wenn dieser Punkt selektiert ist, werden eventuell vorhandene Tastaturkürzel (z.B. ›^Q‹) in Menüeinträgen automatisch rechtsbündig angeordnet, so daß alle Kürzel untereinander stehen.

Außerdem überprüft Interface beim Aufruf der Testfunktion (F9), ob Tastaturkürzel doppelt verwendet wurden und gibt in dem Fall eine Warnung aus.

Pulldown Leerzeichen optimieren

Nach den GEM Konventionen sollten vor jedem Eintrag in einem Pulldown-Menü 2 Leerzeichen und dahinter 1 Leerzeichen stehen. Interface sorgt automatisch dafür, daß die richtige Anzahl von Leerzeichen eingefügt bzw. gelöscht wird. Wer selbst Kontrolle über die Leerzeichen vor und hinter dem Eintrag haben will, kann diese Optimierung hier ausschalten.

Objektdatenanzeige in Zeichen/Pixel

Ist dieser Punkt selektiert, wird in der Infozeile eines Formularfensters angezeigt, welches Objekt unter dem Mauspfel liegt (die Nummer), wie es heißt und wie breit und hoch es ist. Die Breite kann entweder in Buchstabenkoordinaten (Standardeinstellung) oder Pixelkoordinaten angezeigt werden. Bei der Angabe von Buchstabenkoordinaten wird dahinter noch eine Zahl angegeben, die die Pixelabweichung vom Buchstabenraster angibt. Wenn man die CTRL-Taste gedrückt hält, werden die Objektdaten des Parents des unter der Maus liegenden Objekts angezeigt.

Zum Beispiel bedeutet ›PLERDE : 2 w=34/00 h=05/09‹, daß sich unter der Maus das Objekt Nummer 2 mit dem Namen ›PLERDE‹ befindet und daß es eine Breite von 34 Buchstaben besitzt. Die Höhe beträgt 5 Buchstaben + 9 Pixel. Die Objektdatenanzeige wird auch in Menübäumen angezeigt. Außerdem werden alle 16 Zeichen des Objektnamens ausgegeben.

Wenn die Objektdatenanzeige oder die Sizebox eingeschaltet ist, gibt es in Interface merkwürdigerweise Probleme bei der Doppelklickerkennung, wenn zwei Events gleichzeitig eintreffen (MU_BUTTON und MU_M2). Das liegt an dem etwas "eigenwilligen" Verhalten von evnt_multi.

Wir haben nun versucht etwas dagegen zu unternehmen, indem wir den MU_M2-Event wieder durch einen MU_TIMER-Event ersetzt habe. Nach einem Einfachklick wartet Interface jetzt, ob direkt darauf noch ein weiterer Klick folgt. Wenn das der Fall ist, werden beide Klicks zu einem Doppelklick zusammengefaßt. Doppelklicks werden dadurch jetzt wesentlich besser erkannt. Als Nebeneffekt dauert es aber leider manchmal bis zu einer halben Sekunde, bis auf einen Einfachklick reagiert wird. Das passiert aber nur, wenn es ein »verdächtiger« Einfachklick ist (z.B. wenn zwei Events gleichzeitig gemeldet wurden).

Sizebox

Alle, die die Vergrößerungsecke eines Objektes nie finden konnten, können sie jetzt mit dieser Option sichtbar machen.

Objekt-Popup

Mit dieser Option läßt sich das Popup ausschalten, das ansonsten beim Selektieren eines Objektes erscheint. Alle darin enthaltenen Funktionen können deshalb auch über das Objekt-Menü erreicht werden. Dadurch ist das Selektionsverfahren von Interface »kompatibler« zu anderen Programmen (ein Klick auf ein Objekt selektiert es, anstatt das Popup erscheinen zu lassen).

Label-Prefix

Wenn dieser Punkt selektiert ist, stellt Interface jedem neu eingegebenen Label eine einstellbare Anzahl von Zeichen des Baumnamens voraus. Ein Trennzeichen kann auch definiert werden. Die Buchstaben werden nur dann vor das Label gehängt, wenn das Objekt bisher noch keinen Namen besessen hat.

Ein Beispiel: Ein neues Objekt im Baum ›SETTINGS‹ soll den Namen ›OK‹ bekommen. Label-Prefix ist auf 3 Zeichen gesetzt, das Trennzeichen ist ›_‹. Dadurch bekommt das Objekt den Namen ›SET_OK‹ zugewiesen.

Parameter automatisch speichern

Alle Einstellungen von Interface sowie die meisten Daten der aktuellen Desktop-Konfiguration werden beim Beenden von Interface mit Datei: Ende automatisch gespeichert, wenn diese Option eingeschaltet ist. Dadurch findet man Interface nach einem späteren Start wieder im gleichen Zustand vor.

Gespeichert wird:

- Die Position aller Icons
- Die Größe, Position und Sliderposition aller Resourcedatei-Fenster (dazu zählt das Baumfenster, Formularfenster und Menüfenster)
- Die Position der Baum- und Objektkontainer
- Die Größe, Position und Sliderposition des Desktopfensters
- Die Pfade aller geladenen Resourcedateien
- Die Position jedes geöffneten Baumfensters. Wenn man eine Resource lädt, wird das Fenster an der alten Position geöffnet. Interface verhält sich damit ähnlich wie der ATARI-Desktop.

Alle Koordinaten werden bei einem Auflösungswechsel nötigenfalls angepaßt, damit immer noch alle Objekte erreicht werden können.

Im Fenster:**Desktop**

Unter MultiTOS könnte es stören, daß Interface einen eigenen Desktop anlegt. Damit man dort nicht immer zwischen mehreren Desktops unterschiedlicher Programme umschalten muß, kann man Interface mit diesem Schalter dazu zwingen, den eigenen Desktop in ein Fenster zu legen. Die Änderung wird nicht sofort aktiv. Man muß erst die Parameter speichern, Interface verlassen, und es erneut laden.

Menüzeile

Der praktische Nutzen ist vielleicht fraglich. Aber da Interface über diese Möglichkeit verfügt (für die kleinen Farbaufösungen), wurde es jetzt einschaltbar gemacht.

Dialoge

Damit andere Prozesse unter Mutitasking-Betriebssystemen wie Mag!X, MultiGEM oder MultiTOS nicht in ihren Bildschirmausgaben angehalten werden, wenn Interface eine Dialogbox anzeigt, werden mit dieser Option alle Dialoge in Fenster gelegt. Falls kein Fenster mehr zur Verfügung steht, werden Dialoge als normale Dialogboxen angezeigt.

Objektboxen

Damit man die Objektboxen nicht immer erneut über ›O‹ bzw. ›B‹ aufrufen muß, kann man sie in ein Fenster legen und dann auf dem Desktop liegen lassen. Zusammen mit der rechten Maustaste kann man dann Objekte auch aus den im Hintergrund liegenden Objektboxen herausziehen. Unter MultiTOS ist das Drücken der rechten Maustaste nicht nötig.

Raster...

Beim Verschieben von Objekten oder einer Änderung der Objektgröße benutzt Interface ein Raster, in dessen Schrittweite die Koordinaten bestimmt werden können. Die Größe dieses Raster läßt sich getrennt für die vertikale und die horizontale Richtung einstellen. Es stehen die Einstellungen 1 Zeichen, 1/2 Zeichen und 1 Pixel zur Verfügung. Damit GEM unabhängig von der gewählten Bildschirmauflösung Ressourcen immer richtig darstellen kann, sollte die Rastergröße für beide Richtungen immer 1 Zeichen betragen. Verwenden Sie diese Einstellung nicht, besteht die Gefahr, daß Ihr Programm in Auflösungen mit einem anderen Systemfont (z.B. kleiner als 640 x 400) die Dialoge korrekt darstellt. Nur, wenn Sie bewußt eine pixelgenaue Positionierung einsetzen wollen, sollten Sie diese einschalten. Das XY-Raster für Objektpositionen wird für jede Resource getrennt verwaltet. Wenn man das HRD-Format benutzt, wird das aktuelle Raster mit der Resource abgespeichert.

Raster benutzen

Mit diesem Punkt kann man das eingestellte Raster aller Ressourcen auf Pixelraster (wirkt wie ein ausgeschaltetes Raster) umschalten. Bei einer erneuten Anwahl werden die alten Rastereinstellungen restauriert. Der Sinn ist eine schnelle Umschaltung zwischen Buchstaben- und Pixelraster.

Größenanpassung

Wenn Sie den Text eines STRING-Objektes verändern und der neue Text kürzer oder länger als der alte ist, paßt Interface normalerweise die Objektgröße der neuen Textlänge an. Beim Eingeben von kürzeren Texten kann es jedoch teilweise wünschenswert sein, daß die bisherige Länge beibehalten wird. Daher kann mit der Option Größenanpassung die Anpassung für den Fall, daß die neue Textlänge kürzer ist, ausgeschaltet werden.

Objektnummern fest

Durch diese Option kann man verhindern, daß sich Objektnummern bei Aktionen verändern. Statt dessen ändert Interface nur die Verkettung der Objekte, sowohl in Dialogen als auch in Menübäumen. Wenn diese Option selektiert ist, ändert Interface auch nicht mehr die Position unsortierter Menübaumeinträge.

Wenn ein neues Objekt in einen Baum eingefügt wird, wird es an das Baumende angehängt und verändert daher nicht die Objektnummern anderer Objekte in diesem Baum. Das Sortieren in Bäumen ist auch möglich. Dabei wird nur die Objektverkettung »umgehängt«, alle Objekte bleiben an ihrem Platz.

Wenn dieser Schalter gesetzt ist, sollte es leicht möglich sein, Ressourcen fremder Programme zu ändern, ohne daß das Programm dadurch Schaden nimmt. Es kann nur zu Fehlern kommen, wenn ein Programm der Objektverkettung der eigenen Resource folgt. Aber das dürfte fast nie vorkommen.

Parameter sichern

Mit dieser Option werden alle unter Optionen: Einstellungen... beschriebenen Parameter gesichert. Parameter sichern sollten Sie verwenden, wenn Sie das automatische Speichern der Parameter ausgeschaltet haben.

Referenz (Iconeditor)

Das ›Datei‹-Menü

Grafik laden...

Der Iconeditor kann Grafiken im Image-Format (Monochrom und in Farbe) laden. Aus diesen Grafiken können Sie anschließend mit Datei: Grafik ausschneiden Teile der Grafik in den Editor übernehmen.

Interface öffnet nach dem Laden ein Fenster mit der Grafik und wählt automatisch die Option Grafik ausschneiden.

Farbbilder werden in der aktuellen Palette bzw. unter TrueColor in der Palette der Resource angezeigt, damit man sieht, mit welchen Farben der ausgeschnittene Block vom Iconeditor übernommen wird. Interface führt keine Farbumwandlung der XIMG-Grafik an die Palette des Iconeditors durch. Man muß also u.U. solche Grafiken vorher mit einem Zeichenprogramm auf die Palette der Resource umrechnen lassen.

Wenn man einen Block aus einer 256-farbigen Grafik in ein Icon mit 16 Farben einfügen will, werden nur die ersten 4 Planes (16 Farben) ausgeschnitten und in den Iconeditor übernommen (logisch). Dadurch werden alle Farben über der Farbe 15 falsch übernommen. Man hat immerhin den Vorteil, daß alle Umrisse der gewünschten Grafik bereits erkennbar sind, und man sie nur etwas nachbearbeiten muß.

Grafik schließen

Ein Image wird so lange im Speicher gehalten, bis es mit Grafik schließen aus dem Speicher entfernt wird. Danach kann mit Datei: Grafik laden eine andere Grafik geladen werden.

Grafik ausschneiden...

Mit der Option Datei: Grafik ausschneiden können Sie aus einem vorher geladenen Image einen beliebig großen Bereich (allerdings maximal so groß wie das Icon) markieren und in den Iconeditor übernehmen.

Wenn Sie diese Option wählen und noch kein Bild geladen wurde, erscheint zunächst eine Dateiauswahl, mit der Sie die gewünschte Bilddatei auswählen können. Wenn Sie diese Option wählen obwohl das Fenster mit dem Image vorher geschlossen wurde, wird es vor dem Ausschneiden wieder geöffnet. Zur Erinnerung: Das Schließen des Fensters entfernt die Grafik nicht aus dem Speicher.

Als Image sichern...

Mit dieser Option können Sie Icons als Image sichern und so z.B. in einem Grafikprogramm bearbeiten.

Der Iconeditor verwendet dazu das XIMG-Format, das kompatibel zum monochromen Image-Format ist, aber zusätzlich noch eine RGB-Tabelle enthält. Eine Beschreibung des XIMG-Formats steht z.B. im Buch ›Vom Anfänger zum GEM-Profi‹ von Dieter und Jürgen Geiß.

In der XIMG-Datei wird auch die RGB-Palette der Resource gespeichert.

Quellcode laden...

Die Option Quellcode laden... ermöglicht das Laden von Icon-Dateien im ICN-Format, wie es z.B. vom RCS von ATARI gespeichert wird (siehe auch Datei: Quellcode sichern).

Quellcode sichern...

Wenn Sie die Daten eines Icons direkt in ein C-Programm übernehmen wollen (z.B. für eine neue Mausform), können Sie die Grafik als C-Quellcode im ICN-Format sichern. Wenn es sich bei der Grafik um ein Icon handelt, werden Sie vorher gefragt, ob Sie die Grafik- oder Maskendaten sichern wollen.

Auf diese Weise können auch Farbicons im Quellcode gespeichert werden. Dabei bleibt das ICN-Format aus Kompatibilitätsgründen unverändert, es werden nur mehr Grafikdaten gespeichert.

An die Anzahl der verwendeten Planes gelangt man durch:

```
planes = DATASIZE / ((ICON_W / 16) * ICON_H);
```

Die Grafikdaten werden im geräteunabhängigen Format gespeichert. Bevor man das Icon anzeigen kann, muß man es mit "vr_trnfm" in das geräteabhängige Format wandeln.

Die Maske enthält natürlich weiterhin nur 1 Plane.

Windows-Icon laden...

Bei der Bearbeitung von Farbicons, kann man mit dieser Option ein Windows- oder OS/2-Icon laden. Interface unterstützt Windows-Icons mit 2, 16 und 256 Farben. Windows-Icons mit 256 Farben gibt es erst ab Windows 3.1. OS/2 2.0 Icons können noch nicht geladen werden, da das Format scheinbar nicht kompatibel zu älteren OS/2 Versionen ist.

Windows- und OS/2-Icons liegen in vielen Mailboxen und können sicherlich auch von Public Domain Versendern bezogen werden. Dadurch kann man bereits eine Vielzahl von Farbicons auf dem Atari nutzen.

Das ›Bearbeiten‹-Menü

Farbdurchlauf

Wenn die Option Farbdurchlauf nicht gewählt ist und Sie im Editor einen schwarzen Punkt anklicken, wird er weiß und beim nächsten Anklicken wieder schwarz. Bei ausgewähltem Farbdurchlauf wird ein schwarzer Punkt im Editor zunächst grau, d.h. er wird der Maske zugeordnet. Mit dem nächsten Klick wird er weiß, dann schwarz, wieder grau ...

Iconvorschau (Preview)

Wenn Ihnen die Zeichenfläche des Editors zu klein ist, können Sie die Anzeige des Previews mit Bearbeiten: Iconvorschau ausschalten. Die vom Preview beanspruchte Fläche wird dann zusätzlich vom Editor genutzt.

Die Einstellung dieser Option wird mit Optionen: Parameter sichern... gesichert.

Horizontal spiegeln

Spiegelt die Grafik im Editor an der vertikalen Achse, d.h. Pixel die vorher oben links sichtbar waren sind anschließend oben rechts sichtbar.

Vertikal spiegeln

Spiegelt die Grafik im Editor an der horizontalen Achse, d.h. Pixel die vorher oben links sichtbar waren sind anschließend unten links sichtbar.

Ausschneiden

Das Icon wird als (Farb-)Image und im ICO-Format im GEM-Clipboard gespeichert und anschließend im Editor gelöscht.

Kopieren

Das Icon wird als (Farb-)Image und im ICO-Format im GEM-Clipboard gespeichert.

Block kopieren

Ein Teil des Icons wird als (Farb-)Image und im ICO-Format im GEM-Clipboard gespeichert. Nach Auswahl der Option müssen Sie im Editor den Bereich markieren, der gespeichert werden soll. Die Aktion läßt sich mit der rechten Maustaste abbrechen.

Einfügen

Das Image aus dem Clipboard wird in den Editor übernommen. Wenn das einzufügende Bild kleiner ist als das aktuell bearbeitete, können Sie es vorher noch mit der Maus plazieren. In diesem Fall wird die eingefügte Grafik transparent (!) über die

»alte« Grafik gezeichnet, d.h. weiße Teile der eingefügten Grafik löschen keine Punkte der ursprünglichen Grafik. Wenn das einzufügende Bild größer oder genauso groß ist, wie das alte Bild, ersetzt es die alte Grafik komplett.

Löschen

Die Zeichenfläche wird ebenso wie eine eventuell vorhandene Maske vollständig gelöscht.

Das »Raster-Menü

Mit den Optionen des Raster-Menüs verändern Sie die Darstellung des Icons in der Zeichenfläche. Groß bedeutet, daß für jeden Punkt 8 x 8 Pixel gezeichnet werden. Mittel steht für 6 x 6 Pixel und Klein für 4 x 4 Pixel pro Bildpunkt.

Das »Maske-Menü

Die Optionen dieses Menüs können Sie nur auswählen, wenn Sie ein Icon editieren, da Images keine Maske besitzen. Die Maske wird durch kleine Kreuze im Editor angezeigt.

Anzeigen

Die Maske wird nur dann angezeigt, wenn Maske: Anzeigen ausgewählt ist. Nur in diesem Fall kann sie auch mit den Zeichenfunktionen bearbeitet werden.

Ganze Fläche

Die Maske füllt die gesamte Fläche des Icons aus, d.h. alle Punkte der Maske werden gesetzt.

Innen

Das Innere eines Icons, d.h. alle von farbigen Flächen vollständig umschlossenen weißen Flächen werden mit der Maske gefüllt.

Tip: Bei Icons ist die »Farbe« weiß gleichbedeutend mit farblos, so daß der Hintergrund sichtbar ist. Wenn Sie weiß als Farbe einsetzen wollen, benutzen Sie hierzu die Maske.

Umriß

Jeder Punkt der Grafik wird von Punkten der Maske umschlossen.

Löschen

Die Maske wird gelöscht. Nur unter schwarzen Punkten bleibt die Maske erhalten.

Das »Farbe-Menü

Die Optionen des Farbe-Menüs stehen naturgemäß nur zur Auswahl, wenn Sie ein Farbicon bearbeiten.

Ein Farbicon ist in Wirklichkeit kein einzelnes Icon, sondern besteht aus mehreren Icons für die unterschiedlichsten Farbanzahlen. So können Sie z.B. ein »Farbicon« definieren, das nur eine Bitplane hat, also monochrom ist. Wenn man eine Farbicon-Resource lädt, sucht sich das Betriebssystem bei jedem Farbicon das Icon aus, das gerade am besten zur momentan aktiven Auflösung paßt.

Auflösung wählen...

Öffnet einen Dialog, in dem Sie die Farbebene (»Plane«) eines Farbicons wählen können, die Sie bearbeiten wollen. Falls die entsprechende Farbebene noch nicht existiert, wird Sie angelegt.

Auflösung entfernen

Löscht die im Editor angezeigte Farbebene. Wenn für diese Plane auch ein Selektionsbild existiert, wird es ebenfalls gelöscht.

Selektiertes Icon

Bei Icons mit Selektionsbild kann mit Selektiertes Icon zwischen dem Icon und dem Selektionsbild gewechselt werden. Existiert noch kein Selektionsbild in der gewählten Auflösung, wird es angelegt.

Selektion entfernen

Das Selektionsbild der aktuellen Auflösung wird gelöscht.

RSC-Palette anzeigen

Die aktuelle Farbpalette des Systems wird durch die Palette der Resource ersetzt (die mit der Resource gespeichert wird). Dadurch kann man sich ansehen, welche Palette in einer Resource verwendet wurde. Unter TrueColor-Auflösungen ist dieser Punkt nicht anwählbar, da Interface dort immer die Palette der Resource anzeigen kann.

XIMG-Palette anzeigen

Die aktuelle Farbpalette des Systems wird durch die Palette des gerade geladenen Bildes (XIMG) ersetzt. Dadurch kann man sich ansehen, welche Palette in einem Farbbild verwendet wurde.

Standard-Palette

Mit dieser Option können Sie die Palette wieder einschalten, die Interface beim Programmstart vorgefunden hat.

Palette übernehmen

Die aktuelle RGB-Palette des Systems wird in die Resource übernommen (und mit ihr gespeichert).

Bei Farbicons gibt es das Problem, daß man nicht garantieren kann, daß jedermann die gleiche Farbpalette auf seinem Rechner verwendet. Die Farbwerte für die ersten 16 Farben sind zwar mehr oder weniger genau definiert, aber alle darüberliegenden Farben können sich von Rechner zu Rechner (Anwender zu Anwender) unterscheiden. Man kann natürlich auch niemanden daran hindern, die ersten 16 Farben umzudefinieren.

Damit ein Programm die Farbicons in den korrekten Farben zeichnen kann, muß es natürlich wissen, mit welcher Farbpalette diese Icons gezeichnet wurden.

Die mitgelieferten Routinen (>XRSRC<) sorgen dafür, daß die gezeichneten Icons in allen Auflösungen korrekt gezeichnet werden, in denen im Pixel direkt der RGB-Wert enthalten ist. Das sind alle Auflösungen mit 15 und mehr Planes (>= 32768 Farben). Das kann aber natürlich nur funktionieren, wenn Sie eine RGB-Palette in die Resource übernommen haben.

Die RGB-Palette ist eine Erweiterung des Resource-Formats von Interface. ATARI hat selber keine RGB-Palette vorgesehen. Das Resource-Format bleibt trotzdem kompatibel zum ATARI-Format und kann weiterhin problemlos vom Farbicon-AES (Falcon und MultiTOS) eingelesen werden. Die RGB-Tabelle wird dabei einfach ignoriert.

Rein theoretisch könnten die mitgelieferten Farbicon-Routinen natürlich in allen Auflösungen die RGB-Palette für die Farbicons einstellen, aber wir haben darauf verzichtet, weil wir vermuten, daß sich der Benutzer etwas dabei gedacht hat, seine Palette zu verändern. Außerdem wäre es unter Multitasking-Betriebssystemen grau-enhhaft, wenn jedes Programm dauernd die eingestellte Farbpalette umschalten würde.

Die ersten 16 Farben sollten wie folgt definiert sein (siehe dazu auch das ›ATARI Profibuch‹, 10. Auflage auf Seite 284 oder ›Vom Anfänger zum GEM-Profi‹, 1. Auflage auf Seite 76-77):

Nr	Farbe	Rot	Grün	Blau	(Wertebereich von 0-255)
0	weiß		255	255	255
1	schwarz		0	0	0
2	rot		255	0	0
3	grün		0	255	0
4	blau		0	0	255
5	cyan		0	255	255
6	gelb		255	255	0
7	magenta		255	0	255
8	hellgrau		192	192	192
9	dunkelgrau		128	128	128
10	dunkelrot		182	0	0
11	dunkelgrün		0	182	0
12	dunkelblau		0	0	182
13	dunkelcyan		0	182	182
14	dunkelgelb		182	182	0
15	dunkelmagenta		182	0	182

Diese Farben entsprechen auch weitestgehend den verwendeten Windows und OS/2 Iconfarben. Nur wenn diese Farben korrekt eingestellt sind, kann Interface diese Icons in den richtigen Farben darstellen.

Sie können diese Farben im Kontrollfeld einstellen. Auf der Interface-Diskette befindet sich auch ein neues CPX-Modul für XControl, das auf jedem TOS und auf jeder Grafikkarte funktioniert, und mit dem die Farben auch korrekt eingestellt werden können.

Speichern der Resource-Datei

Damit Sie ihre alten Resource-Dateien auch unter Interface verwenden können, unterstützt Interface alle bekannten Parameterdatei-Formate anderer Resource Construction Kits. Die Formate können sowohl gelesen als auch gespeichert werden.

Um Interface anzuweisen, beim Speichern ein bestimmtes Format zu wählen, selektieren Sie das Icon der Resource-Datei auf dem Desktop und wählen die Option Datei: Info... In dem darauf angezeigten Dialog können Sie das Format auswählen. Interface erkennt beim nächsten Laden das Format automatisch und speichert die Resource-Datei wieder in dem gewählten Format.

Unterschiedliche Resource-Formate

Nachfolgend finden Sie eine Auflistung der verschiedenen Parameterdatei-Typen und der Programme, die diese Dateien schreiben.

- DEF: dieses Dateiformat wird vom ersten Digital Research RCS verwendet.
- RSD: entspricht dem DEF-Format. Es wurde vom Kuma RCS eingeführt. Da in Modula Dateien mit der Endung DEF verwendet wurden, benannte Kuma das DEF-Format kurzerhand in RSD um.
- DFN: dieses Format wird von den neueren Digital Research RCS ab Version 2.0 verwendet. Leider wird in diesem Format nicht mehr gespeichert, für welche Sprachen Headerdateien ausgegeben werden sollen. Dieses Format ist nur wichtig, wenn mit Interface Resource-Dateien für MSDOS-Programme entwickelt werden sollen, da das einzige MSDOS-RCS nur DFN-Dateien lesen kann.
- HRD: dieses ist das neueste Format, es wurde vom WERCS RCS eingeführt. Es ist am platzsparendsten und speichert die meisten Informationen (z.B. Raster, Namensschreibweise u.a.). Es sollte den anderen Formaten wenn möglich immer vorgezogen werden, da auch nur in diesem Format 16 Zeichen lange Objektnamen abgespeichert werden können. Alle anderen Formate unterstützen nur 8 Zeichen für die Objektnamen.

DEF und RSD Format

Das DEF-Format ist das ursprünglich von Digital-Research eingeführte Resource-Format.

Offset	Länge	Beschreibung
--------	-------	--------------

0	2	Anzahl der Datensätze in der Datei (nur im ersten Datensatz ungleich 0)
2	2	Header-Ausgabe: 1 = C, 2 = Pascal, 4 = Modula, 8 = Basic, 16 = GFA 4 1 Bei Objekten: Index des zugehörigen Baums
5	1	Index des Baumes bzw. Objektes
6	1	1 wenn Objekt, 0 wenn Baum
7	1	Typ-Code (siehe unten)
8	8	Name des Objektes/Baums mit Null aufgefüllt

Typ	Codes
0	Objekt im Baum oder Menü
1	PANEL (wird von Interface durch Typ 3 ersetzt)
2	Menübaum
3	Formularbaum
4	Alertbox
5	Free-String
6	Free-Image

DFN Format

Das DFN-Format entspricht im wesentlichen dem DEF-Format. Der Unterschied liegt darin, daß die High- und Low-Bytes der Einträge vertauscht sind und die Bytes 3 und 4 (Header-Ausgabe) entfallen. Dieses Dateiformat ist kompatibel zum Digital Research RCS auf MSDOS-Rechnern.

HRD Format

Das Wercs-Format ist in einen Anfangsrecord und sich in der Länge ändernde Folge-records aufgeteilt. Beachten Sie, daß die Typ-Definition anders als bei den anderen Definitionsdateien ist.

Der Kopf der HRD-Datei

Offset	Länge	Beschreibung
0	2	Version: Momentan 1
2	1	Auto Namen: 1 wenn Wercs automatisch Namen vergeben soll (von Interface nicht verwendet, dort immer 0).
3	1	Sprach-Flag: 1 = C, 2 = PASCAL, 4 = MODULA, 8 = BASIC, 16 = GFA, 128 = RSH-Ausgabe (nur in Interface)
4	1	Auto-Pos: 0 Auto-Pos ausgeschaltet, 1 = Halb-Pos eingeschaltet, 2 = Auto-Pos eingeschaltet
5	1	Label Schreibweise: 0 = Groß/Klein, 1 = Groß, 2 = Klein
6	1	Auto-Größe: 0 = Auto-Größe aus, 1 = Auto-Größe ein
7	1	reserviert

Der Folge-Record

Offset	Länge	Beschreibung
0	1	Typ: siehe unten
1	1	reserviert
2	2	Baum-Index, d.h. Nummer des Baums
4	2	Objekt-Index
6	?	Name des Objektes mit 0 abgeschlossen

Typ	Codes
0	Dialog
1	Menü
2	Alertbox
3	Freier Text - Free String
4	freier Bitblock - Free Image
5	Objekt
6	Dateiende
7	Wenn Record für Präfix steht (???)

Das Icon-Format ICO

Interface verwendet das ICO-Format, wenn man im Iconeditor ein Icon auf das GEM-Clipboard kopiert. Das ICO-Format wurde von Stefan Münch in seinem Programm ›IconEdi‹ eingeführt und hat nicht das geringste mit dem Windows oder OS/2 ICO-Format gemein. Wir zitieren jetzt die Erläuterung des Formats aus der IconEdi-Anleitung:

IconEdi speichert Icons und Images in einem Format, das auf üblichen GEM-Strukturen beruht, trotzdem aber sehr leicht zu handhaben und zu implementieren ist.

Dateien dieses Formats haben einen Kopf, der mit einer Kennung beginnt (einer »magischen Zahl«), über Version und Kopflänge Auskunft gibt und auf einen Objektblock zeigt. Ab dort ist dann weiterverzeigert. Die Verzeigerung bezieht sich immer auf den Dateianfang, und die Adressen liegen im Motorola-Format vor. Als Dateiextension sollte ›ICO‹ gewählt werden.

Der Kopf sieht so aus (die Notation ist an die im Profibuch verwendete angelehnt):

Offset	Struktur
--------	----------

```

typedef struct
{
0      char icon_magic[4]; /* enthält "ICON"; $49434F4E */
4      int icon_version;   /* Versionsnummer, wie beim TOS */
      /* Start bei $0000 (so wie hier */
      /* vorgestellt */
6      int icon_headerlength; /* Länge des Headers in Words, */
      /* Standard: 5 */
8      int *objektblock     /* Zeiger auf den Objektblock */
}
ICONHEADER;
```

Die Entscheidung, was für ein Objekt die Datei enthält, wird anhand ob_type getroffen und ob_spec zeigt auf eine entsprechende Struktur. IconEdi unterstützt nur Icons und Images; prinzipiell kann in diesem Format aber jedes Objekt gespeichert werden.

Interface anpassen

Interface wurde so programmiert, daß Sie es leicht an Ihre eigenen Bedürfnisse anpassen können, indem Sie die Resourcedatei von Interface ändern. Machen Sie aber unbedingt vor solch einer Änderung eine Sicherheitskopie der Resource-Datei! Wenn Sie die Resource-Datei von Interface ändern, dürfen Sie nur den Text von Objekten ändern. Verschieben Sie niemals einzelne Objekte und ändern Sie nie die Größe der Objekte mit der Maus, da sich dadurch die Reihenfolge der Objekte ändern und Interface nicht mehr korrekt auf seine Resource zugreifen kann.

Interface bietet auch für solche Änderungen eine zuverlässige Hilfe: die Option Optionen: Objektnummern fest. Durch diesen Menüpunkt wird verhindert, daß sich Objektnummern durch Ihre Aktionen verändern. Statt dessen ändert Interface nur die Verkettung der Objekte, sowohl in Dialogen als auch in Menübäumen.

Tastaturanpassung

Alle Tastaturaufufe der Interface-Menüs und die Tastaturbedienung von Dialogen können nachträglich geändert werden. Laden Sie dazu die Resource-Datei von Interface und suchen Sie den Baum, in dem Sie die Tastaturanpassung ändern wollen. In einem tastaturbedienbaren Dialog-Objekt steht vor dem entsprechenden Buchstaben eine nach rechts offene eckige Klammer. Um diesem Objekt eine neue Taste zuzuordnen, müssen Sie diese Klammer vor einen anderen Buchstaben stellen. Der Buchstabe wird dann später unterstrichen dargestellt. Wie sie sehen, kann immer nur eine Tastenkombination verwendet werden, die einem Buchstaben des Objekttextes entspricht. Außerdem sollten Sie darauf achten, einen Buchstaben nicht in zwei verschieden Objekten zu markieren, da sonst nur eins (das erste) über die Tastatur angewählt werden kann. Wenn Sie ein Tastaturkürzel in den Menüs ändern wollen, müssen Sie einfach nur hinter den entsprechenden Menüeintrag das neue Kürzel schreiben. Auch hier darf ein Eintrag natürlich nur einmal vorkommen. Mehrfaches Vorkommen können Sie durch den Test des Menüs (Bearbeiten: Objektbaum testen) feststellen, wenn Sie vorher im Einstellungen....-Dialog des Optionen-Menüs Menü-Tastaturkürzel anordnen auswählen.

Anpassung von Dateiextensions

Änderung der Resourceformat-Extensions

Öffnen Sie in der Interface-Resource den Baum "RSCINFO". Dort sehen Sie die Buttons RSD, DEF, DFN und HRD. Wenn Sie z.B. die RSD-Datei lieber mit der Endung XYZ abspeichern wollen, dann müssen Sie einfach nur den Text des Buttons RSD in XYZ umändern.

Änderung der Headerdatei-Extensions

Alle Extensions der Headerdateien befinden sich in der Interface-Resource als Free Strings mit den Namen BASEXT, MODEXT1, MODEXT2, GFAEXT, PASEXT und CEXT (für Basic, Modula usw.). Ändern Sie einfach den Text des entsprechenden Free Strings in die neue Extension.

Änderungen für andere Sprachen

Änderungen für Basic

Für die Basic-Headerdatei werden in der Interface-Resource die Free Strings VORTEXT, TXTZEILE und NACHTEXT verwendet. Überall, wo sich in diesen Free Strings der Platzhalter "%s" befindet, wird später der Objektname oder der Baumname eingesetzt. Die wichtigsten Free Strings und ihre Bedeutung:

VORTEXT : Die erste Zeile der Headerdatei. An der Stelle des Platzhalters wird später der Free String FRRSHEAD eingesetzt. Im Platzhalter von FRRSHEAD steht dann der Name der Resource-Datei. TXTZEILE: Zuweisung der einzelnen Labels. 1. Platzhalter: Name des Labels, 2. Platzhalter: Nummer des Labels, 3. Platzhalter: Free String FRRSOBJ mit dem Objekttyp und dem Namen des Baumes.

NACHTEXT: Die letzte Zeile der Headerdatei Die Platzhalter entsprechen den Konventionen der Sprache C. Wenn Sie mehrere Zeilen z.B. in VORTEXT unterbringen wollen, dann müssen Sie die einzelnen Zeilen hintereinander schreiben, und sie durch die beiden Zeichen ›\n‹ (Zeilenvorschub in C) voneinander trennen.

Änderungen für Lattice C

Für alle Lattice C Fans befindet sich in der Interface-Resource der Free String OBSPEC_CAST mit dem Inhalt ›(LONG)‹ für die Sourcecodeausgabe. Da Lattice C mit (LONG) nicht zurechtkommt, sondern vor ob_spec zwingend auf (VOID *) besteht, kann man es durch Änderung der Interface-Resource umdefinieren.

Änderungen für Modula-2

Fast alle Strings der Modula-Headerausgabe liegen jetzt in der Interface-Resource und sind dadurch frei definierbar.

Ändern der Alerts

Die Default-Buttons und Abbruch-Buttons der Alertboxen sind konfigurierbar. Dazu ein Beispiel:

Beim Verlassen des Programms fragt Interface, ob die geänderte Resource-Datei gespeichert werden soll. In diesem Alert ist der Verlassen-Button immer selektiert, d.h. wenn Sie Return drücken würden, wird die Datei nicht gespeichert, sondern Interface wird sofort beendet.

Wenn Ihnen das zu gefährlich sein sollte, dann laden Sie die Interface-Resource, und fügen vor dem Speichern-Button des Alerts einen Punkt als erstes Zeichen ein. Dadurch ignoriert Interface die Grundeinstellung, und nimmt diesen Button als Default-Button.

Wenn Sie in der Alertbox den Verlassen-Button mit der Undo-Taste bedienen wollen, dann fügen Sie bitte als erstes Zeichen vor dem ›Verlassen‹-Text einen Doppelpunkt ein.

Diese Konfigurationsmöglichkeit ist mit allen Alertboxen machbar.

Nochmals kurz: ›.‹ steht für den Default-Button (wird mit Return selektiert) und ›:‹ kennzeichnet den Abbruch-Button (wird mit Undo selektiert).

Ein EXTOBFIX-Programms erstellen

Allgemeines

Interface stellt in seinen Dialogen und Menüs zahlreiche Features zur Verfügung, die vom Betriebssystem nicht unterstützt werden. Dazu verwendet es die MyDials, die zum Lieferumfang von Interface gehören, und die Sie selbst ebenfalls in Ihre C- und Basic-Programme integrieren können. Damit Interface auch andere Libraries beim Test eines Menüs oder Dialogs unterstützen kann, wurde das EXTOBFIX-Programm eingeführt. Zum besseren Verständnis ist es empfehlenswert, sich zunächst mit den MyDials zu beschäftigen.

EXTOBFIX?!

Interface versucht gleich nach dem Programmstart im eigenen Verzeichnis das Programm ›EXTOBFIX.PRG‹ zu laden. Wenn dieses Programm geladen werden konnte, ersetzt Interface Teile der Baum-Testroutinen durch Routinen des EXTOBFIX-Programms. Das EXTOBFIX kann sowohl die Anpassung von G_USERDEF-Objekten als auch die komplette Formular-Testroutine und die Alert-Testroutine ersetzen. Falls Sie in Ihrem Programm eigene Objekttypen verwenden (z.B. Checkboxes, Radiobuttons o.ä.), können Sie diese Objekte beim Testen von Ihren eigenen Routinen, die sich dazu in einem EXTOBFIX.PRG befinden müssen, zeichnen lassen.

Grundsätzlicher Aufbau

Sehen Sie sich dazu bitte den C-Quellcode ›EXTOBFIX.C‹ im Ordner ›MYDIAL.EXT‹ der Interface-Originaldiskette an. Das EXTOBFIX kann natürlich auch in anderen Programmiersprachen geschrieben werden, aber in C ist es am einfachsten.

Die Hauptroutine

Die Hauptroutine macht nichts anderes, als eine Warnmeldung auszugeben und das Programm wieder zu beenden, falls es nicht von Interface gestartet wurde (siehe ›main()‹). Da das EXTOBFIX im Speicher gehalten werden muß, bis Sie Interface beenden, wird die Hauptroutine nie von Interface angesprungen, weil das EXTOBFIX sonst wieder aus dem Speicher entfernt würde.

Die Initialisierung

Interface sucht zu Beginn im EXTOBFIX nach der Zeichenfolge ›06101964‹ oder ›06101965‹ (8 Byte »Magic«). Direkt dahinter erwartet es einen Zeiger auf die Initialisierungsroutine des EXTOBFIX.

Interface springt dann die Initialisierungsroutine an und übergibt ihr im Register D0 einen Wert. Wenn dieser Wert ungleich 0 ist, dann soll sich das EXTOBFIX initialisieren. Wenn dieser Wert 0 beträgt, soll sich das EXTOBFIX deinitialisieren. Im Fehlerfall oder nach der Deinitialisierung muß das EXTOBFIX im Register D0 den Langwert 0L, bei erfolgreicher Initialisierung einen Zeiger auf ein Zeigerarray zurückgeben.

Das Zeigerarray

In diesem Array stehen drei Zeiger auf Routinen des EXTOBFIX-Programms. Der erste Zeiger zeigt auf die ›fix_objs()‹ Routine, die normalerweise die G_USERDEF Objekte anpaßt und mit Zeigern auf die EXTOBFIX Zeichenroutinen versieht. Der zweite Zeiger zeigt auf die Dialog-Testroutine. Diese Routine ersetzt die komplette Interface-Testroutine der Dialogboxen. Der dritte Zeiger zeigt auf die Alertbox-Testroutine. Wenn das EXTOBFIX eine dieser Routinen nicht zur Verfügung stellt, muß der Zeiger den Wert 0 erhalten (siehe dazu EXTOBFIX für die Geiß-Routinen).

Die fix_objs Routine

Alle Koordinaten eines Baumes wurden von Interface bereits ins Geräteabhängige Format konvertiert. Außerdem wurde eine Kopie aller OBJECT-Strukturen, aller TEDINFO-Strukturen und aller te_ptext-Strings der EDITABLE G_FTEXT und G_FBOXTEXT TEDINFO-Strukturen angelegt. Das bedeutet, daß die EXTOBFIX Testroutine diese Daten verändern darf, da sie nach Beendigung des Tests wieder von Interface restauriert werden. Übergabeparameter in den CPU Registern:

A0: Zeiger auf das erste Objekt des Baumes.

D0: Wenn das Wort in D0 ungleich 0 ist, dann ist der Baum eine Dialogbox, sonst ein Pulldown-Menü. Die fix_objs Routine muß jetzt den gesamten Baum nach G_USERDEF-Objekten durchsuchen und in diese ggf. im ob_spec Feld einen Zeiger auf eine USERBLK-Struktur eintragen. Die USERBLK-Struktur muß in ub_code einen Zeiger auf eine eigene Zeichenroutine des EXTOBFIX für dieses Objekt enthalten. Im Parameter ub_parm sollte eine Kopie des alten ob_spec Wertes abgelegt werden.

Die Routine zum Testen von Dialogen

Diese Routine muß alle Objekte des Baumes anpassen (siehe fix_objs), die Dialogbox zentrieren, zeichnen und ausführen. Nach der Ausführung muß die Dialogbox auch wieder korrekt entfernt werden (entweder Hintergrund restaurieren, oder Redraw-Message durch form_dial(FMD_FINISH) senden). Übergabeparameter:

A0: Zeiger auf das erste Objekt des Baumes

D0: Nummer des default Editobjektes (Integer), ist immer 0.

D1: Zeiger auf die Namenssuchroutine von Interface (Long)

A1: Zeiger auf die WINDOW Struktur des Baumfensters. Dieser Zeiger muß der Namenssuchroutine von Interface übergeben werden.

Wenn Sie in dieser Routine den Namen eines Objekts ermitteln wollen, dann müssen Sie dazu die Namenssuchroutine von Interface aufrufen. Diese Routine braucht folgende Übergabeparameter:

A0: Zeiger auf die WINDOW Struktur des Baumfensters.

D0: Objektnummer.

A1: Zeiger auf einen mindestens 18 Zeichen langen Textbuffer.

Die Routine schreibt den Namen des Objektes mit 0 terminiert in den Textbuffer. Wenn kein Name gefunden werden konnte, wird im Textbuffer als erstes Zeichen ein Nullbyte abgelegt.

Die Routine zum Testen von Alerts

Diese Routine sollte eine Alertbox zeichnen und ausführen.

Übergabeparameter:

D0: Immer 1 (Integer). Soll der Default Exitbutton der Alertbox sein.

A0: Zeiger auf den Alertstring. Die Nummer des Alerticons kann Werte von 0 - Z annehmen (36 unterschiedliche Zeichen).

Eine eigene Hilfeseite

Interface unterstützt in jeder Extobfix-Datei eine Hilfeseite. Diese Hilfeseite kann in Interface in jedem Objekt-Dialog angezeigt werden. Der Zeiger auf den Baum der Hilfeseite muß im Funktionsarray an letzter Stelle definiert werden (siehe neue MyDial-Extobfix-Datei). Extobfix-Dateien mit Hilfeseite erhalten den Magic »06101965« statt »06101964«.

Der Baum wird von Interface an die erweiterten MyDial-Objekttypen angepaßt. Die ersten beiden Objekte (FlyDial-Ecke und Überschrift) müssen immer die Objektnummern 1 und 2 besitzen. Alle weiteren Objekte sind frei definierbar, die Anzahl der Objekte und die Größe des Dialogs ist ebenfalls nicht vorgegeben.

Wenn eine Extobfix-Datei mit Hilfeseite geladen wurde, wird von Interface bei Druck auf Help immer diese Seite angezeigt. Ansonsten wird eine Default-MyDial-Hilfeseite (schöne Wortkonstruktion ...) angezeigt.

Interface »ferngesteuert«

Interface wurde mit einer Accessory-Schnittstelle ausgestattet. Dadurch können Accessories auf alle internen Daten von Interface zugreifen und eigene Funktionen zur Verfügung stellen. Andere Funktionen, wie z.B. erweiterte Zeichenfunktionen im Iconeditor, lassen sich ebenfalls über die Accessory-Pipeline realisieren. Da Interface auf Routinen des Buches »Vom Anfänger zum GEM Profi« (2. Auflage) aus dem Hüthig Verlag basiert, ist die Lektüre dieses Buch Voraussetzung für jeden, der ein Programm schreiben möchte, das über die Accessory-

Schnittstelle auf Routinen von Interface zugreift. Da in der 2. Auflage einige Änderungen vorgenommen wurden, reicht die 1. Auflage des Buches nicht aus.

Eine genaue Erläuterung aller Routinen würde den Rahmen dieser Anleitung sprengen. Bisher hat sich - nach unserer Kenntnis - auch noch niemand die Mühe gemacht, Interface auf diesem Weg zu erweitern. Wenn Sie Erläuterungen zur Accessory-Schnittstelle benötigen, wenden Sie sich bitte schriftlich an no! Software oder via Modem über das MausNet direkt an den Autor von Interface: Olaf Meisiek @ FL

Ein Resource-Programm als Beispiel

In diesem Abschnitt soll an einem kleinen Programm in C gezeigt werden, wie man eine Resource-Datei lädt und eine Dialogbox daraus anzeigt. Die dazu benutzte Resource (»DEMO«) wurde im Handbuch schon mehrfach beschrieben.

Wenn Sie noch keine Erfahrung in der Programmierung von GEM-Applikationen haben, empfehlen wir die Lektüre der Bücher »Vom Anfänger zum GEM-Profi« und des »ATARI ST Profibuch«, die in Kapitel 1 kurz vorgestellt werden.

```

/*****
/*
/* Modul: DEMO.C
/* Datum: 23.02.93
/*
*****/

#include <string.h>
#include <stdio.h>

#include <portab.h>
#include <vdi.h>
#include <aes.h>

#include "demo.h"

/***** VARIABLES *****/

WORD   gl_apid;    /* Applikationsnummer */
OBJECT *planet;    /* Zeiger auf den Baum der Planeten-Dialogbox */

/***** FUNCTIONS *****/

BOOLEAN init_global (VOID);
BOOLEAN term_global (VOID);

/*****

main ()

{ WORD x, y, w, h;
  WORD button;
  BYTE alerttxt[400], ort[10];
  /* Applikation anmelden und Resource laden */
  if (init_global () == FALSE)
    return (0); /* Fehler, Programm sofort beenden */
  form_center (planet, &x, &y, &w, &h); /* Dialog zentrieren */
  /* Bildschirmspeicher reservieren */
  form_dial (FMD_START, 0, 0, 0, 0, x, y, w, h);
  objc_draw (planet, 0, 8, x, y, w, h); /* Dialogbox zeichnen */
  button = form_do (planet, 0) & 0x7fff; /* Dialog ausführen */
  if (button == PLOK) /* Dialogbox bei OK auswerten */

```

```

{
    if (planet[PLVENUS].ob_state & SELECTED)
        strcpy (ort, "Venus");

    if (planet[PLERDE].ob_state & SELECTED)
        strcpy (ort, "Erde");

    if (planet[PLMARS].ob_state & SELECTED)
        strcpy (ort, "Mars");
        sprintf (alerttxt,
            "[2][Sie kommen wirklich|vom Planeten %s?][ Nein |Ja]",
            ort);

    if (form_alert (2, alerttxt) == 2)
        form_alert (1, "[1][ |Faszinierend!][ Bye ]");
    else
        form_alert (1, "[3][ |Schade...][ Ende ]");
}

/* Bildschirmspeicher freigeben, Redraw erzeugen */
form_dial (FMD_FINISH, 0, 0, 0, 0, x, y, w, h);
term_global (); /* Applikation beenden */
return (0); /* Programm beenden */
}

/*****
/* Initialisieren des Moduls */
*****/

BOOLEAN init_global ()

{ WORD i;
  i = appl_init (); /* Applikationsnummer besorgen */
  gl_apid = i;
  if (gl_apid < 0) return (FALSE);

  graf_mouse (BUSY_BEE, NULL); /* Biene anzeigen */
  if (! rsrc_load ("DEMO.RSC"))
  {
    graf_mouse (ARROW, NULL); /* Pfeil anzeigen */
    form_alert (1, "[3][Resource-File|DEMO.RSC?][ EXIT ]");
    return (FALSE);
  } /* if */
  graf_mouse (ARROW, NULL); /* Pfeil anzeigen */

  rsrc_gaddr (R_TREE, PLANET, &planet); /* Adresse der Dialogbox */
  return (TRUE);
} /* init_global */

/*****
/* Terminieren des Moduls */
*****/

BOOLEAN term_global ()

```

```
{
  if (gl_apid >= 0)
  {
    rsrc_free ();           /* Ressourcen freigeben */
    appl_exit ();          /* Applikation beenden */
  } /* if */
  return (TRUE);
} /* term_global */
```

Die MyDials

Allgemeines

MyDials?!

Die MyDials sind eine Routinensammlung, die »fliegende« und über Tastatur bedienbare Dialoge auf einfache Weise ermöglichen. Sie bieten echte Radiobuttons (kleine runde Knöpfe), Checkboxes (ankreuzbare Kästchen), Popup-Menüs, komfortable Eingaberoutinen in Editfelder, verschiebbare Dialoge, tastaturbedienbare Buttons, Unterstützung der Help- und Undo-Tasten, Sonderzeicheneingabe und noch einiges mehr.

✖ Da die MyDials ständig weiterentwickelt werden, können an dieser Stelle nicht alle Funktionen erläutert werden. Wir verweisen auf die gesonderte MyDial-Anleitung im ASCII-Format auf den Interface-Disketten. Dort sind auch evtl. Änderungen zu finden.

Jeder, der Turbo C, Pure C oder einen Linker besitzt, der Turbo C bzw. Pure C-Librarys linken kann, kann die MyDials in seine Programme integrieren. Leider sind die Pure C Librarys nicht Digital Research-kompatibel, d.h. bis auf wenige Ausnahmen kann man MyDials nur in Verbindung mit Pure C, -Pascal bzw. Turbo C einsetzen. Der Einsatz der MyDials unter GFA- und Omikron-Basic ist ebenfalls möglich. Auf den Disketten befinden sich die dafür notwendigen Routinen. Da unsere Mitarbeiter keine speziellen Kenntnisse dieser Programmiersprachen haben, sollten Sie Fragen zum Basic schriftlich stellen. Wir werden sie dann an den Autor der jeweiligen Programmteile weiterleiten.

Für Modula-2 (Megamax-Modula) gibt es ein Programm, das Pure C-Librarys zu Modula-2 Programmen linken kann; dort sollte es also wieder machbar sein.

Auf besonderen Wunsch können wir jedem Interessenten auch eine Digital Research-kompatible MyDial-Library zuschicken. Das wird aber nur den wenigsten weiterhelfen, da die MyDials einige Funktionen aus der Pure C-Library aufrufen. Uns ist leider keine Möglichkeit bekannt, diese Funktionen ebenfalls in die MyDial-Library zu übernehmen. Daher würde eine Digital Research-kompatible Library nur denjenigen weiterhelfen, die außerdem eine Library besitzen, die die von den MyDials benötigten Routinen bereitstellt. Das könnte für einige andere C-Compiler auf dem Atari zutreffen.

Anmerkung: Die MyDials sind kein eigentlicher Bestandteil des Interface-Programmpakets, sondern eine Zugabe, für deren Vollständigkeit und Verwendbarkeit no! Software keine Haftung übernimmt.

Voraussetzung für die Verwendung der MyDials in eigenen Programmen ist, daß Sie Interface gekauft haben. Die MyDials sind weder Public Domain noch Freeware, d.h., daß die Weitergabe an Dritte der Weitergabe einer Raubkopie entspricht und dementsprechend strafbar ist. Die Anwendung der MyDials in eigenen - auch kommerziellen - Programmen bedarf keiner besonderen Genehmigung der no | Software GmbH. Im Programm ist allerdings ein Vermerk aufzunehmen, der auf die Verwendung der MyDials hinweist und no | Software als Inhaber des Copyrights an den MyDials benennt. Als Beispiel: ›Xyz verwendet die MyDials, Copyright (C) 1994 by no | Software GmbH.

Zu den MyDials gehören...

Zur Verwendung der MyDials benötigen Sie mehrere Dateien, auf die im folgenden kurz eingegangen wird. Bitte beachten Sie, daß die Beschreibung der MyDials von einer Verwendung mit Turbo C oder Pure C ausgeht.

NKCC

Im selbst entpackenden Archiv ›NKCC.TOS‹ finden Sie den ›National Key Code Converter‹, eine Library von Harald Siegmund, die von den MyDials verwendet wird. Diese Library ermöglicht es, daß jede Tastenkombination länderunabhängig korrekt erkannt und ausgewertet werden kann.

›NKCC_TOS.O‹ muß mit den MyDials gelinkt werden. Eine Anleitung zu NKCC steht in der Datei ›NKCC.DOC‹. Warnung: In dem Ordner befindet sich der Vollständigkeit halber auch die ›NKCC.O‹ Library. Diese Library sollte unter keinen Umständen verwendet werden, da sie unter MultiTOS oder Mag!X nicht fehlerfrei läuft. Diese Warnung gilt nur für die auf der Interface-Diskette mitgelieferte Version. Es ist durchaus möglich, daß es inzwischen eine neuere Version von Harald Siegmund (dem Autor von NKCC) gibt, die diese Probleme nicht mehr bereitet. Die MyDials benötigen aber in jedem Fall nur ›NKCC_TOS.O‹.

MYDIALG.LIB

In dieser Datei sind alle MyDial-Routinen enthalten. Außerdem enthält sie einige Routinen aus dem Modul ›GLOBAL‹ vom Buch ›Vom Anfänger zum GEM-Profi‹, die die MyDials benötigen.

MYDIAL.LIB

In dieser Datei sind ebenfalls alle MyDial-Routinen enthalten. Wenn Sie in ihrem Programm Routinen aus dem Buch ›Vom Anfänger zum GEM-Profi‹ verwenden und auch das Modul ›GLOBAL.C‹ eingebunden haben, dann müssen Sie diese Library verwenden. Sie müssen dann aber aus Ihrem Modul ›GLOBAL.C‹ die Routine ›popup_menu‹ löschen, da diese mit erweitertem Funktionsumfang bereits in den MyDials enthalten ist.

Bitte benutzen Sie die Software aus der 2. Auflage des Buches. Wenn Sie diese noch nicht haben sollten, dann wenden Sie sich bitte an den Hüthig Verlag. Wenn Sie dem Verlag Ihre Originaldiskette mit 10,- DM (Schein/Scheck) zuschicken, bekommen Sie die neueste Software-Version.

Die Deklarationen der MyDial-Routinen stehen in der Datei ›MYDIAL.H‹.

PORTAB.H

Bitte includen Sie diese Headerdatei vor der MyDial-Headerdatei. In ›PORTAB.H‹ sind alle für die MyDials wichtigen Definitionen enthalten. Näheres über diese Datei finden Sie im Buch ›Vom Anfänger zum GEM-Profi‹. Wer ›PORTAB.H‹ bereits besitzt, braucht es nicht durch diese Version zu ersetzen.

Funktionsweise der MyDials

Damit die MyDials Dialoge verschieben können und tastaturbedienbare Objekte bearbeiten können, müssen Sie etwas Vorarbeit leisten. Die MyDials überprüfen nämlich den Objekttyp und stellen bei besonderen Typen eigene Routinen für diese Objekte zur Verfügung. In der Regel handelt es sich bei diesen Objekten um sogenannte »erweiterte« Objekttypen.

Mit Interface kann man sehr leicht solch ein Objekt erzeugen. Wenn sie ein Objekt in Interface editieren, können sich rechts oben im Dialog das Feld Erweiterter Typ sehen. Dort trägt man einfach eine Zahl ein, um den MyDials später mitzuteilen, daß dieses Objekt speziell behandelt werden soll.

Für jede Objektart gibt es eine andere Nummer, die man eintragen kann. Manchmal muß man zusätzlich zur Nummer auch einen bestimmten »normalen« Objekttyp verwenden, oder bestimmte Flags setzen. In der Interface-Resource finden sie sehr viele Beispiele für solche erweiterten Objekte.

Die erweiterten Objekttypen

Ich werde jetzt alle von den MyDials unterstützten erweiterten Objekttypen aufzählen. Dabei werden der erweiterte Typ, der normale Objekttyp (soweit notwendig), die benötigten Objektflags und -states, sowie die Flags, die auf keinen Fall gesetzt werden dürfen, angegeben. Alle Flags, die weder unter die eine, noch unter die andere Kategorie fallen, dürfen bei Bedarf zusätzlich gesetzt werden.

Verschiebe-Objekt

Normaler Typ: beliebig

Erweiterter Typ: 17

Gesetzte Flags: TOUCHEXIT

Anmerkung: Wenn man mit der Maus irgendein Objekt mit dem erweiterten Typ 17 anklickt und die Maus bewegt, kann man dadurch den gesamten Dialog verschieben. Bei einem Doppelklick wird der Dialog zentriert.

FlyDial-Ecke

Normaler Typ: IBOX

Erweiterter Typ: 17

Gesetzte Flags: OUTLINED, CROSSED

Anmerkung: Dieses Objekt entspricht in der Funktion dem Verschiebeobjekt. Es wird aber von den MyDials als »Eselsohr« gezeichnet und sollte in der oberen rechten Ecke des Dialogs plaziert werden.

Radiobutton

Normaler Typ: BUTTON oder STRING

Erweiterter Typ: 18

Gesetzte Flags: RBUTTON (Radiobutton)

Anmerkung: Ein Radiobutton mit diesem Typ ist ein runder Button, neben dem der Text steht. Da der Button relativ klein ist, kann man ihn auch selektieren, indem man den daneben stehenden Text anklickt.

Checkbox

Normaler Typ: BUTTON

Erweiterter Typ: 18

Gesetzte Flags: -

Unge setzte Flags: RBUTTON, EXIT, SHADOWED

Anmerkung: Eine Checkbox sieht fast wie ein Radiobutton aus. An Stelle des runden Buttons wird aber eine quadratische Box gezeichnet. Wenn das Flag SELECTED gesetzt ist, wird eine Checkbox durchgekreuzt dargestellt. Wenn das Flag CHECKED gesetzt ist, aber SELECTED nicht gesetzt ist, wird die Checkbox mit einem Punktmuster gefüllt. Aus diesen 3 Zuständen kann man sich leicht einen »TriState-Button« programmieren, der z.B. von Interface verwendet wird, um in mehreren Objekten Flags gleichzeitig zu setzen (siehe Bearbeiten: Flags setzen...).

Popup-Button

Normaler Typ: `BUTTON`

Erweiterter Typ: `18`

Gesetzte Flags: `SHADOWED`

Ungesetzte Flags: `RBUTTON, EXIT`

Anmerkung: Dieser Button entspricht dem Exit-Button, darf aber nicht das `EXIT`-Flag zu besitzen. In der Interface-Resource wird er nur im Fileselektor für das Pfad-Popup verwendet. Er steht links neben dem Pfad, und enthält sinnigerweise den Text ›Pfad‹. Diesen Objekttyp sollte man normalerweise nicht für Popups verwenden, wenn der Popup-Button den Text des zuletzt angewählten Eintrages beinhalten soll. Interface verwendet für die eigenen Popups immer einen ganz normalen Box-Text-Button ohne erweiterten Objekttyp.

Exit-Button

Normaler Typ: `BUTTON`

Erweiterter Typ: `18`

Gesetzte Flags: `EXIT`

Ungesetzte Flags: `RBUTTON`

Anmerkung: Ein Exitbutton entspricht einem ganz normalen Button. Der einzige Unterschied ist, daß er tastaturbedienbar ist.

String

Normaler Typ: `STRING`

Erweiterter Typ: `18`

Ungesetzte Flags: `RBUTTON, EXIT`

Anmerkung: Dies ist ein tastaturbedienbarer String. Man könnte ihn z.B. für die einzelnen Einträge eines Popups verwenden, damit man einen Eintrag direkt über Tastatur auswählen kann. In diesem Fall muß der erweiterte Typ `24` sein.

Text, Boxtext

Normaler Typ: `TEXT` bzw. `BOXTTEXT`

Erweiterter Typ: `18`

Ungesetzte Flags: `RBUTTON, EXIT`

Anmerkung: Dies ist ein tastaturbedienbarer Text. (Siehe String)

Undo-Button

Erweiterter Typ: `18`

Gesetzte Flags: `ob_flags 11`

Anmerkung: Wenn `ob_flags 11` in irgendeinem Objekt vom erweiterten Typ `18` gesetzt wurde, kann man dieses Objekt auch über die Undo-Taste anwählen. Interface benutzt das z.B. für alle ›Abbruch‹-Buttons.

Unterstrich (Überschriften)

Normaler Typ: STRING oder BUTTON

Erweiterter Typ: 19

Anmerkung: Solch ein Objekt wird von den MyDials als unterstrichener String dargestellt. Interface benutzt diesen Objekttyp für alle Dialogüberschriften. Der Unterstrich hat die Länge der Objektbreite.

Boxtitel

Normaler Typ: BUTTON

Erweiterter Typ: 20

Anmerkung: Ein Boxtitel wird als großer Rahmen gezeichnet, in dem der Text des Objekts links oben als Überschrift steht. Boxtitel werden von Interface z.B. in dem Resource-Info Dialog eingesetzt.

Help-Button

Erweiterter Typ: 21

Anmerkung: Alle Objekte vom erweitertem Typ 21 lassen sich durch die Help-Taste auswählen. Der Typ und die Flags des Objekts sind egal, es hat sich aber eingebürgert, einen Help-Button als BoxText mit den Flags EXIT, OUTLINED und SHADOWED darzustellen. Der Text sollte Kleinschrift beinhalten und zentriert sein. Help-Buttons gibt es in Interface in jeder Objekt-Dialogbox.

Circle-Button

Normaler Typ: BOXCHAR

Erweiterter Typ: 22

Anmerkung: Ein Circle-Button wird in der Regel immer zusammen mit einem Popup verwendet. Der Button wird als kleine Box mit einem darin befindlichen »Kreispeil« dargestellt. Mit diesem Objekt kann man schnell den nächsten popup-Eintrag aufrufen, ohne erst das Popup anwählen zu müssen. Circle-Buttons werden von Interface z.B. in der BoxText-Dialogbox verwendet.

Undo-Button (zweite Version)

Erweiterter Typ: 31

Anmerkung: Ein Undo-Button entspricht in der Bedienung dem Help-Button (Typ 21). Natürlich dient hier die Undo-Taste zur Auswahl des Objekts.

Mehrzeilige Eingabefelder

Normaler Typ: FTEXT oder FBOXTEXT

Erweiterter Typ: 0 (Null)

Gesetzte Flags: EDITABLE, ob_flags 10

Anmerkung: Diesen Objekttypen gab es schon in den Interface 1.0 MyDials, er wurde aber nicht dokumentiert, da er etwas komplizierter zu handhaben ist. Mehrzeilige Eingabefelder werden von Interface z.B. in der FreeString-Dialogbox eingesetzt. Dabei verhalten sich mehrere aufeinanderfolgende Eingabefelder, als wenn sie ein einziges Feld wären.

Slider

Normaler Typ: (normalerweise) BOX oder BOXTEXT

Erweiterter Typ: 25

Anmerkung: Dieser Objekttyp dient dazu, einen Bereich im Dialog zu schaffen, in dem die Objekte automatisch verkleinert werden können. Man stelle sich ein Fenster im Dialog vor, bei dem die Randelemente so in ihrer Größe angepaßt werden, daß es keine Überschneidungen gibt und keine Probleme beim Scrollen auftreten. Siehe auch Funktion: dial_slinit.

Programmierung mit den MyDials

Ablaufschema

Am Programmstart:

- Initialisierung von NKCC
- Initialisierung der MyDials
- Initialisierung der Programm-Resource mit den MyDials

Danach können Sie in Ihrem Programm die MyDial-Routinen benutzen.

Am Programmende:

- Deinitialisierung der MyDials
- Deinitialisierung von NKCC

Ein Beispielprogramm

Das folgende Beispielprogramm soll einen Eindruck vermitteln, wie die MyDials in ein Hauptprogramm eingebunden werden. Dabei werden NKCC und die MyDials initialisiert. Die MyDials öffnen intern eine Workstation, damit sie nicht die Parameter des Hauptprogramms verstellen. In der alten MyDial-Version mußte NKCC ein VDI-Workstationhandle übergeben werden. Das ist nicht mehr nötig, da man nur noch die NKCC_TOS.LIB verwenden sollte, die keine VDI- und AES-Funktionen enthält.

```
#include <portab.h>
#include <aes.h>
#include <vdi.h>
```

```

#include <nkcc.h>
#include <mglobal.h>
#include <mydial.h>

WORD phys_handle, vdi_handle;

main()
{
    /* NKCC und die MyDials werden initialisiert */
    nkcc_init (NKI_NO200HZ, 0); /* Initialisiere NKCC */
    nkcc_set (0);
    if (dial_init (Malloc, Mfree, NULL, NULL, NULL) == TRUE)
    {
        /* Das Hauptprogramm folgt */

        dial_alert (NULL, "[0][Die Initialisierung!hat geklappt!][[OK]",
            1, 1, ALI_LEFT);

        /* Das war das Hauptprogramm... */

        dial_exit (); /* MyDials deinitialisieren */
    }
    nkcc_exit (); /* NKCC deinitialisieren */
    return (0);
}

```

Die Pure C-Projektdatei könnte für das kurze Programm z.B. so aussehen:

```

                ; MYDEMO.PRJ
                ; -----
mydemo.prg; Name
=                ; list of modules follows...
pcstart.o ; startup code
mydemo      ; Hauptprogramm
mydialg.lib ; Mydial lib
pcstdlib.lib ; standard lib
pctoslib.lib ; TOS lib
pcgemlib.lib ; AES and VDI lib
nkcc_tos.o; NKCC lib

```

Die Funktion der einzelnen MyDial-Routinen

Initialisierung

dial_init

GLOBAL BOOLEAN dial_init (VOID *alc, VOID *fr, WORD *mnum, MFORM **mform, BOOLEAN *gs);

- alc Zeiger auf eine Speicherreservierungs-Routine (z.B. ›Malloc‹)
- fr Zeiger auf eine Routine, die den reservierten Speicher wieder freigibt (z.B. ›Mfree‹).
- mnum Zeiger auf eine Integer-Variable, in der der zuletzt verwendete Maustyp steht (z.B. ›ARROW‹). Der Wert wird gebraucht, damit die MyDials immer die richtige Mausform restaurieren können. Die Maustypen werden dem AES-Befehl ›graf_mouse‹ übergeben. Eine Aufzählung aller möglichen Mausformen steht z.B. im Profibuch in der Erläuterung des AES-Befehls ›graf_mouse‹.
- mform Zeiger auf einen Zeiger, der auf eine selbstdefinierte Mausform zeigt. Dieser Zeiger wird nur gebraucht, wenn *mnum den Wert 255 (›USER_DEF‹) enthält.
- gs Wenn diese Variable den Wert TRUE enthält, zeichnen die MyDials Grow- und Shrinkboxen, sonst nicht.

Man braucht nicht alle Werte anzugeben. Wenn das eigene Programm diese Variablen nicht enthalten sollte, kann man auch NULL-Pointer übergeben. Der kürzestmögliche dial_init Aufruf sieht so aus:

```
ret = dial_init (Malloc, Mfree, NULL, NULL, NULL);
```

Wenn dial_init den Wert FALSE zurückgibt, hat die Initialisierung nicht geklappt. Das Programm sollte dann mit einer Fehlermeldung beendet werden. Die Initialisierung kann nur schiefgehen, wenn nicht genügend Speicher vorhanden ist.

dial_exit

GLOBAL BOOLEAN dial_exit (VOID);

- dial_exit deinitialisiert die MyDials. Wenn die MyDials am Programmstart initialisiert werden konnten, muß am Programmende dial_exit aufgerufen werden, damit die MyDials z.B. ihre eigene Workstation schließen können.

dial_fix

```
GLOBAL VOID dial_fix (OBJECT *tree, BOOLEAN is_dialog);
```

tree Adresse des Baumes

is_dialog Wenn TRUE, dann ist es eine Dialogbox, wenn FALSE, dann ist es ein Menübaum

dial_fix muß am Programmstart des Hauptprogramms nach der Initialisierung der MyDials für jeden Baum des Hauptprogramms einmal aufgerufen werden. Die MyDials initialisieren in dial_fix die erweiterten Objekttypen und tragen darin ihre Zeichenroutinen ein. Außerdem paßt dial_fix die Werte ›ob_width‹ und ›ob_height‹ der OBJECT-Struktur aller Icons und Images an die gültige Auflösung an, da das AES dabei bei ›src_load‹ Fehler macht.

Zu jedem benutzerdefinierten Objekt legen die MyDials die Struktur OBBLK an (Definition in ›MYDIAL.H‹). Diese Struktur enthält die USERBLK-Struktur und den ehemaligen Objekttyp, damit man auch nach der Anpassung den alten Objekttyp erfahren kann.

Die Dialogroutinen

Wenn man die Dialogroutinen einsetzen will, muß man unbedingt vorher den Bildschirm reservieren und nachdem der Dialog verschwunden ist, wieder freigeben.

Das sieht so aus:

```
wind_update (BEG_UPDATE); /* GEM Bildschirmausgaben anmelden */
/* Dialogbearbeitung */
wind_update (END_UPDATE); /* Bildschirm für andere Prozesse */
/* wieder freigeben */
```

Die meisten Programmierer rufen ›wind_update‹ direkt nach dem Haupt ›event_multi‹ auf, und können sich einen zweiten Aufruf vor den Dialogroutinen natürlich sparen. Wenn man ›wind_update‹ vergessen sollte, werden alle Mausklicks an andere Applikationen durchgereicht und selektieren z.B. Objekte auf dem Desktop, obwohl man eigentlich nur die Verschiebeecke des Dialogs angeklickt hatte...

HndlDial

GLOBAL WORD HndlDial (OBJECT *tree, WORD def, BOOLEAN grow_shrink, RECT *size, BOOLEAN *ok);

tree Adresse des Dialogbaumes

def Default-Editobjekt, in der Regel ROOT (ROOT = 0)

grow_shrink Bei TRUE Grow/Shrinkbox zeichnen, bei FALSE nicht zeichnen

size Ausgangsposition der Grow/Shrinkbox, bei NULL wird die Bildschirmmitte genommen

ok Bei TRUE konnte der Hintergrund restauriert werden. Bei FALSE führen die MyDials noch ein ›form_dial‹ (FMD_FINISH,...) aus, damit eine Redraw-Message abgeschickt wird. Wenn Sie zwei Dialogboxen schachteln (d.h. aus einer Box heraus noch eine zweite aufrufen), dann müssen Sie bei ok == FALSE die erste Box neu zeichnen, da der Hintergrund der zweiten Box beim Verlassen aus Speichermangel nicht restauriert werden konnte.

Als Rückgabewert erhalten Sie den Index des Objektes, durch das der Dialog verlassen wurde.

HndlDial führt eine komplette Dialogabarbeitung durch. Der Dialog wird gezeichnet, ausgeführt und wieder vom Bildschirm entfernt. Intern verwendet HndlDial dazu die Routinen open_dial, dial_draw, dial_do und close_dial (in dieser Reihenfolge).

open_dial

GLOBAL BOOLEAN open_dial (OBJECT *tree, BOOLEAN grow, RECT *size, DIALINFO *dialinfo);

tree Adresse des Dialogbaumes

grow Bei TRUE Grow/Shrinkbox zeichnen, bei FALSE nicht zeichnen

size Ausgangsposition der Grow/Shrinkbox, bei NULL wird die Bildschirmmitte genommen

dialinfo open_dial merkt sich in der Dialinfo-Struktur die Größe der Dialogbox inklusive aller Attribute und den Speicherbereich, in dem der Hintergrund der Dialogbox gesichert wurde. Die Dialinfo-Struktur wird u.a. von close_dial benötigt, um den Bildschirm wieder zu restaurieren. DIALINFO ist in »MYDIAL.H« deklariert.

Rückgabewert: Bei TRUE konnte der Hintergrund der Dialogbox nicht gespeichert werden. Bei FALSE führen die MyDials noch in der Funktion close_dial ein »form_dial (FMD_FINISH,...)« aus, damit eine Redraw-Message abgeschickt wird. Wenn Sie zwei Dialogboxen schachteln (d.h. aus einer Box heraus noch eine zweite aufrufen), dann müssen Sie bei ok == FALSE nach close_dial die erste Box neu zeichnen, da der Hintergrund der zweiten Box beim Verlassen aus Speichermangel nicht restauriert werden konnte.

open_dial zentriert den Dialog, zeichnet eine Grow/Shrinkbox und sichert den Desktopbereich, über dem die Dialogbox gezeichnet werden soll. Außerdem wird die DIALINFO-Struktur mit den entsprechenden Werten initialisiert. open_dial zeichnet den Dialog nicht. Das müssen Sie anschließend noch selber mit dial_draw oder »objc_draw« erledigen.

Der Dialog wird von open_dial nur zentriert, wenn sowohl die X- als auch die Y-Position des Dialoges 0 ist (ob_x und ob_y = 0). Dadurch erscheint der Dialog bei allen folgenden Aufrufen an der letzten Position, an die er verschoben wurde. Wenn man ein erneutes Zentrieren erzwingen will, muß man nur die X- und Y-Position des ROOT-Objekts wieder auf Null setzen.

close_dial

GLOBAL BOOLEAN close_dial (BOOLEAN shrink, RECT *size, DIALINFO *dialinfo);

shrink Bei TRUE Grow/Shrinkbox zeichnen, bei FALSE nicht zeichnen

size Endposition der Grow/Shrinkbox, bei NULL wird die Bildschirmmitte genommen

dialinfo Diese Struktur wurde von open_dial initialisiert. close_dial findet darin die Größe der Dialogbox und die Adresse des Hintergrundbuffers.

Rückgabewert: close_dial gibt immer TRUE zurück.

close_dial zeichnet eine Grow/Shrinkbox und restauriert den Bildschirm wieder. Wenn der Bildschirmhintergrund in open_dial aus Speichermangel nicht gesichert werden konnte, sendet close_dial noch eine Redraw-Message (FMD_FINISH) an das AES.

dial_center

GLOBAL VOID dial_center (OBJECT *tree);

tree Adresse des Dialogbaumes

dial_center initialisiert die Dialogbox so, daß sie genau in der Bildschirmmitte gezeichnet wird. Da dial_center intern von HndlDial bzw. open_dial ausgeführt wird, braucht man diese Funktion in der Regel nicht (es sei denn, man verzichtet auf HndlDial oder open_dial).

dial_start

```
GLOBAL BOOLEAN dial_start (OBJECT *tree, DIALINFO *di);
```

tree Adresse des Dialogbaumes

dialinfo dial_start merkt sich in der Dialinfo-Struktur die Größe der Dialogbox inklusive aller Attribute und den Speicherbereich, in dem der Hintergrund der Dialogbox gesichert wurde. Die Dialinfo-Struktur wird u.a. von close_dial benötigt, um den Bildschirm wieder zu restaurieren. DIALINFO ist in ›MYDIAL.H‹ deklariert.

Rückgabewert: Bei TRUE konnte der Hintergrund der Dialogbox nicht gespeichert werden. Bei FALSE führen die MyDials noch in close_dial ein ›form_dial (FMD_FINISH,...)‹ aus, damit eine Redraw-Message abgeschickt wird. Wenn Sie zwei Dialogboxen schachteln (d.h. aus einer Box heraus noch eine zweite aufrufen), dann müssen Sie bei ok == FALSE nach close_dial die erste Box neu zeichnen, da der Hintergrund der zweiten Box beim Verlassen aus Speichermangel nicht restauriert werden konnte.

dial_start initialisiert die DIALINFO-Struktur und buffert den Bildschirmbereich, über dem die Dialogbox gezeichnet werden soll. Vor dial_start muß man dial_center oder eine eigene Funktion aufrufen, damit im Dialogbaum die Zeichenposition gespeichert wird. dial_start wird von open_dial aufgerufen. Wenn man HndlDial oder open_dial verwendet, muß man dial_start nicht benutzen.

dial_draw

```
GLOBAL VOID dial_draw (DIALINFO *di);
```

di DIALINFO-Struktur, die durch open_dial oder dial_start initialisiert wurde.

dial_draw zeichnet den Dialog. dial_draw führt intern nur ein ›objc_draw‹ aus, wobei ›objc_draw‹ die Werte der DIALINFO-Struktur übergeben werden. Man kann also an Stelle von dial_draw auch ›objc_draw‹ verwenden, muß dann aber mehr Parameter übergeben.

dial_do

GLOBAL WORD dial_do (DIALINFO *di, WORD edit_obj);

di DIALINFO-Struktur, die durch open_dial oder dial_start initialisiert wurde.

edit_obj Default Editobjekt, in der Regel ROOT (ROOT = 0)

Als Rückgabewert erhalten Sie den Index des Objektes, durch das der Dialog verlassen wurde.

dial_do entspricht weitestgehend ›form_dial‹, ist aber wesentlich komfortabler. dial_do ruft automatisch die Verschiebefunktion auf (falls die FlyDial-Ecke angeklickt wurde) und wertet alle Shortcuts aus. Man kann den Cursor mit der Maus positionieren, mehrzeilige Eingabefelder ausfüllen und noch einiges mehr.

dial_do kann auch Eingaben in FBoxText-Objekte mit dem SMALL-Font vornehmen. ›form_do‹ würde dabei bestenfalls Pixelmüll auf dem Bildschirm anzeigen.

dial_move

GLOBAL VOID dial_move (DIALINFO *di, WORD x, WORD y, WORD w, WORD h);

di DIALINFO-Struktur, die durch open_dial oder dial_start initialisiert wurde.

x, y, w, h Rechteck, innerhalb dessen der Dialog verschoben wird. Normalerweise sollte man dafür die Desktopwerte ohne Menüzeile angeben.

dial_move verschiebt einen Dialog innerhalb eines Begrenzungsrechtecks. Wenn die Maustaste losgelassen wird, wird dial_move verlassen. dial_move wird von dial_do automatisch aufgerufen. Daher braucht man diese Funktion nur, wenn man die Dialogbearbeitung ohne dial_do komplett selbst vornehmen muß. In diesem Fall sollte man dial_move aufrufen, wenn der Benutzer ein Objekt mit dem erweiterten Objekttypen 17 angeklickt hat.

dial_end

GLOBAL VOID dial_end (DIALINFO *di);

di DIALINFO-Struktur, die durch open_dial oder dial_start initialisiert wurde.

dial_end restauriert den Hintergrund der Dialogbox und gibt den Speicher wieder frei. Wenn der Hintergrund nicht gebuffert werden konnte, sendet dial_end eine Redraw-Message (FMD_FINISH) an das AES. Da dial_end intern von dial_close aufgerufen wird, muß man diese Funktion in der Regel nicht benutzen.

dial_slinit

GLOBAL VOID dial_slinit (OBJECT *tree, WORD closer, WORD head, WORD edge, WORD vslbox, WORD up, WORD down, WORD hslbox, WORD left, WORD right);

tree Zeiger auf den Objektbaum.

closer, head, edge, vslbox, up, down, hslbox, left, right:

Objekte entsprechend den Fensterrandelementen

Anmerkung: Initialisieren von Sliderobjekten in einer Resource. Es wird nur eine Positionsanpassung durchgeführt, damit sich Objekte mit einem Rand von 1 Pixel außerhalb nicht überlappen. Übergabeparameter sind die Objektindizes der einzelnen Sliderelemente, für nicht vorhandene Objekte wird NIL übergeben. Allerdings müssen bei übergebenen Sliderboxen auch die Pfeile vorhanden sein. Die 'slbox'-Objekte sind der gesamte Sliderbereich, nicht nur der verschiebbare Teil.

Eigene Tastatúrauswertung in dial_do

Man kann in dial_do eine eigene Tastaturroutine »einklinken«. dial_do springt dann bei jeder Tastenfunktion erst diese Tastaturroutine an. Nur wenn diese meldet, daß es die Taste nicht auswerten konnte, wird die Taste danach noch von dial_do ausgewertet.

get_Keybd

GLOBAL FORMKEYFUNC get_Keybd (VOID);

Rückgabewert: Die Adresse der momentan eingeklinkten Routine. Bei NULL ist keine Routine eingeklinkt.

get_Keybd liefert die Adresse der momentan eingeklinkten Routine. Die Funktion wird z.B. gebraucht, wenn man aus einer Dialogbox mit einer eingeklinkten Routine eine zweite aufruft, in die ebenfalls eine eigene Tastaturroutine eingeklinkt werden soll. In diesem Fall muß man sich nämlich die Adresse der alten Funktion merken und die neue eintragen. Wenn die zweite Dialogbox verlassen wurde, muß man die Tastaturroutine der ersten Dialogbox wieder restaurieren. FORMKEYFUNC ist in »MYDIAL.H« deklariert und wird in bei der Erläuterung von set_Keybd beschrieben.

set_Keybd

```
GLOBAL VOID set_Keybd (FORMKEYFUNC fun);
```

fun Ein Zeiger auf eine eigene Funktion, die die Tastaturereignisse auswerten soll. Dieser Routine werden mehrere Parameter übergeben, die im großen und ganzen den Parametern von ›form_keybd‹ entsprechen.

Mit set_Keybd kann man eine eigene Tastaturroutine in dial_do einklinken, die die Tasten ggf selbst auswertet. Um den Vorgang etwas besser zu erläutern, folgt ein Ausschnitt aus dem Interface Fileselektor-Source. Der Fileselektor klinkt die Routine ›fsel_keybd‹ in dial_do ein, um alle Sondertasten auszuwerten.

Die einzelnen Parameter der FORMKEYFUNC-Struktur haben folgende Bedeutungen (siehe dazu auch fsel_keybd-Source):

tree	Adresse des Dialogbaumes
edit_obj	Objektnummer des aktuellen EDIT-Objektes
next_objlst	momentan immer 0 und wird nur aus Kompatibilitätsgründen zu ›form_keybd‹ übergeben
kr	Enthält den ASCII- und Scancode vom zweitletzten evnt_multi-Parameter (pkr). Wird von NKCC benötigt, um einen normalisierten Tastaturcode zu erzeugen.
ks	Enthält den Zustand der Sondertasten und entspricht dem drittletzten Parameter von ›evnt_multi‹ (pks).
onext_obj	Enthält das nächste Editobjekt (das Objekt, auf dem der Cursor stehen soll). Könnte man ändern, um den Cursor auf ein bestimmtes Objekt zu stellen.
okr	Entspricht kr. Mit diesem Parameter kann man eine andere Taste an die MyDials zurückgeben. Die MyDials werten dann diese neue Taste aus.

Wichtig: Die mit set_Keybd eingeklinkte Routine wird von den MyDials automatisch wieder indial_end auf NULL gesetzt, sobald der Dialog verlassen wurde. Man muß die Routine also bei dem nächsten Dialogaufruf wieder erneut einklinken.

```

/*****
Keyboard-Handler
*****/

LOCAL WORD fsel_keybd(OBJECT *tree, WORD edit_obj, WORD next_obj,
                      WORD kr, WORD ks, WORD *onext_obj, UWORD *okr)

{ WORD selobj, nkcc_key, clicks = 1, ascii;

  nkcc_key = normkey (ks, kr); /* Tastencode in NKCC-Code wandeln */

```

es folgen einige gelöschte (in diesem Zusammenhang unwichtige) Zeilen

```

...
{
    ascii = nkc_key & 0x00FF;

    switch ( nkc_key )
    {
        case NKF_FUNC|NKF_CTRL|NK_ESC:          /* ESCAPE          */
            selobj = SELHEAD;
            break;
        case NKF_FUNC|NKF_CTRL|SP:              /* SPACE            */
            selobj = SELHEAD;
            break;
        case NKF_FUNC|NKF_CTRL|NK_BS:           /* BACKSPACE        */
            selobj = SELCLOSE;
            break;
        case NKF_FUNC|NKF_CTRL|NK_CLRHOME:      /* CLR/HOME         */
            klicks = 2;
            selobj = USELECT;
            break;
        case NKF_FUNC|NKF_CTRL|NKF_SHIFT|NK_CLRHOME: /*SHIFT-CLR/HOME*/
            klicks = 2;
            selobj = DSELECT;
            break;
        case NKF_FUNC|NKF_CTRL|NK_UP:           /* Cursor oben      */
            selobj = USELECT;
            break;
        case NKF_FUNC|NKF_CTRL|NK_DOWN:         /* Cursor unten     */
            selobj = DSELECT;
            break;
        case NKF_FUNC|NKF_CTRL|NKF_SHIFT|NK_UP: /* SHIFT Cursor oben */
            selobj = BSELECT;
            break;
        case NKF_FUNC|NKF_CTRL|NKF_SHIFT|NK_DOWN: /*SHIFT Cursor unten*/
            selobj = SSELECT;
            break;
        case NKF_FUNC|NKF_CTRL|NKF_SHIFT|NK_LEFT: /* Cursor links    */
            selobj = INFOL;
            break;
        case NKF_FUNC|NKF_CTRL|NKF_SHIFT|NK_RIGHT: /* Cursor rechts   */
            selobj = INFOR;
            break;
    }

    if ( selobj != NIL && !(tree[selobj].ob_flags & HIDETREE) )
    {
        *okr = 0;
        return (form_button(tree, selobj, klicks, onext_obj));
    }
}

return (NIL);
}

```

Der Fileselektor wertet die Tasten aus. Wenn eine Tastenkombination erkannt wurde, führt der Fileselektor ein ›form_button‹ auf das entsprechende Objekt aus und gibt den Return-Wert von ›form_button‹ zurück.

Entweder wertet man die Taste selbst aus und gibt einen Wert zurück, den auch ›form_button‹ zurückgeben würde, oder man gibt NIL zurück, damit die Taste danach doch noch von den MyDials ausgewertet wird. Man kann den Rückgabewert auch verändern, indem man den neuen ASCII-Wert in okr zurückgibt. Die MyDials würden dann die Taste ins Editfeld eintragen, deren Wert in okr zurückgegeben wurde.

Ein Beispiel:

```
*okr = 127; return (NIL);
```

An dem Source kann man auch sehr schön sehen, wie leicht man Tastenkombinationen mit NKCC abfragen kann, ohne daß man eine Scancode-Tabelle benötigt. Da NKCC außerdem auch auf ausländischen Tastaturen immer korrekte Werte liefert, sollten Sie nicht mehr darauf verzichten.

Mehrzeilige Eingabefelder

In Interface gibt es mehrere Dialogboxen, die mehrzeilige Eingabefelder verwenden (z.B. Free String). Diese Felder werden weitestgehend von den MyDials verwaltet, trotzdem ist der Einsatz nicht besonders leicht, da das Hauptprogramm einige Dinge erledigen muß.

Alle zusammenhängenden Eingabefelder, die zu einem großen Feld zusammengefaßt werden sollen, müssen das selbe Parent besitzen. Wenn man mehrere mehrzeilige Eingabeobjekte in einer Dialogbox verwenden will, muß man sie in unterschiedlichen Parents unterbringen. Im Prinzip funktioniert das genauso wie bei Radiobuttons.

Man muß am Programmstart den benötigten Speicherplatz für den vollständigen String zur Verfügung stellen. In Interface läuft es wie folgt ab:

```
GLOBAL VOID fix_objs (tree)
OBJECT *tree;
{
    WORD    obj;
    OBJECT  *ob;
    UWORD   type, xtype;
    WORD    par, new, len;
    BYTE    *txt;

    if (tree != NULL)
    {
        obj = NIL;

        do
```

```

{
    ob      = &tree[++obj];          /* Adresse des Objekts holen */
    type    = ob->ob_type & 0xFF;     /* Objekttyp                */
    xtype   = ob->ob_type >> 8;       /* erweiterter Objekttyp     */

    switch (type)
    {
        case G_FTEXT :
        case G_FBOXTEXT:
            /* Länge eines mehrzeiligen Eingabeobjektes berechnen */
            if (ob->ob_flags & FLAGS10)
            { len = 0;
              par = parent (tree, obj);
              if (par == NIL) par = ROOT;
              new = tree[par].ob_head;
              while (new != NIL && new != par)
              { switch (tree[new].ob_type & 0xff)
                { case G_FTEXT :
                  case G_FBOXTEXT:
                      if (tree[new].ob_flags & FLAGS10)
                          len += ((TEDINFO *)tree[new].ob_spec)->te_txtlen-1;
                      break;
                }
              }
              new = tree[new].ob_next;
            }

            /* Für diese Länge Speicher allozieren, und die einzelnen */
            /* Felder zu einem Feld zusammenfassen                      */
            if (len)
            { if ((txt = mem_alloc (len + 1)) == NULL)
              dial_alert (NULL, alertmsg[NOMEMORY], 1, 1, ALI_LEFT);
              else
              { memset (txt, 0, len + 1);
                len = 0;
                new = tree[par].ob_head;
                while (new != NIL && new != par)
                { switch (tree[new].ob_type & 0xff)
                  { case G_FTEXT :
                    case G_FBOXTEXT:
                        if (tree[new].ob_flags & FLAGS10)
                        { ((TEDINFO *)tree[new].ob_spec)->te_ptext =
                          &txt[len];
                          len +=
                            ((TEDINFO*)tree[new].ob_spec)->te_txtlen-1;
                          undo_flags (tree, new, FLAGS10);
                          /* ob_flags 15 setzen, wenn das Editobjekt */
                          /* erfolgreich angepaßt werden konnte      */
                          do_flags (tree, new, FLAGS15);
                        }
                      break;
                    }
                }
                new = tree[new].ob_next;
            }
            }
    }
}

```

```

        }
    }
    break;
}
} while (!(ob->ob_flags & LASTOB));
} /* if */
} /* fix_objs */

```

In der Routine, die den Dialog aufruft und auswertet, muß folgendes geschehen:

```

LOCAL BOOLEAN edit_alert (window, obj)
WINDOWP window;
WORD    obj;
{
    /* unwichtiges gelöscht! */

    text = (BYTE *) (kf->tree);
    icon = al_token ((BYTE *) kf->tree, al_str, &txtnum, &butnum);

    /* Dialogfelder initialisieren, dabei Stringrest mit 0 auffüllen */
    for (i = 0; i < 5; i++)
    { if (!strcmp (al_str[i], " "))
        memset (((TEDINFO *) alerttree[ALZ1+i].ob_spec)->te_ptext, 0,
                strlen (
                    ((TEDINFO *) alerttree[ALZ1+i].ob_spec)->te_pvalid));
        else
            strncpy (((TEDINFO *) alerttree[ALZ1+i].ob_spec)->te_ptext,
                    al_str[i],
                    strlen (
                        ((TEDINFO *) alerttree[ALZ1+i].ob_spec)->te_pvalid));
    }

    /* Dialogabarbeitung */
    /* ... */
    /* Auswertung des Dialoginhaltes: */

    if (exit_obj == ALOK)
    {
        for (i = 0; i < 5; i++)
        { strcat (alstrg,
                ((TEDINFO *) alerttree[ALZ1+i].ob_spec)->te_ptext,
                strlen (
                    ((TEDINFO *) alerttree[ALZ1+i].ob_spec)->te_pvalid));
            strcat (alstrg, "|");
        }
        /* und so weiter... */
    }

    return (ret);
}

```

Um es kurz auszudrücken: Intern liegen die einzelnen Eingabefelder als ein einziger großer String im Speicher. Damit die einzelnen Felder trotzdem voneinander getrennt werden können, muß der unbenutzte Teil bei der Initialisierung mit Nullbytes aufgefüllt werden. Sonst könnte bei der Abarbeitung noch alter »Buchstabenmüll« auftauchen.

Bei der Auswertung muß man darauf achten, daß man mit »strncpy« arbeitet. Wenn Zeile 1 und 2 zusammenhängen (d.h. nicht mehr durch Nullbytes getrennt sind), würde man sonst bei einem »strcpy« auch sofort Zeile 1 und 2 gemeinsam geliefert bekommen. Wenn man das ganze in einer for...next Schleife abarbeitet, würde der nächste »strcpy«-Befehl dann nochmals Zeile 2 liefern!

find_next

```
GLOBAL WORD find_next (OBJECT *tree, WORD obj);
```

tree Adresse des Dialogbaumes

obj Startobjekt

Rückgabewert ist der Objektindex der nächsten Eingabezeile.

find_next sucht ab obj die nächste Zeile eines mehrzeiligen Eingabeobjektes.

find_next wird intern von den MyDials benutzt. Interface benötigt diese Routine für die Fensterdialoge, um das nächste Eingabeobjekt zu finden.

find_prev

```
GLOBAL WORD find_prev (OBJECT *tree, WORD obj);
```

tree Adresse des Dialogbaumes

obj Startobjekt

Rückgabewert ist der Objektindex der vorherigen Eingabezeile

find_prev sucht ab obj die vorhergehende Zeile eines mehrzeiligen Eingabeobjektes.

find_prev wird intern von den MyDials benutzt. Interface benötigt diese Routine für die Fensterdialoge, um das vorherige Eingabeobjekt zu finden.

Popup-Routinen

popup_menu

GLOBAL WORD popup_menu (OBJECT *tree, WORD obj, WORD x, WORD y, WORD center_obj, BOOLEAN relative, WORD bmsk, BOOLEAN *ok);

tree Adresse des Objektbaums, in dem sich das Popupmenü befindet
obj Wenn der Dialogbaum nur aus dem Popup besteht, ist obj = ROOT. Wenn sich mehrere Pops in dem Dialogbaum befinden, gibt obj das Parent der einzelnen Objekteinträge an (siehe auch Interface-Resource, Baum ›POPUPS‹).

x, y Position der linken oberen Popup-Ecke (bzw. des Objektzentrums, wenn center_obj != NIL)

center_obj Enthält den Index des zu zentrierenden Popup-Eintrags

relative TRUE: Popup wird an der Mausposition gezeichnet, x und y sollten dann 0 sein.

bmsk Gibt den Mausbutton an, mit dem das Popup verlassen wird (1 = links, 2 = rechts, 3 = links oder rechts)

ok TRUE, wenn Hintergrund gesichert werden konnte, sonst FALSE. Wenn ein Nullpointer übergeben wird, wird ok nicht gesetzt.

Als Rückgabewert erhalten Sie den ausgewählten Eintrag, oder NIL, wenn kein Eintrag ausgewählt wurde.

popup_menu zeichnet ein Popup und führt die komplette Bearbeitung durch. Im Popup dürfen beliebige Objekte stehen. Jedes selektierbare Objekt wird invertiert, wenn die Maus darüber steht oder wenn es mit den Cursortasten angefahren wird. Man kann auch tastaturbedienbare Strings im Popup unterbringen, und diese über den Shortcut anwählen.

Wenn ok FALSE enthält, muß die aufrufende Routine den Dialog an der Popup-Position neu zeichnen, da der Speicherplatz nicht reichte, um den Popup-Hintergrund zu sichern. Die Pops schicken in diesem Fall eine Redraw-Message ans AES.

popup_select

GLOBAL WORD popup_select (WORD handle, OBJECT *dialtree, WORD btn, OBJECT *poptree, WORD obj, BOOLEAN docheck, WORD docycle, BOOLEAN *ok));

handle Fensterhandle oder NIL, dann kein Redraw über Fensterhandle

dialtree Dialog-Baum

btn Index des Objekts, das das Popup aufruft (erlaubte Typen: BoxText, Button, String, Userdef-Boxtext oder Userdef-Button)

poptree Adresse des Objektbaums, in dem sich das Popupmenü befindet

obj Index des Pops. Die einzelnen Einträge müssen aus Strings oder tastaturbedienbaren Strings (Userdef 19) bestehen.

docheck Aktiven Popup-Eintrag (mit Button-Text) mit Haken versehen
 docycle DO_POPUP: Popup anzeigen;
 DO_CYCLE, DO_POPNEXT: nächsten Eintrag darstellen;
 DO_POPPREV: vorherigen Eintrag darstellen
 ok TRUE, wenn Hintergrund gesichert werden konnte, sonst FALSE. Wenn
 ein Nullpointer übergeben wird, wird ok nicht gesetzt.

Rückgabewert: Der ausgewählte Eintrag, oder NIL, wenn kein Eintrag ausgewählt wurde.

popup_select verwaltet einen Popup-Button mit einem Cycle-Button. Im Popup-Button wird immer der Text angezeigt, der zuletzt im Popup-Menü ausgewählt wurde. Wenn ein Cycle-Objekt angeklickt wurde, kann man an popup_select DO_CYCLE übergeben, damit die Routine in den Popup-Button den nächsten Text des Poppups einträgt.

Wenn man popup_select mit DO_POPUP aufruft, wird ein Popup angezeigt und abgearbeitet. Das Popup wird (wenn möglich) immer so positioniert, daß der zuletzt angewählte Eintrag genau über dem Popup-Button steht.

Alerts

do_alert

```
GLOBAL WORD do_alert (WORD defbut, CONST BYTE *txt);
```

do_alert entspricht in den Parametern ›form_alert‹. Der einzige Unterschied liegt darin, daß man nicht nur drei Icons, sondern sechzehn Icons zur Verfügung hat. Die Zahl des gewünschten Icons wird hexadezimal übergeben, d.h. die Werte von 0-F sind erlaubt. Die einzelnen Icons kann man sich in Interface ansehen.

Die MyDial-Alerts verfügen selbstverständlich über tastaturbedienbare Buttons. Vor dem Zeichen, das als Shortcut verwendet werden soll, gibt man einfach eine eckige Klammer ein. Ein gültiger Alert mit einem Shortcut auf dem ›O‹ von ›OK‹ könnte also so aussehen:

```
do_alert (1, "[5][Testalert][[OK]]");
```

Wenn man eine Textzeile des Alert zentriert oder rechtsbündig ausgeben möchte, gibt man dafür einfach ein spezielles Sonderzeichen an der ersten Position des Strings ein. Diese Sonderzeichen sind in ›MYDIAL.II‹ als ALCENTER (\001) und ALRIGHT (\002) definiert.

Ein Beispielalert:

```
do_alert (1, "[6][linksbündig|\001zentriert|\002rechtsbündig][[OK]]");
```

dial_alert

```
GLOBAL WORD dial_alert (OBJECT *alicon, CONST BYTE *string, WORD defbut, WORD undobut, WORD align);
```

- alicon** Zeiger auf die Objektstruktur eines Icons. Bei NULL wird die Iconangabe im String verwendet.
- string** Alertstring wie in ›form_alert‹
- defbut** Default-Exit-Button
- undobut** Default Undo-Button; wird bei Druck auf die Undo-Taste selektiert
- align** Positionieren des Alerttextes: ALI_LEFT, ALI_CENTER, ALI_RIGHT zentrieren den kompletten Alerttext linksbündig, zentriert oder rechtsbündig. Die 3 Konstanten sind in ›VDI.H‹ definiert. Einzelne Zeilen können durch voranstellen von ALCENTER oder ALRIGHT anders positioniert werden.

Rückgabewert: dial_alert gibt den gleichen Wert wie ›form_alert‹ zurück

dial_alert wird von do_alert aufgerufen. Mit dial_alert kann man ein eigenes Image im Alert anzeigen und einen Undo-Button definieren. Wenn der Text eines Buttons mit einem Punkt beginnt, wird er als Default-Button verwendet. Beginnt er mit einem Doppelpunkt, ist es der Default-Undo-Button. In diesen beiden Fällen werden die Werte von defbut und undobut ignoriert.

get_icon

```
GLOBAL BITBLK *get_icon (WORD icon);
```

- icon** Ziffer zwischen 0 - 15

Rückgabewert: Zeiger auf die BITBLK-Struktur des entsprechenden Alertbox-Icons der MyDials.

al_token

```
GLOBAL WORD al_token (CONST BYTE *f_alert, BYTE str[8][50], WORD *txtnum, WORD *butnum);
```

- f_alert** Zeiger auf den Alertstring
- str** Zeiger auf einen Buffer, auf den al_token die Strings aufteilen soll
- txtnum** Liefert die Anzahl der Textzeilen des Alerts
- butnum** Liefert die Anzahl der Alertbuttons

Rückgabewert: Nummer des Alert-Icons (z.B. für get_icon)

al_token teilt einen Alertstring in einzelne Strings auf und schreibt sie in str. Die ersten 5 Strings enthalten den normalen Text, die restlichen 3 Strings enthalten die Buttontexte

Diverse nützliche Hilfsroutinen der MyDials

getcookie

GLOBAL BOOLEAN getcookie (LONG cookie, LONG *p_value);

cookie Wert des zu suchenden Cookies

p_value Zeiger auf die Info-Struktur des Cookies.

Rückgabewert: Bei TRUE wurde der Cookie gefunden, sonst FALSE.

getcookie sucht einen Cookie. Die MyDials suchen nach dem ›VSCR‹- Cookie, um Dialoge auch unter Bigscreen und anderen virtuellen Bildschirmmanagern immer im sichtbaren Teil zu positionieren.

parent

GLOBAL WORD parent (OBJECT *tree, WORD obj);

tree Adresse des Dialogbaumes

obj Adresse des Objekts

Rückgabewert: parent liefert das übergeordnete Objekt (Parent) eines Objekts. Wenn es kein Parent gibt, weil obj = ROOT war, liefert parent NIL zurück.

get_obspec

GLOBAL LONG get_obspec (OBJECT *tree, WORD obj);

tree Adresse des Dialogbaumes

obj Adresse des Objekts

Rückgabewert = ob_spec-Wert des Objekts.

get_obspec liefert auch bei USERDEF-Objekten den korrekten ob_spec-Wert. Der ob_spec-Wert wird bei der Initialisierung des Objekts durch einen Zeiger auf eine USERBLK-Struktur überschrieben. In dieser USERBLK-Struktur wird der alte ob_spec-Wert wieder abgelegt. get_obspec liefert bei USERDEF-Objekten den ursprünglichen ob_spec-Wert aus der USERBLK-Struktur zurück.

set_obspec

GLOBAL VOID set_obspec (OBJECT *tree, WORD obj, LONG obspec);

tree Adresse des Dialogbaumes

obj Adresse des Objekts

obspec Neuer ob_spec-Wert

set_obspec ersetzt den alten ob_spec-Wert eines Objekts durch einen neuen Wert. Bei USERDEF-Objekten wird der neue Wert in die USERBLK-Struktur eingetragen.

normkey

```
GLOBAL UWORD normkey (WORD ks, WORD kr);
```

- ks** Der Zustand der Sondertasten. ks entspricht dem drittletzten Parameter von `evnt_multi` (pks).
- kr** der ASCII- und Scancode vom zweitletzten `evnt_multi`-Parameter (pkr). Wird von `›NKCC‹` benötigt, um einen normalisierten Tastaturcode zu erzeugen.

Mit `normkey` kann man den `›evnt_multi‹`-Tastencode in den normalisierten `NKCC`-Tastencode wandeln. Dadurch kann man wesentlich leichter und sicherer alle Sondertasten und Tastenkombinationen abfragen, da `›NKCC‹` tastaturunabhängige Werte liefert. Siehe dazu auch `›NKCC.H‹` und die `›NKCC‹`-Anleitung auf den Interface-Disketten.

get_idx

```
GLOBAL WORD get_idx (OBJECT *tree, WORD obj, WORD cpos);
```

- tree** Adresse des Dialogbaumes
- obj** Adresse des Objekts
- cpos** Cursorposition innerhalb eines Editobjekts

Rückgabewert: Die absolute Cursorposition. Wenn das Editobjekt so aussieht `›Datum: __/__/____‹`, und der Cursor auf der ersten Position (0) steht, beträgt die absolute Position 7, da vor der ersten Cursorposition 7 Buchstaben der Maske stehen.

`get_idx` wird von den MyDials intern für die MyDial-Funktion `obj_edit` gebraucht. Außerdem wird es von Interface für die Fensterdialoge verwendet.

form_Button

```
GLOBAL WORD form_Button (WORD handle, OBJECT *tree, WORD obj, WORD clicks, WORD *nextobj);
```

- handle** Fensterhandle
- tree** Adresse des Dialogbaumes
- obj** Objekt Nummer des zu bearbeitenden Objektes
- clicksj** Anzahl der zu simulierenden Mausklicks
- nextobj** Enthält nach der Rückkehr das neue aktuelle Objekt

Rückgabewert: siehe `TOS-›form_button‹`

`form_Button` entspricht der `TOS`-Funktion, bis auf das Berücksichtigen des Fensterhandles. Damit ist die Bedienung von Dialogen im Hintergrund möglich.

form_Keybd

```
GLOBAL WORD form_Keybd (OBJECT *tree, WORD edit_obj, WORD next_obj, WORD kr, WORD ks, WORD *onext_obj, UWORD *okr);
```

- tree** Adresse des Dialogbaumes
- edit_obj** Objekt Nummer des aktuellen EDIT-Objektes

- next_obj** Ist momentan immer 0 und wird nur aus Kompatibilitätsgründen an `form_keybd` übergeben
- kr** Enthält den ASCII- und Scancode vom zweitletzten `›evnt_multi‹`-Parameter (`pk`). Wird von `›NKCC‹` benötigt, um einen normalisierten Tastaturcode zu erzeugen.
- ks** Enthält den Zustand der Sondertasten und entspricht dem drittletzten Parameter von `›evnt_multi‹` (`pks`).
- onext_obj** Enthält nach der Rückkehr das nächste Editobjekt (das Objekt, auf dem der Cursor stehen soll).
- okr** Entspricht `kr`.

Rückgabewert: siehe `›form_keybd‹`

`form_Keybd` entspricht `›form_keybd‹`. Es ruft nur vor der Shortcut-Auswertung und vor der Ausführung von `›form_keybd‹` die einklinkbare Tastaturroutine auf (siehe `set_Keybd`). `form_Keybd` wird intern von den MyDials und von Interface für die Fensterdialoge verwendet.

shortcut

GLOBAL WORD shortcut (OBJECT *tree, WORD startobj, WORD ks, WORD kr);

- tree** Adresse des Dialogbaumes
- startobj** Anfangsobjekt, dessen Childs auf Shortcuts untersucht werden sollen
- ks** Enthält den Zustand der Sondertasten und entspricht dem drittletzten Parameter von `›evnt_multi‹` (`pks`).
- kr** Enthält den ASCII- und Scancode vom zweitletzten `›evnt_multi‹`-Parameter (`pk`). Wird von `›NKCC‹` benötigt, um einen normalisierten Tastaturcode zu erzeugen.

Rückgabewert: Objektindex des Objekts, daß den Shortcut enthält. Wenn der Shortcut nicht gefunden wurde, wird NIL zurückgegeben.

`shortcut` wird intern von den MyDials und von Interface für die Fensterdialoge verwendet.

obj_edit

```
GLOBAL WORD obj_edit (OBJECT *ob_edtree, WORD ob_edobject, WORD kstate, WORD
ob_edchar, WORD *ob_edidx, WORD ob_edkind, WORD mode, WORD *next_obj, WORD
winhndl);
```

ob_edtree	Adresse des Dialogbaumes
ob_edobject	Objektnummer des aktuellen EDIT-Objektes
kstate	Enthält den Zustand der Sondertasten und entspricht dem dritten Parameter von ›evnt_multi‹ (pks).
ob_edchar	Enthält den ASCII- und Scancode vom zweitletzten ›evnt_multi‹-Parameter (pkr). Wird von ›NKCC‹ benötigt, um einen normalisierten Tastaturcode zu erzeugen.
ob_edidx	Cursorposition innerhalb des Objekts
ob_edkind	Art der Editfunktion ED_INIT: obj_edit initialisieren, Cursor einschalten ED_CHAR: Zeichen verarbeiten ED_END : Cursor wieder ausschalten
mode	TRUE: obj_edit macht keine Bildschirmausgaben und fügt das Zeichen nur in den Editstring ein. Dadurch verhindert man ein hektisches Geflacker, wenn mehrere Zeichen auf einmal eingefügt werden sollen (z.B. Clipboard-String in Editfeld einfügen). TRUE wird nur von obj_edit intern verwendet, um sich rekursiv selbst aufzurufen. FALSE: Bildschirmausgaben sind erlaubt (Default-Mode)
next_obj	Enthält das nächste Edit-Objekt. Wird u.U. von obj_edit geändert.
winhndl	winhndl ist ein Fensterhandle. Wenn winhndl ungleich NIL ist, wird der Cursor über die Rechteckliste des Fensters gezeichnet, zu der winhndl gehört. Wird von Interface für die Fensterdialoge verwendet.
obj_edit	wird von den MyDials intern und von Interface für die Fensterdialoge verwendet. Durch das eigene obj_edit wurden die ganzen besonderen Tastaturfunktionen der MyDials verwirklicht.

scrap_clear

```
GLOBAL WORD scrap_clear (VOID);
```

scrap_clear löscht das GEM-Clipboard.

Alle MyDial-Clipboard-Routinen können nur funktionieren, wenn irgend ein Programm bereits den Clipboard-Pfad mit scrp_write eingestellt hat. Das Hauptprogramm sollte also vor dem MyDial-Einsatz (wenn nötig) den Clipboard-Pfad setzen und ggf. einen Clipboard-Ordner erzeugen.

ascii_head

```
GLOBAL VOID ascii_head (BYTE *header);
```

header Ein Zeiger auf die neue Überschrift der MyDial ASCII-Tabelle.

Da man die MyDial-Resource bestenfalls mit einem Diskettenmonitor ändern könnte gibt es diesen Aufruf, um damit den einzigen deutschsprachigen Text der MyDials zu ändern. `ascii_head` ändert nur den Zeiger des Textes, d.h. der übergebene Text muß auch weiterhin an der alten Stelle bleiben und darf nicht überschrieben werden.

my_menu_key

```
GLOBAL BOOLEAN my_menu_key (OBJECT *menu, MKINFO *mk, WORD *title, WORD *item);
```

menu Menübaum

mk Struktur, die die gedrückte Taste beschreibt

title Enthält als Ergebnis den Menütitel von item

item Enthält als Ergebnis den Menüeintrag, zu dem der Shortcut paßt

Rückgabewert: TRUE, wenn der Shortcut gefunden wurde, sonst FALSE

`my_menu_key` sucht in einem Menübaum nach einem Shortcut. In `mk` muß die Taste beschrieben werden, nach der die Funktion suchen soll. Die `MKINFO`-Struktur ist in `GLOBAL.H` definiert. Die `MKINFO`-Struktur muß wie folgt mit den `evnt_multi`-Werten gefüllt werden:

```
/* direkt hinter dem Haupt-evnt_multi folgt: */ mk.scan_code = mk.kreturn;
```

```
mk.kreturn = normkey (mk.kstate, mk.kreturn);
```

```
if (mk.kreturn & NKF_SHIFT) /* beide SHIFT Tasten setzen */
```

```
mk.kreturn |= NKF_SHIFT;
```

```
mk.kreturn &= KF_CAPS; /* CAPSLOCK löschen */
```

```
mk.shift = (mk.kstate & (K_RSHIFT | K_LSHIFT)) != 0;
```

```
mk.ctrl = (mk.kstate & K_CTRL) != 0;
```

```
mk.alt = (mk.kstate & K_ALT) != 0;
```

`mk.kstate` und `mk.kreturn` enthalten vor der Ausführung dieser Zeilen die Rückgabewerte `kstate` und `kreturn` von `evnt_multi`.

Wenn man die Geiß-Librarys verwendet, kann man die Geiß-Routine `is_menu_key` durch `my_menu_key` ersetzen. `my_menu_key` erkennt auch kombinierte Shortcuts wie z.B. Shift Control U.

get_sysfnt

GLOBAL BOOLEAN get_sysfnt (WORD vdi_handle, WORD *font_id, WORD *gl_point, WORD *gl_wchar, WORD *gl_hchar);

vdi_handle aktuelle Workstation

font_id liefert die Systemfont ID (siehe ›vst_font‹)

gl_point liefert die Punktgröße des Systemfonts (siehe ›vst_point‹)

gl_wchar liefert die Breite des Systemfonts gl_hchar liefert die Höhe des Systemfonts (siehe ›vst_height‹)

get_sysfnt wird von den MyDials beim Öffnen der Workstation verwendet, um den aktuellen Systemfont und die Zeichengröße zu erfragen. Die Funktion liefert auch unter MultiTOS korrekte Ergebnisse. Außerdem setzt die Funktion für die übergebene Workstation den ermittelten Systemfont.

Ein Beispielprogramm für die Dialogbearbeitung

HndlDial arbeitet einen Dialog komplett ab, deshalb erspare ich mir dafür ein Beispiel. Wenn man einen Dialog mit open_dial und close_dial abarbeiten muß, um u.U. Touchexit-Objekte auszuwerten und Teile des Dialoges neu zu zeichnen, kann man wie folgt vorgehen:

```
LOCAL VOID show_form (tree, editobj)
OBJECT *tree;
WORD editobj;
{ WORD but, zw;
  DIALINFO di;

  open_dial (tree, TRUE, NULL, &di);

  dial_draw (&di);

  /* Solange Touchexit-Objekte angeklickt wurden, soll der Dialog */
  /* nicht verlassen werden. */

  do
  {
    but = dial_do (&di, editobj) & 0x7fff;
    tree[but].ob_state &= ~SELECTED;
    objc_draw (tree, ROOT, MAX_DEPTH, di.x, di.y, di.w, di.h);
  }
  while (tree[but].ob_flags & TOUCHEXIT);

  close_dial (FALSE, NULL, &di);
}
```

Dialoge in Fenster legen

Es ist nicht besonders schwer, einen Dialog in ein Fenster zu legen, es ist nur viel Arbeit. Im Prinzip funktioniert es so:

1. Man öffnet ein Fenster mit der Größe des Dialoges.
2. Man zeichnet den Dialog in das Fenster mit dem Clipping der Fensterrechteckliste.
3. Der Fenster-form_do (den Sie noch programmieren müssen) wird angesprochen, wenn...
 - sich der Dialog im obersten Fenster befindet und Ihr Programm ein Tastaturevent gemeldet bekommt.
 - Ihnen ein Button-Event gemeldet wurde und unter der Maus der Fensterdialog liegt.
4. Wenn ein Exit- oder Touchexit-Objekt angeklickt wurde, wird eine Routine angesprochen, die das Objekt auswertet und zurückmeldet, ob der Dialog nach der Auswertung geschlossen werden soll.

Damit das ganze auch funktioniert, muß man natürlich wissen, wie der ›form_do‹ (oder besser noch dial_do) Befehl intern arbeitet. Denn man muß ja die Ereignisse genauso verarbeiten, damit es keine Unterschiede in der Handhabung des Dialoges gibt. Deshalb folgt jetzt der MyDial-dial_do Source:

```

/*****/

GLOBAL WORD dial_do (di, edit_obj)
DIALINFO *di;
WORD      edit_obj;

{ WORD  but;

  do
  {
    but = form_xdo (di->Tree, edit_obj);
    if ((di->Tree[but & 0x7fff].ob_type >> 8) == DIALMOVER)
      /* Dial Mover */
      {
        graf_mouse (FLAT_HAND, NULL);
        dial_move (di, mydesk.x, mydesk.y, mydesk.w, mydesk.h);
        graf_mouse (*my_mousnumber, *my_mousform);
      }
    else
      return (but);
  }
  while (TRUE);
}

/*****/

```

```

LOCAL WORD find_obj (tree, start_obj, which)
OBJECT *tree;          /* routine to find the next editable */
WORD start_obj;        /* text field, or a field that is as */
WORD which;            /* marked as a default return field. */

{ WORD obj, flag, theflag, inc;

  obj = 0;
  flag = EDITABLE;
  inc = 1;

  switch (which)
  {
    case FMD_BACKWARD:
      inc = -1;          /* fall thru */
    case FMD_FORWARD:
      obj = start_obj + inc;
      break;
    case FMD_DEFLT:
      flag = DEFAULT;
      break;
  }

  while (obj >= 0)
  {
    theflag = tree[obj].ob_flags;

    if (theflag & flag)
      return (obj);

    if (theflag & LASTOB)
      obj = -1;
    else
      obj += inc;
  }

  return (start_obj);
}

/*****/

LOCAL WORD fm_inifld (tree, start_fld)
OBJECT *tree;
WORD start_fld;

{
  if (!start_fld)        /* position cursor on the starting field */
    start_fld = find_obj(tree, 0, FMD_FORWARD);

  return (start_fld);
}

/*****/
/* Neue form_do Routine */

```

```

/*****/

LOCAL WORD form_xdo (tree, start_fld)
OBJECT *tree;
WORD start_fld;

{ WORD edit_obj;
  WORD next_obj;
  WORD which, cont;
  WORD idx, len, wbox;
  WORD mx, my, mb, ks, br;
  UWORD kr, knorm;
  RECT work;
  TEDINFO *ob_spec;

  wind_update (BEG_UPDATE);
  wind_update (BEG_MCTRL);

  /* set starting field to edit, if want first editing field
     then walk tree */
  next_obj = fm_inifld (tree, start_fld);
  edit_obj = 0;

  cont = TRUE; /* interact with user */
  while (cont)
  {
    /* position cursor on the selected editing field */
    if ((next_obj != 0) && (edit_obj != next_obj))
    {
      edit_obj = next_obj;
      next_obj = 0;
      obj_edit(tree, edit_obj, 0, 0, &idx, EDINIT, FALSE,
               &edit_obj, NIL);
    }

    /* wait for mouse or key */
    which = evnt_multi(MU_KEYBD | MU_BUTTON,
                      0x02, 0x01, 0x01,
                      0, 0, 0, 0, 0,
                      0, 0, 0, 0, 0,
                      NULL,
                      0, 0,
                      &mx, &my, &mb, &ks, &kr, &br);

    if (which & MU_KEYBD) /* handle keyboard event*/
    {
      knorm = normkey (ks, kr);
      {
        if ((tree[edit_obj].ob_flags && FLAGS15) &&
            (knorm == (NKF_FUNC|NK_UP) &&
             find_prev (tree, edit_obj) != NIL) ||
            (knorm == (NKF_FUNC|NK_DOWN) &&
             find_next (tree, edit_obj) != NIL))
        {
          obj_edit(tree, edit_obj, ks, kr, &idx, EDCHAR, FALSE,

```

```

        &edit_obj, NIL);
    next_obj = edit_obj;
    kr = 0;
}
else
    cont = form_Keybd(tree, edit_obj, next_obj, kr, ks,
        &next_obj, &kr);
}

if (kr)
{
    obj_edit(tree, edit_obj, ks, kr, &idx, EDCHAR, FALSE,
        &edit_obj, NIL);
    next_obj = edit_obj;
}
else
    if ((tree[edit_obj].ob_flags & EXIT) &&
        edit_obj != next_obj && cont)
    {
        next_obj = edit_obj;
        cont = edit_obj = FALSE;
    }
}

if (which & MU_BUTTON)                /* handle button event */
{
    next_obj = objc_find(tree, ROOT, MAX_DEPTH, mx, my);
    if (next_obj == NIL)
    {
        mybeep();
        next_obj = 0;
    }
    else
    {
        cont = form_button(tree, next_obj, br, &next_obj);

        if ((tree[edit_obj].ob_flags & EXIT) &&
            edit_obj != next_obj && cont)
        {
            next_obj = edit_obj;
            cont = edit_obj = FALSE;
        }
        if (cont && (tree[next_obj].ob_flags & EDITABLE))
        {
            if (next_obj != edit_obj)
            {
                obj_edit(tree, edit_obj, ks, 0, &idx, EDEND, FALSE,
                    &edit_obj, NIL);
                edit_obj = next_obj;
                next_obj = 0;
            }
            objc_rect(tree, edit_obj, &work, FALSE);

            ob_spec = (TEDINFO *)get_obspec(tree, edit_obj);

```

```

    if (ob_spec->te_font == SMALL)
        wbox = 6;
    else
        wbox = mygl_wbox;

    switch (ob_spec->te_just)
    {
        case TE_RIGHT:
            work.x += work.w - wbox * (ob_spec->te_tmplen - 1);
            break;
        case TE_CNTR:
            work.x += (work.w - wbox * (ob_spec->te_tmplen - 1)) / 2;
            break;
    }
    idx = (mx - work.x + wbox / 2) / wbox;
    len = ob_spec->te_tmplen - 1;
    if (idx > len)
        idx = len;
    idx = get_idx (tree, edit_obj, idx);
    obj_edit(tree, edit_obj, ks, 0, &idx, EDCHAR, FALSE,
            &edit_obj, NIL);
}
}
}
/* handle end of field, clean up */
if ((!cont) || ((next_obj != 0) && (next_obj != edit_obj)))
    obj_edit(tree, edit_obj, ks, 0, &idx, EDEND, FALSE,
            &edit_obj, NIL);
}

wind_update(END_MCTRL);
wind_update(END_UPDATE);

/* return exit object, hi bit may be set if
   exit obj. was double-clicked */
return (next_obj);
}

```

Alle Routinen, die von `form_xdo` aufgerufen werden, sind in `MYDIAL.H` von außen zugänglich. Man muß also eigentlich nur noch das aufgetretene Event an die Routine übergeben und es anstelle des `form_xdo-evnt_multi` auswerten. Etwas komplizierter ist es zwar schon, aber mit diesen Routinen sollte jeder, der sich etwas mit Fenstern auskennt, seine Dialoge in Fenster legen können.

Etwas leichter haben es alle, die die Geiß-Routinen aus dem Buch »Vom Anfänger zum GEM-Profi« in der zweiten Auflage benutzen. In dieser Auflage werden nämlich bereits Fensterdialoge benutzt und man muß eigentlich nur noch im Modul `WINDOW.C` die Routinen `click_window` und `key_window` an die MyDials anpassen.

In Interface kann man jederzeit zwischen Fensterdialogen und normalen Dialogen umschalten, weil Interface einfach die Dialogüberschrift in den Fenstertitel übernimmt und das Fenster so klein macht, daß die FlyDial-Ecke und die Dialogüberschrift nicht angezeigt werden.

form_xdo basiert übrigens auf dem Original Digital Research ›form_do‹-Source. Ich hab's nur »etwas« an die MyDials angepaßt.

Resourcen von mehr als 64 KB Größe benutzen

Auf den Interface-Disketten finden sie das selbst entpackende Archiv ›XRSRC.TOS‹, das die beiden Dateien ›XRSRC.C‹ und ›XRSRC.H‹ enthält. Wenn Sie diese Dateien in Ihr C-Programm aufnehmen, können Sie damit (fast) beliebig lange Ressourcen laden (theoretisch bis zu 4 Gigabyte).

Sie müssen dazu keine großen Änderungen an Ihrem Programm vornehmen, sondern nur eine Initialisierungsfunktion aufrufen, vor einigen AES-Kommandos ein ›x‹ einfügen und einen weiteren Parameter mit angeben. Folgende Kommandos müssen umbenannt werden:

```

rsrc_load in xrsrc_load
rsrc_free in xrsrc_free
rsrc_gaddr in xrsrc_gaddr
rsrc_saddr in xrsrc_saddr
rsrc_obfix in xrsrc_obfix

```

Bis auf xrsrc_obfix müssen Sie jedem Befehl noch einen Zeiger auf ein 15 Integer großes Array übergeben. Darin merken sich die XRSRC-Routinen alle wichtigen Daten der Resource. Wenn Sie zwei oder mehr Ressourcen laden wollen, brauchen Sie nur für jede weitere Resource ein anderes Array zu verwenden.

Bevor Sie xrsrc_load aufrufen, müssen Sie die XRSRC-Routinen mit dem Befehl init_xrsrc initialisieren. init_xrsrc müssen die folgenden Parameter übergeben werden:

vdi_handle	Handle der VDI-Workstation, die die XRSRC-Routinen benutzen sollen
desk	Zeiger auf eine x,y,w,h-Struktur mit X-Position, Y-Position, Breite und Höhe des Desktops
gl_wbox	Breite eines Zeichens
gl_hbox	Höhe eines Zeichens

Wenn man keine Farbicons verwendet, kann man für vdi_handle 0 übergeben, da die XRSRC-Routinen diese Variable dann nicht benötigen.

An `gl_wbox` und `gl_hbox` kommt man mit folgendem Aufruf:

```
graf_handle (&gl_wbox, &gl_hbox, &gl_wattr, &gl_hattr);
```

desk erhält man mit:

```
wind_get (DESK, WF_WXYWH, &desk.g_x, &desk.g_y, &desk.g_w, &desk.g_h);
```

desk ist eine Struktur aus 4 Integern, die z.B. in ›AES.H‹ als GRECT deklariert ist.

Die veränderte RSHDR-Struktur ist in ›XRSRC.H‹ definiert.

Der Source liegt in C und Basic vor. Die GFA- und Omikron-Routinen wurden aber nicht von uns entwickelt, sondern uns von interessierten Programmierern zur Verfügung gestellt. Bei Problemen mit diesen Routinen werden wir Anfragen an die entsprechenden Autoren weiterleiten. Die Omikron-Version der Routinen enthielt zum Zeitpunkt als diese Anleitung geschrieben wurde noch keine Farbicon-Unterstützung.

Wenn irgend jemand die Routinen in eine andere Sprache übersetzen möchte, helfen wir ihm gerne dabei. Falls jemand die Routinen in einer anderen Sprache brauchen sollte und sie nicht selbst übersetzen kann, sollte er bei no! Software anfragen, ob es inzwischen für die gewünschte Sprache eine Version gibt.

Die folgenden Erläuterungen beziehen sich auf die C-Version der XRSRC-Routinen.

Farbicon-Unterstützung

Seit Interface 2.2 unterstützen die XRSRC-Routinen auch die Falcon-Farbicons für alle TOS-Versionen. Die Farbicon-Unterstützung kann man mit dem Compilerschalter `COLOR_ICONS` ein- bzw. ausschalten. Wenn man `COLOR_ICONS` auf `FALSE` setzt, werden die Farbicon-Routinen nicht mit-compiliert.

Mit dem Schalter `SAVE_MEMORY` kann man bestimmen, ob die Farbicon-Routinen möglichst speichersparend vorgehen sollen. Wenn `SAVE_MEMORY` auf `TRUE` gesetzt wird, versuchen die Routinen die Farbicons nach der Wandlung in das geräteabhängige Format die geräteunabhängigen Daten durch die geräteabhängigen Daten zu ersetzen. Nur wenn die geräteabhängigen Daten länger als die alten Daten sein sollten (z.B. in Auflösungen mit mehr als 256 Farben), wird dafür Speicher angefordert.

`SAVE_MEMORY` sollte man nur dann auf `FALSE` setzen, wenn das eigene Programm auch später noch auf die geräteunabhängigen Daten zugreifen muß. Das ist z.B. für einen Resource-Editor oder einen Iconeditor notwendig.

Wenn nicht genug Speicher für die Wandlung vorhanden sein sollte, werden die Farbicons in normale monochrome Icons gewandelt, damit sie trotzdem noch angezeigt werden können.

Anzeige der Farbicons

Der `xrsrc_load`-Befehl wandelt beim Laden der Resource automatisch alle darin befindlichen Farbicons in `USERDEFS` und trägt auch gleich eine eigene Zeichenroutine dafür ein. Dadurch können Sie die Farbicons wie ganz normale Objekte mit `>objc_draw<` zeichnen lassen und brauchen sie nicht anders als »normale« Icons in Ihrem Programm behandeln.

Besonderheiten des Farbicon-Formats

Der `>ob_spec<`-Zeiger

Da die `XRSRC`-Routinen die Farbicons in `USERDEFS` gewandelt haben, kommt man nicht mehr direkt über den `>ob_spec<`-Wert an die `ICONBLK`-Struktur. `>ob_spec<` zeigt jetzt auf eine `USERBLK`-Struktur, in der der alte `>ob_spec<`-Wert im `>ub_parm<`-Feld gesichert wurde.

Wenn man die `MyDials` verwendet, hat man es leicht: Man benutzt einfach den `get_obspec` Aufruf und erhält immer den gewünschten Wert. Diese Funktion überprüft erst, ob es sich um ein `USERDEF`-Objekt handelt und liefert in dem Fall den `>ob_spec<`-Parameter aus der `USERBLK`-Struktur.

Ohne die `MyDials` muß man diese Funktion in das eigene Programm einbinden. Damit das nicht zu schwer fällt, folgt jetzt der Source:

```
LONG get_obspec (OBJECT *tree, WORD obj)
{
    if ((tree[obj].ob_type & 0xff) == G_USERDEF)
        return (((USERBLK *)tree[obj].ob_spec)->ub_parm);
    else
        return (tree[obj].ob_spec);
}
```

Wenn man ein Icon mit einem anderen Text versehen will, schreibt man in C folgendes:

```
strcpy (((ICONBLK *)get_obspec (tree, obj))->ib_ptext, "Neuer Text");
```

Die Farbicon-Struktur

Die neuen Farbicons sind von Atari wie folgt dokumentiert worden:

```
#define G_CICON          33      /* Farbicon-Objektyp */

typedef struct cicon_data
{
    WORD num_planes;      /* number of planes in the following data */
    WORD FAR *col_data;   /* pointer to color bitmap in standard form */
    WORD FAR *col_mask;   /* pointer to single plane mask of col_data */
    WORD FAR *sel_data;   /* pointer to color bitmap of selected icon */
    WORD FAR *sel_mask;   /* pointer to single plane mask of selected
                           icon */
    struct cicon_data *next_res; /* pointer to next icon for a different
                                resolution */
} CICON;

typedef struct cicon_blk
{
    ICONBLK monoblk; /* default monochrome icon */
    CICON *mainlist; /* list of color icons for different resolutions */
} CICONBLK;
```

Am besten ist es, wenn Sie diese Definitionen direkt in Ihre ›AES.H‹-Headerdatei übernehmen.

Ein Farbicon sieht jetzt so aus, daß der ›ob_spec‹-Pointer der OBJECT-Struktur auf eine CICONBLK-Struktur zeigt. Da der erste Eintrag der CICONBLK-Struktur eine ICONBLK-Struktur beinhaltet, kann man auf diesen Teil der Farbicons genauso wie auf ein monochromes Icon zugreifen.

Ein Farbicon besteht real aus einer beliebigen Anzahl von einzelnen Bildern für beliebige Auflösungen. Das Betriebssystem sucht sich im ›src_load‹-Befehl aus dieser Iconliste immer das Icon aus, das am besten zur aktuellen Auflösung paßt. Dabei können alle Icons verwendet werden, die genausoviel oder weniger Planes besitzen. Der Zeiger mainlist der CICONBLK-Struktur zeigt auf eine CICON-Struktur, in der die Bilddaten für ein Icon enthalten sind, das num_planes Planes benutzt.

Der next_res-Zeiger dieser CICON-Struktur kann dann wieder auf eine weitere CICON-Struktur für eine andere Auflösung zeigen. Das geht solange, bis next_res NULL enthält.

Eine CICON-Struktur enthält 4 Zeiger. Die ersten beiden werden für das normale Bild und die Maske verwendet, die Zeiger 3 und 4 zeigen auf das Selektionsbild und die Selektionsmaske. Das Selektionsbild wird angezeigt, wenn das Icon den Status SELECTED besitzt (also angeklickt wurde).

Es ist auch möglich, daß ein Icon kein Selektionsbild besitzt. Dann enthalten der dritte und vierte Zeiger jeweils einen NULL-Pointer. Wenn ein Farbicon kein Selektionsbild besitzen sollte, wird es im selektierten Zustand »dunkel« gezeichnet, indem ein schwarzes Punktraster über das Icon gelegt wird.

Redraw-Probleme

Wenn sich die Grafikdaten des normalen Bilds und des Selektionsbilds eines Farbicons nicht genau decken, kann es zu Redraw-Fehlern kommen, da die Zeichenroutinen immer nur den Teil des Icons neu zeichnen, in dem wirklich Bilddaten stehen (Icons werden immer transparent gezeichnet). Man umgeht das Problem, indem man nicht nur das Icon neu zeichnet, sondern auch den Hintergrund. Statt

```
objc_draw (tree, ICON, MAX_DEPTH, desk.x, desk.y, desk.w, desk.h);
```

sollte man den Hintergrund mit dem Clipping des Icons zeichnen:

```
objc_offset (tree, ICON, &x, &y);
```

```
w = tree[ICON].ob_width;
```

```
h = tree[ICON].ob_height;
```

```
objc_draw (tree, ROOT, MAX_DEPTH, x, y, w, h);
```

ROOT steht für das ROOT-Objekt eines Baums (also 0), MAX_DEPTH entspricht der maximalen Zeichentiefe des AES (8).

Geräteabhängige Icons

Manchmal muß man wissen, welche der vielen Auflösungsstufen eines Icons eigentlich von den Zeichenroutinen verwendet wird. Die XRSRC-Routinen sorgen dafür, daß das geräteabhängige Icon immer als erste CICON-Struktur direkt nach der CICONBLK-Struktur folgt. Das bedeutet, daß der ›mainlist‹-

Parameter der CICONBLK-Struktur immer auf das geräteabhängige Icon zeigt, das von den Zeichenroutinen verwendet wird. Der ›next_res‹-Parameter der geräteabhängigen CICON-Struktur zeigt dann auf die erste geräteunabhängige CICON-Struktur.

Diese Information ist für alle interessant, die einen eigenen Desktop programmiert haben und darin dummy-OBJECT-Strukturen angelegt haben, die später auf ein Icon in der Resource zeigen sollen. Wenn es sich bei diesen dummy-Objekten bereits um Farbicons mit minimaler Größe handelt, muß man nur die Größenwerte der ICONBLK-Struktur anpassen und den ›mainlist‹-Parameter auf den ›mainlist‹-Parameter des gewünschten Icons »umbiegen«.

Kompatibilität der Farbicons

Interface unterstützt Farbicon-Ressourcen, die sich an das Farbicon-Format V2.1 vom 8. September 1992 halten. Das gilt nicht für alle Falcons, die noch mit einer TOS-Version vor diesem Datum ausgeliefert wurden, da das Farbicon-Format nach der ATARI-Messe 1992 noch geringfügig geändert wurde.

Interface erweitert das Farbicon-Format noch um eine RGB-Tabelle, damit Farbicons in High- oder True-Color von den XRSRC-Routinen in den korrekten Farben dargestellt werden können. Die Erweiterung bereitet keinerlei Probleme; sie wird vom ›rsrc_load‹-Befehl vom AES 4.0x (Falcon, MultiTOS) ignoriert.

C-Source Resourcen

Initialisierung der Resource

Da die Resource bereits weitestgehend von Interface initialisiert wurde, entfällt die ganze komplizierte Anpassung aller Zeiger. Man muß die Resource nur noch mit folgender Routine an die aktuelle Auflösung anpassen:

```
for (i = 0; i < NUM_OBS; i++) rsrc_obfix (&rs_object[i], 0);
```

NUM_OBS ist in der ›*.RH‹ Datei definiert, die zusätzlich zu der ›*.RSH‹ Datei abgespeichert wird.

Resourcen in CPX-Modulen

Da die Interface C-Sourcecodeausgabe bereits optimiert wurde, stimmt sie nicht mehr mit der Ausgabe des alten Atari RCS überein. Besondere Probleme gibt es bei der CPX-Programmierung, da die CPX Routinen nur das alte Format akzeptieren. Die Lösung besteht darin, daß man den CPX-Aufruf zur Initialisierung der CPX-Resource nicht benutzt, sondern statt dessen folgende Routine ausführt:

```
if (!xcpb->SkipRshFix)
{
    for (i = 0; i < NUM_OBS; i++) (*xcpb->rsh_obfix) (&rs_object[i], 0);
}
```

Die Variablennamen halten sich an die Schreibweise des CPX-Entwicklungspakets von ATARI. In dieser Routine werden nur alle Objektpositionen der Resource angepaßt. Alle weiteren Anpassungen entfallen, da die entsprechenden Variablen bereits von Interface initialisiert wurden.

Farbicons in C-Source-Resources

Natürlich muß man auch in C-Source-Resources die Farbicons in USERDEFS wandeln, damit man sie auf jeder TOS-Version darstellen kann. Auf den Interface-Disketten befinden sich Routinen, die diese Arbeit übernehmen.

Wenn Sie eine Resource mit Farbicons anpassen wollen, sollten Sie am Programmstart nach dem Öffnen Ihrer Workstation folgende Funktion aus dem Modul ›XRSRCFIX.C‹ aufrufen:

```
#ifdef rgb_palette
init_xsrc (vdi_handle, rs_object, NUM_OBS, NUM_CIB, rgb_palette);
#else
init_xsrc (vdi_handle, rs_object, NUM_OBS, NUM_CIB, NULL);
#endif
```

vdi_handle muß das Workstation-Handle enthalten, das die XRSRC-Routinen benutzen sollen.

Am Programmende muß noch die Funktion

```
term_xsrc (NUM_CIB);
```

aufgerufen werden, damit der für die Farbicons angeforderte Speicher wieder freigegeben wird. NUM_OBS und NUM_CIB sind in der ›*.RH‹-Datei definiert, die zusätzlich zu der ›*.RSH‹-Datei abgespeichert wird. Im Ordner CRSHTTEST befindet sich ein Programm, in dem diese Funktionen demonstriert werden.

Die init_xsrc-Routine paßt auch die Objektkoordinaten an, damit man nicht mehr ›rsrc_obfix‹ für jedes Objekt der Resource aufrufen muß.

Das AV-Protokoll

Das AV-Protokoll dient dem Austausch von Nachrichten zwischen Accessories und einer Shell. Entworfen wurde es für die Kommunikation zwischen Gemini und Accessories.

Interface unterstützt Teile des AV-Protokolls von Gemini und kann dementsprechend mit Accessories, die das Protokoll ebenfalls unterstützen »zusammenarbeiten«. Deshalb folgt jetzt eine kurze Auflistung der von Interface unterstützten AV-Kommandos:

AV_PROTOKOLL:

Interface meldet, daß es das Protokoll versteht.

AV_SENDKEY:

Accessories können nicht ausgewertete Tasten an Interface weitermelden.

AV_OPENWIND:

Ein Accessory kann Interface bitten, ein bestimmtes Laufwerksfenster zu öffnen (z.B. Treeview).

AV_ACCWINDOPEN und

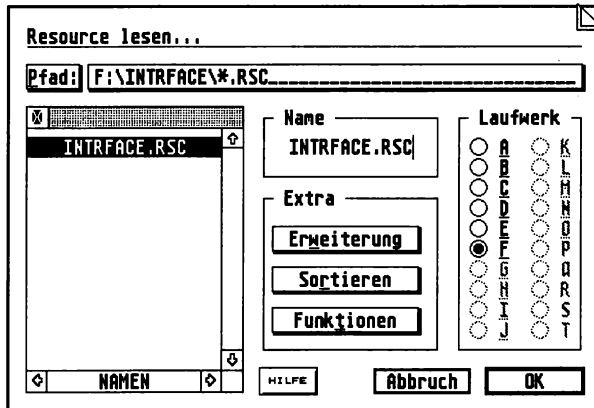
- p>AV_ACCWINDCLOSE :
- Accessories melden Interface, daß sie ein Fenster geöffnet haben. Dadurch kann Interface diese Fenster in der eigenen Liste verwalten und bei Control W auf das richtige Fenster umschalten.
- AV_STARTPROG:
- Ein Accessory übergibt Interface einen Pfad mit einem Dateinamen (z.B. FSearch). Wenn dieser Name die Endung ›RSC‹ besitzt, lädt Interface automatisch diese Resource-Datei.
- VA_START:
- Wie AV_STARTPROG.
(Da VA_START eigentlich nur für Accessories gedacht ist und Interface nicht als Accessory arbeitet, kann VA_START nur unter Multitasking-Betriebssystemen genutzt werden.)
- VA_START wird z.B. von Mag!X unterstützt. Wenn man im Mag!X-Desktop eine Resource-Datei öffnet, sendet Mag!X an Interface eine VA_START Nachricht und bittet Interface darin, die Resource-Datei zu laden. Voraussetzung dafür ist natürlich, daß man Interface als Anwendung für Resource-Dateien (›*.RSC‹) im Mag!X-Desktop angemeldet hat.
- Wenn man ein Recource-Datei Icon von Interface auf das Fenster eines Accessories schiebt, sendet Interface diesem Accessory die Nachricht VA_DRAGACCWIND mit dem Pfad des Icons, und »bittet« damit das Accessory, ein Fenster mit diesem Pfad zu öffnen (z.B. Treeview).

Die Dateiauswahl von Interface

Die Dateiauswahl von Interface kann mit dem Button GEM-Fileselektor im Einstellungen...-Dialog des Optionen-Menüs ein- oder ausgeschaltet werden. Ist der Button ausgewählt, benutzt Interface die Dateiauswahl des Betriebssystems bzw. eine andere Alternative Dateiauswahl wie z.B. »Selectric« oder der modulare FSEL Dateimanager von 'no'.

Die Arbeit am Computer hat - ob es einem gefällt oder nicht - auch etwas mit der Verwaltung von Daten, sprich Dateien zu tun. Wir haben uns große Mühe gegeben, Interface in dieser Hinsicht so komfortabel wie möglich zu machen und haben z.B. eine Dokumentenverwaltung integriert. Darüber hinaus haben wir eine neue Dateiauswahl entwickelt, die sehr einfach zu handhaben ist und eine ganze Reihe besonderer Funktionen aufweist.

Bei aktiver Auswahl melden sich alle Programme bei allen Dateifunktionen mit der hier abgebildeten Box.



Wie Sie der Abbildung entnehmen können, bietet die Dateiauswahl neben sämtlichen Elementen des GEM-Originals noch eine ganze Anzahl weiterer Möglichkeiten, auch wenn sich einige noch hinter Popup-Menüs verbergen.

Die Titelzeile der Dateiauswahl informiert Sie nicht nur darüber, welcher Dialog denn gerade aktiv ist, sondern auch, für welche Option er aufgerufen wurde. Unterhalb des Titels erscheint der aktuelle Zugriffspfad, den Sie zunächst einmal in der gewohnten Weise editieren können. Dazu gibt es aber noch einige Besonderheiten, auf die später eingegangen wird.

Das Dateifenster

Den (logischen) Mittelpunkt der Box bildet das Dateifenster, in dem die Dateien des ausgewählten Disketten- oder Festplattenlaufwerks angezeigt werden. Da nicht immer alle Dateien eines Laufwerks innerhalb des Fensters dargestellt werden können, befindet sich am rechten Rand ein Rollbalken, mit dem Sie durch die anzuzeigenden Namen scrollen können.

Beim Slider besteht übrigens schon der erste Unterschied gegenüber der GEM-Dateiauswahl. Interface merkt sich nämlich die Position des weißen Sliderfeldes beim Verlassen der Dateiauswahl, so daß beim nächsten Aufruf wieder die selben Dateien angezeigt werden. Das ist vor allem für Festplattenbesitzer, die viele Dateien in einem Ordner haben, interessant, da sie sich dann nicht jedesmal wieder durch alle Dateien »durchwühlen« müssen. Außerdem wird beim Verschieben des Sliders sofort der entsprechende Teil der Dateiliste angezeigt.

Am oberen Rand des Dateifensters finden Sie eine »graue« Leiste, die die ausgewählte Dateiendung anzeigt, sowie das Schließsymbol, das Sie auch von normalen Fenstern her kennen. Durch das Anklicken der Leiste erreichen Sie ein erneutes Einlesen des Dateiverzeichnisses vom aktuellen Laufwerk. Das ist notwendig, wenn Sie die Diskette gewechselt oder einen neuen Zugriffspfad über die Tastatur eingegeben haben, ohne abschließend Return zu drücken. Das Schließsymbol dient dem Schließen des angezeigten Ordners und der Anzeige der Dateien aus dem nächsthöheren Verzeichnis (sofern vorhanden).

Kommen wir wieder zu den Namen, die im Fenster angezeigt werden. Hier gibt es zwei Gruppen, zum einen die Dateien und zum anderen die Ordner, wobei letztere durch ein kleines Symbol vor ihrem Namen gekennzeichnet sind. Jede Gruppe für sich ist alphabetisch geordnet.

Dateien auswählen

Da es sich um eine DateiAUSWAHLbox handelt, können die einzelnen Einträge durch Anklicken mit der Maus ausgewählt werden. Das Anklicken eines Ordners bewirkt, daß er geöffnet und sein Inhalt an Dateien und eventuell weiteren Ordnern im Dateifenster angezeigt wird. Das Schließen eines Ordners geschieht mittels des Symbols am oberen Fensterrand, in der oben beschriebenen Form.

Sobald Sie dagegen einen Dateinamen anklicken, wird dieser schwarz hinterlegt und zusätzlich in das Textfeld mit der Überschrift Dateiname, rechts neben dem Fenster, übernommen. Dieses Feld zeigt immer den Namen der Datei an, die beim Auswählen des Ok-Buttons geladen oder gespeichert würde. Sie können also, wenn Sie genau wissen, welche Datei Sie laden wollen, einfach deren Namen in dieses Feld eintragen und dann Ok anklicken (oder Return drücken, was die gleiche Wirkung hat).

Eine andere Möglichkeit zur schnellen Auswahl besteht darin, den im Fenster sichtbaren Namen einer Datei mit einem Doppelklick auszuwählen. Dadurch wird der Name übernommen und gleich darauf die Dateiauswahl automatisch mit Ok verlassen.

Das Betriebssystem der ATARI Computer verwaltet von jeder abgespeicherten Datei nicht nur den Namen, sondern auch ihre Länge, ihr Erstellungs- bzw. Änderungsdatum und die zugehörige Uhrzeit. Diese Daten werden zwar von der Standard-Auswahlbox bei der Anzeige völlig ignoriert, bei Interface sind sie aber anzeigbar. Zu diesem Zweck befindet sich am unteren Rand des Fensters ein Textfeld mit Pfeilen an beiden Seiten. Beim Aufruf der Dateiauswahl steht hier immer Namen, was nichts anderes bedeutet, als daß gerade die Dateinamen angezeigt sind. Wenn Sie den rechten Pfeil anklicken, erscheinen nacheinander die Einträge Länge, Zeit und Datum, mit dem linken Pfeil kehrt sich die Reihenfolge um. Außerdem werden natürlich statt der Dateinamen jeweils die entsprechenden Informationen angezeigt.

Zugriffspfade und -masken

Um auf Inhaltsverzeichnisse und Dateien zugreifen zu können, braucht das Betriebssystem einen sogenannten Zugriffspfad. Dieser besteht aus der Angabe des Laufwerks, auf dem sich die Dateien befinden, sowie eventuell aus den Namen der Ordner, in denen gesucht werden soll, jeweils durch das Zeichen ›\‹ getrennt. Hinter dem Laufwerksbuchstaben muß zusätzlich noch ein Doppelpunkt stehen. Ein solcher Pfad wäre z.B.:

›G:\Interface\DOKUMENT\‹

Diese Pfadangabe besagt, daß auf Laufwerk ›G:‹ der Inhalt des Ordners DOKUMENT angezeigt werden soll, der sich wiederum im Ordner Interface befindet.

Normalerweise ändern Sie den Pfad, indem Sie Ordner anklicken oder (via Schließsymbol) wieder schließen. Wie Sie aber an der Abbildung der Dateiauswahlbox sehen können, wird der Zugriffspfad oben in der Box in einem Eingabefeld angezeigt, so daß Sie ihn auch über die Tastatur editieren können. Sollte Ihre Eingabe mehr Zeichen benötigen, als momentan sichtbar, so wird das Eingabefeld während der Eingabe nach links wegschrollen. Wenn Sie einen neuen Pfad eingegeben haben, müssen Sie noch das entsprechende Verzeichnis einlesen lassen, indem Sie die Titelleiste des Dateifensters anklicken (s.o.), oder Return drücken.

In den meisten Fällen ist die direkte Eingabe des Pfades aber gar nicht notwendig, da bei Interface alle Elemente des Zugriffspfades auch mit der Maus einstellbar sind. So auch das aktuelle Laufwerk, dessen Daten angezeigt werden sollen. Dafür befinden sich am rechten Rand der Dateiauswahl Radio Buttons für die beim Betriebssystem angemeldeten Laufwerke, die durch einfaches Anklicken aktiviert werden. Daraufhin wird der Laufwerksbuchstabe in den Zugriffspfad übernommen und das neue Verzeichnis automatisch eingelesen. Eine Besonderheit dabei ist, daß die Dateiaus-

wahl von Interface sich die Pfade aller Laufwerke merkt und nach dem Umschalten von z.B. G: auf A: und zurück auf G: wieder den Inhalt des dort zuletzt eingestellten Ordners anzeigt.

Ein Element des Zugriffspfades wurde bisher nicht erwähnt. Wenn Sie sich die Abbildung der Auswahlbox genau angesehen haben, ist Ihnen vielleicht aufgefallen, daß am Ende des dort angezeigten Pfades die Zeichenfolge ›*.CYP‹ steht. Dabei handelt es sich um eine Zugriffsmaske (hört sich komplizierter an, als es ist), die festlegt, welche Dateinamen im Fenster sichtbar sind. Die Ordnernamen sind davon grundsätzlich nicht betroffen, das heißt, sie werden immer angezeigt.

Wie Sie wahrscheinlich wissen, setzen sich Dateinamen immer aus zwei Teilen zusammen. Den ersten Teil bildet mit bis zu acht Buchstaben der eigentliche Name der Datei. Darauf folgt, durch einen Punkt getrennt, eine maximal drei Zeichen lange Namenserverweiterung, auch Dateikennung genannt. Diese Kennung wird dafür benutzt, Dateien in bestimmte Kategorien einzuteilen. Im obigen Beispiel sollen demzufolge Dateien mit der Endung ›CYP‹ angezeigt werden, also Dokumente im Interface-Format.

Was aber bedeutet der Stern vor dem Punkt? Das ist ein Platzhalter, der für einen beliebigen Namen steht. Mit der Zugriffsmaske ›*.CYP‹ werden also alle Dateien mit der Endung ›CYP‹ angezeigt, unabhängig davon, welchen Namen sie haben. Ein anderes Beispiel: die Maske ›E*.CYP‹ bewirkt, daß von allen Dateien mit der Endung ›CYP‹ nur diejenigen angezeigt werden, die mit ›E‹ beginnen.

Es gibt noch einen weiteren Platzhalter, nämlich das Fragezeichen ›?‹, das stellvertretend für beliebige Buchstaben eingesetzt werden kann. So kann mit der Maske ›*.P?C‹ erreicht werden, daß ausschließlich (Grafik-) Dateien mit den Endungen ›PAC‹, ›PIC‹, usw. angezeigt werden. Ein letztes (extremes) Beispiel: mit der Angabe ›C????.AC?‹ würde dafür gesorgt, daß nur Dateien angezeigt werden, deren fünf Zeichen langer Name mit ›C‹ beginnt und die eine Kennung haben, die drei Zeichen umfaßt und mit ›AC‹ beginnt, wie ›CLICK.ACX‹ oder ›CLOCK.ACC‹. Neu ist, daß Sie mehr als eine Erweiterung angeben können, und so mit ›*.CYP;*.TXT‹ sowohl Interface-Dokumente als auch Textdateien zur Auswahl stehen. Die gewünschten Erweiterungen müssen durch Kommata voneinander getrennt werden. Mit der Maus können ebenfalls mehrere Extensions ausgewählt werden, wenn bei der Auswahl aus dem Popup die Shift-Taste gedrückt wird.

Beim Aufruf einer Dateifunktion wird immer eine bestimmte Zugriffsmaske vorgegeben. Trotzdem kann es nötig sein, eine andere Maske einzugeben, z.B. wenn Sie die Anzeige auf bestimmte Namen einschränken oder bei bestimmten Funktionen auf ein anderes Dateiformat - mit einer anderen Kennung - zugreifen wollen. Die

Eingabe einer neuen Zugriffsmaske oder einer Kennung kann über das Pfad-Textfeld geschehen und wird nach Anklicken des Auswahlfensters aktiviert. Interface bietet hierfür aber noch einige Erleichterungen, die weiter unten erläutert werden.

Als aufmerksamer Betrachter und Leser werden Sie bemerkt haben, daß noch kein Wort über die Darstellung des Eingabefeldes für die Zugriffspfade verloren wurde. Offensichtlich handelt es sich doch um Popup-Menüs. Stimmt. Interface merkt sich bis zu acht beliebige Zugriffspfade, die aus einem Popup-Menü gewählt werden können, nachdem Sie das Feld Pfad: angeklickt haben. Das entsprechende Verzeichnis wird sofort eingelesen und angezeigt.

Beim ersten Aufruf des Popup-Menüs werden Sie feststellen, daß noch keinerlei Eintrag vorgenommen wurde. Dies bleibt Ihnen überlassen, schließlich sollen Sie hier Ihre »bevorzugten« Pfade eintragen. Dazu rufen Sie mit einem Doppelklick auf das Pfad-Textfeld das Popup-Menü auf, das - weil Sie noch keinen Pfad eingetragen haben - absolut leer ist. Sie können jetzt eine beliebige Zeile durch Anklicken auswählen. Daraufhin wird der aktuelle Pfad an die gewählte Position im Popup übertragen und kann von diesem Zeitpunkt an dort ausgewählt werden. Zum Ändern eines Eintrages gehen Sie ebenso vor. Unter dem Feld zur Namenseingabe befindet sich das eigentliche Herzstück der Dateiauswahl. Was hier mit Spezielles betitelt ist, werden Sie in Zukunft nicht mehr missen wollen.

Das Popup-Menü Erweiterung stellt eine Auswahl der wichtigsten Dateikennungen zur Verfügung. Die Wahl einer der Endungen fügt diese als Endung an die Pfadangabe an und sortiert die Dateiauswahl sofort neu, so daß Sie sich in den meisten Fällen den »Umweg« über die Texteingabe sparen können.

Das Popup-Menü Sortieren bestimmt die Reihenfolge der Einträge in der Dateiauswahl. Während die Optionen Name, Datum und Länge wohl kaum einer Erläuterung bedürfen, zwei Sätze zu Art und unsortiert. Bei der Sortierung nach der Dateart werden die Dateien alphabetisch nach ihrer Endung sortiert. Dateien gleicher Endung werden unsortiert aufgelistet, also so, wie sie auf der Diskette oder Festplatte in die Verwaltungsliste eingetragen wurden (ziemlich unsortiert).

Spezielle Funktionen

Damit kennen Sie alle Funktionen, die für die Auswahl von Dateien von Bedeutung sind. Die Dateiauswahl von Interface bietet jedoch im Popup-Menü Funktionen ein paar zusätzliche Möglichkeiten, die den Umgang mit Dateien sehr erleichtern können.

Dateinamen ändern

Mit der Funktion Namen ändern aus dem Popup-Menü Funktionen können Sie den Namen einer Datei ändern. Dafür muß, vor Aufruf der Funktion im Popup, die gewünschte Datei angeklickt werden, so daß der Dateiname schwarz hinterlegt ist. Daraufhin erscheint die abgebildete Dialogbox, die den bisherigen Namen der Datei als editierbare Vorgabe beinhaltet. Verlassen Sie den Dialog mit Ok, wird der Name entsprechend geändert und die Dateiliste neu sortiert. Sollte der gewählte Name bereits vorhanden sein, erfolgt eine entsprechende Meldung.

Dateien löschen

Wenn Sie eine Datei nicht mehr benötigen, können Sie sie mit der Funktion löschen aus dem Popup-Menü Funktionen aus dem Dateiverzeichnis entfernen. Dafür muß, vor Aufruf der Funktion im Popup, die gewünschte Datei angeklickt werden, so daß der Dateiname schwarz hinterlegt ist. Es erscheint dann eine Sicherheitsabfrage, in der Sie bestätigen können, daß Sie die Datei wirklich löschen wollen. Bestätigen Sie diese Abfrage, wird die Datei unwiederbringlich gelöscht (oder kennen Sie ein Programm, das gelöschte Dateien zuverlässig wieder herstellt?)!

Dateien kopieren

Eine sehr nützliche Funktion, die Sie in den wenigsten anderen Programmen finden werden, ist das Kopieren von Dateien. Das geschieht durch die Auswahl einer Datei im Dateifenster und die anschließende Wahl der Funktion kopieren im Popup-Menü Funktionen. Daraufhin erscheinen mehrere Dialogboxen, die Sie Schritt für Schritt über die nötigen Aktionen informieren.

Sollte eine Datei mit dem gleichen Namen auf dem Zielpfad bereits vorhanden sein, werden Sie gefragt, ob Sie diese Datei überschreiben wollen, wodurch deren bisheriger Inhalt verlorengeht. Achtung! Obwohl das Überschreiben durchaus sinnvoll sein kann, sollten Sie genau prüfen, ob nicht wichtige Daten gelöscht werden!

Ordner anlegen

Mit der Funktion neuer Ordner aus dem Popup-Menü Funktionen können Sie im gerade angezeigten Verzeichnis einen neuen Ordner anlegen. Dazu erscheint der bereits oben abgebildete Dialog zur Namenseingabe. Verlassen Sie den Dialog mit Ok, wird der Ordner angelegt und die Dateiliste neu sortiert. Sollte der gewählte Name bereits für einen Ordner vergeben sein, erfolgt eine entsprechende Meldung.

Disketten formatieren

Auch das Formatieren von Disketten ist aus Interface heraus möglich. Wählen Sie dazu die Funktion formatieren aus dem Popup-Menü Funktionen.

In der Gruppe Laufwerk wählen Sie das Diskettenlaufwerk aus, in dem die zu formatierende Diskette liegt. Der Button doppelseitig formatieren gibt an, ob die Diskette mit ca. 360 oder ca. 720 KB formatiert werden soll. Mit Ok starten Sie die Option.

Laufwerksinformationen anzeigen

Last not least können Sie sich mit der Funktion Disk-Info aus dem Popup-Menü Funktionen einige Daten zum aktuellen Laufwerk und dem aktuellen Pfad anzeigen lassen.

Damit Sie vor allem die Pfadeinstellungen bei jedem Programmstart vorfinden, sollten Sie unter Optionen: Parameter sichern... die Parameter, zu denen auch die Pfade gehören, speichern.

Tastatursteuerung

Und wie die Dialoge in Interface ist auch die Dateiauswahl vollständig über die Tastatur steuerbar. Zur besseren Tastatursteuerung wurde der ›Control-Modus‹ eingeführt. Bei gleichzeitig gedrückter Control-Taste bekommen einige Tasten eine neue/andere Funktion:

Cursor hoch/runterwählt	den ersten Eintrag in der Dateiliste aus und ermöglicht danach die Auswahl aller übrigen Ordner und Dateien. Bei Bedarf wird die Dateiliste automatisch gescrollt.
Shift Cursor hoch/runter	scrollt die Dateiliste »seitenweise«
Shift Cursor links/rechts	wechselt die Darstellung in der Dateiliste von Namen, Länge, Zeit und Datum zur jeweils nächsten Option.
Clr-Home	zeigt den Anfang der Dateiliste
Shift Clr-Home	zeigt das Ende der Dateiliste
Return	wählt eine Datei aus oder öffnet (falls ausgewählt) einen Ordner.
Backspace	wechselt zum übergeordneten Ordner.
Leerstaste	Liest das Directory neu ein
Buchstaben	Wählt eine Datei direkt aus. Wird z.B. der Buchstabe E gedrückt, so wird die erste Datei ausgewählt, die mit I beginnt. Die nächste Zeicheneingabe schränkt die Auswahl weiter ein, z.B. auf IN... Die aktuelle Maske wird im Kopf der Dateiliste angezeigt.
Shift Buchstaben	Wirkt wie die Eingabe von Buchstaben, aber auf die Erweiterungen.
Esc	Löscht die Auswahl durch Buchstabeneingabe.
Tab	Schaltet dauerhaft in den Control-Modus. Oben rechts im Dialog wird zur Kontrolle das Wort ›CONTROL‹ angezeigt. Ein weiterer Druck auf Tab schaltet den Control-Modus wieder aus.

Index

I...

3D-Dialoge 59
64 KB 124

A...

Abbrechen 53
Accessory-Pipeline 84
- Schnittstelle 84
Alert 7, 42
Alerts 111
Alternate 12, 20
al_token 112
Animierte Icons 49
Anzeige der Farbicons 126
ascii_head 117
ATARI RCS 7
Aufdecken 40, 58
Auflösung entfernen 69
Auflösung wählen 69
Aufziehen 7
Ausrichten eines Icons 48
Ausrufungszeichen 42
Ausschneiden 48, 54, 67
AV-Protokoll 130

B...

Basic 78
Bäume 18, 22, 26, 54
Bearbeiten 30
Beispiel 85
Beispielprogramm 94, 118
Boxtext 92
Boxtitel 93

C...

C-Source 129
Checkbox 8, 13, 34, 88, 91
CHECKED 37
cicon_blk 127
cicon_data 127
Circle-Button 93
Clipboard 48
close_dial 100
Control 12, 20
CPX-Module 129
CROSSED 37

D...

Datei n 57
Dateiauswahl 132
Dateien auswählen 133
Dateien kopieren 137
Dateien löschen 137
Dateiextensions 78
Dateifenster 133
Dateinamen ändern 137
DEF 72f
DEFAULT 36
Deinitialisierung der MyDials 94
Deinitialisierung von NKCC 94
Desktop 9
DFN 72f
Dialog 6, 22, 30, 74
Dialogboxen 12
Dialogroutinen 97
dial_alert 112
dial_center 100
dial_do 102f
dial_draw 101
dial_end 102
dial_exit 96
dial_fix 97

dial_init 96
dial_move 102
dial_slinit 102
dial_start 101
Digital Research 7
DISABLED 37
Disketten formatieren 137
Doppelklick 7
do_alert 111
DRAW 3D 37

E...

EDITABLE 36
Editier-Funktionen 15
Editieren 38, 57
Einfügen 54, 67
Einrasten 40, 57
Einstellungen 58
Ende 53
Entfernen 41, 58
Erweiterter Typ 31, 44
Esc 41
EXIT 36
Exit-Button 92
Extended 36
Extended Flags 31, 44
Externe Objektanzeige 59
EXTOBFIX 81

F...

Farbdurchlauf 67
Farben und Hintergrund 32
Farbicon 43f, 46, 49f, 68, 125
Farbicon-Format/Struktur 126f
Fenster 17
find_next 109
find_prev 109
Flags setzen 55

FlyDial 8
FlyDial-Ecke 91
Formular 22
Formularbaum 22, 26, 73
form_Button 114
form_Keybd 114
Fragezeichen 42
Freie Strings 7
Funktionstasten 10

G...

GEM Fileselector 58
Geräteabhängige Icons 128
getcookie 113
get_icon 112
get_idx 114
get_Keybd 103
get_obspec 113
get_sysfnt 118
Grafik ausschneiden 65
Grafik laden 65
Grafik schließen 65
Größenanpassung 63
Grow/Shrinkboxen 59
G_BOX 34
G_BOXCHAR 34
G_BOXTEXT 34
G_BUTTON 34
G_CICON 34, 127
G_FBOXTEXT 34
G_FTEXT 34
G_IBOX 34
G_ICON 34
G_IMAGE 34
G_STRING 34
G_TEXT 34
G_TITLE 34
G_USERDEF 34f, 81f

H...

Help-Button 93
 HIDETREE 40
 Hilfe 53
 Hilfsroutinen 113
 HndlDial 98
 Horizontal spiegeln 67
 HRD 72, 74

I...

IBOX 24
 ICO-Format 75
 Icon 8, 42f, 45, 65ff, 71, 75
 IconEdi 75
 Iconeditor 45, 65
 Iconvorschau 67
 Image 7, 43, 45
 - sichern 65
 Index 141
 Info 52
 Initialisierung 96
 - der MyDials 94
 - der Programm-Resource 94
 - von NKCC 94
 Innen 68
 installieren 5

K...

Kompatibilität der Farbicons 128
 kopieren 37, 48f, 54, 67

L...

Label-Prefix 61
 Lange Überschriften 58
 Lattice C 78
 Laufwerksinformat. anzeigen 138
 Letzte Fassung 51
 Lieferumfang 5

Löschen 26, 38, 41, 68

M...

Maske 32f, 43, 46, 68
 Maskenfeld 32f
 Mausfunktionen 16
 Mausklick 7
 MausNet 4, 84
 Mehrzeilige Eingabefelder 93, 106
 Meisiek 4, 84
 Menü 11, 17f
 Menübaum 18
 Menüeinträge 28
 Menütitel 27, 117
 Modem 4, 84
 Modula-2 79
 Muster 32
 MyDials 8, 88
 my menu key 117

N...

Name 30, 43, 73f
 Neu 51
 Neuer Typ 58
 NKCC 89, 94
 normkey 114
 no | where 4

O...

Objekt-Popup 61
 Objektbaum testen 56
 Objektdatenanzeige 60
 Objekte 19, 22, 24, 26, 35, 54
 Objektflag 36, 38
 Objektnamen 59
 Objektnummern fest 63
 Objektstatus 37
 Objekttyp 31, 34, 43

Objekttyp ändern 41
obj_edit 116
ob_spec 126
Öffnen 51
Öffnen von Dateien 11
open_dial 99
Ordner anlegen 137
OUTLINED 37

P...

Palette 69f
PANEL 7
Papierkorb 26
Parameter 61
Parameter sichern 63
parent 113
Platzhalter 32f
Plazieren 40, 58
Popup-Button 92
Popup-Menü 38
Popup-Routinen 110
popup_menu 110
popup_select 110
Preview 67
Profibuch 2
Pulldown Leerzeichen 60
Pulldown-Menüs 6

Q...

Quellcode laden 66
Quellcode sichern 66

R...

Radiobutton 7, 91
Raster 62
Raster benutzen 63
RBUTTON 36
Redraw-Probleme 128

Registrierkarte 3
Resource-Formate 72
Resourcevergleich 56
RSD 72f

S...

Schließen 51
scrap_clear 116
SELECTABLE 36
SELECTED 37, 127
Selektieren 8
Selektieren mehrerer Objekte 38
set_Keybd 104
set_obspec 113
SHADOWED 37
Shift 12, 20
shortcut 115
Sicherheitskopie 5
Sichern 51
Sichern als 52
Sizebox 60
Slider 94
Sortieren 38f, 57
Speichern 26, 71
Speicherverwaltung 17
Status 31, 37, 44
Stopschild 42
String 92
Suchen 55

T...

Tastaturanpassung 77
Tastaturauswertung 103
Tastaturfunktionen 20
Tastaturkürzel 59
Tastatursteuerung 139
Testen 21, 26, 83
Text 92

TOUCHEXIT 36

U...

Umriß 68

Undo 41, 45

Undo-Button 92f

Unterstrich 93

USERBLK-Struktur 35, 82, 97, 113, 126

V...

Vergrößern 24

Verkleinern 24

Verschiebe-Objekt 91

verschieben 37

Verstecken 40, 57

Vertikal spiegeln 67

Verwerfen 51

Vom Anfänger zum GEM-Profi 2

W...

Wechseln 57

WHITEBAK 37

Windows-Icon 66

X...

XIMG 48, 65f, 69

Z...

Zeichenfläche verändern 48

Zeichenfunktionen 47

Zeichnen 46

zentrieren 40, 44, 48, 57